C++

```
__tile_global__ void gather_safe(int* __restrict__ arr, int* __restrict__ out,
↪ int N) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;
    using i32x8 = ct::tile<int, ct::shape<8>>;

    arr = ct::assume_aligned(arr, 16_ic);
    out = ct::assume_aligned(out, 16_ic);

    int bx       = ct::bid().x;
    auto offsets = 8 * bx + ct::iota<i32x8>();   // element-level offsets,
↪one per lane
    auto mask    = offsets < N;                  // boolean tile: true where
↪the offset is in-bounds

    auto ptrs = arr + offsets;                   // tile of pointers, one per
↪offset
    auto tile = ct::load_masked(ptrs, mask, 0);  // masked lanes get the pad
↪value 0
    ct::store_masked(out + offsets, tile, mask); // masked lanes are skipped
↪on the store
}
```

2.4.7. Control Flow

From the programmer's perspective, a tile kernel follows a single control flow path per block. Scalar values in conditions and loop bounds drive the control flow, while tile operations within the body are distributed across hardware threads by the compiler.

Not every control-flow construct is supported. For example, returning from inside a loop is not allowed in tile code. See the API reference for each language (CUDA Tile C++ general principles, cuTile Python control flow) for the full list of restrictions.

2.4.7.1 Loops

A common pattern is to iterate over tiles from an array, processing each in turn.

In C++, `ct::irange` is a forward range representing an increasing sequence of integers from a lower bound up to but excluding an upper bound, separated by an optional step. Using `ct::irange` provides the compiler with structured information about the iteration bounds, which may be used to better optimize the generated code. For the optimization to apply, the loop variable must be bound through a range-for expression over `ct::irange`.

In Python, the built-in `range()`, `for`, `while`, and nested loops are all supported in tile code.

The step argument must be strictly positive; negative-step ranges are not supported.

The following single-block kernels sum all tiles of a 1D array:

C++

```
__tile_global__ void tile_sum(float* __restrict__ arr, float* __restrict__
↪out, int num_tiles) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;
    using f32x8 = ct::tile<float, ct::shape<8>>;

    arr = ct::assume_aligned(arr, 16_ic);
    out = ct::assume_aligned(out, 16_ic);

    auto inView  = ct::partition_view{ct::tensor_span{arr, ct::extents{8 *
↪num_tiles}},
                                      ct::shape{8_ic}};
    auto outView = ct::partition_view{ct::tensor_span{out, ct::extents{8_ic}},
                                      ct::shape{8_ic}};

    auto acc = ct::full<f32x8>(0.0f);
    // range-for over ct::irange gives the compiler structured iteration
↪bounds.
    for (auto k : ct::irange(0, num_tiles)) {
        auto tile = inView.load(k);
        acc = acc + tile;                                  // accumulate the k-
↪th tile into acc
    }
    outView.store(acc, 0);                                 // write the final
↪result as the 0-th tile of out
}
```

Python

```
@ct.kernel
def tile_sum(arr, out, TILE: ct.Constant[int], N_TILES: ct.Constant[int]):
    # Intended grid: (1,) -- a single block sums all tiles of arr.
    acc = ct.zeros((TILE,), dtype=ct.float32)
    for k in range(N_TILES):                               # range() works
↪natively in tile code
        tile = ct.load(arr, index=(k,), shape=(TILE,))
        acc = acc + tile                                   # accumulate the k-th
↪tile into acc
    ct.store(out, index=(0,), tile=acc)                    # write the final
↪result as the 0-th tile of out
```

2.4.7.2 Conditionals

Standard `if`/`else` conditionals work normally. Because each block follows a single control flow path, the considerations for *branch divergence within a warp* do not apply to tile kernels.

C++

```
__tile_global__ void conditional_load(float* __restrict__ arr, float* __
↪restrict__ out, int N) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;
    using f32x8 = ct::tile<float, ct::shape<8>>;

    arr = ct::assume_aligned(arr, 16_ic);
    out = ct::assume_aligned(out, 16_ic);

    auto inView  = ct::partition_view{ct::tensor_span{arr, ct::extents{N}},
↪ct::shape{8_ic}};
    auto outView = ct::partition_view{ct::tensor_span{out, ct::extents{N}},
↪ct::shape{8_ic}};

    int bx   = ct::bid().x;
    int nb_x = ct::num_blocks().x;

    auto tile = ct::full<f32x8>(0.0f);    // default for the last-block branch
    // Scalar condition -> one control-flow path per block; no divergence to
↪reason about.
    if (bx < nb_x - 1) {
        tile = inView.load(bx);           // all blocks except the last
    }
    outView.store_masked(tile, bx);       // masked to handle a potentially
↪partial final tile
}
```

Python

```
@ct.kernel
def conditional_load(arr, out, TILE: ct.Constant[int]):
    bid = ct.bid(0)
    # Scalar condition -> one control-flow path per block; no divergence to
↪reason about.
    if bid < ct.num_blocks(0) - 1:
        tile = ct.load(arr, index=(bid,), shape=(TILE,))    # all blocks
↪except the last
    else:
        tile = ct.zeros((TILE,), dtype=ct.float32)          # last block:
↪emit zeros
    ct.store(out, index=(bid,), tile=tile)
```

2.4.8. Element-wise Arithmetic and Broadcasting

Tiles support standard element-wise arithmetic. When two operands have compatible but different shapes, the smaller is broadcast to match before the operation is performed.

2.4.8.1 Broadcasting

Broadcasting follows NumPy semantics: scalars are duplicated across the tile, singleton dimensions (length 1) are stretched to match the corresponding dimension of the other operand, and a lower-rank operand is aligned to the trailing dimensions of the higher-rank operand by treating the missing leading dimensions as singletons. If two corresponding dimensions are both non-singleton and unequal, the operation is ill-formed.

The example below exercises both singleton stretching and rank promotion in a single addition: a rank-2 tile of shape 8x2 is rank-promoted to 1x8x2, then broadcast with a rank-3 tile of shape 4x1x2 to the common shape 4x8x2.

C++

```
auto x = ct::iota<ct::tile<int, ct::shape<8, 2>>>();      // 8x2   (rank 2)
auto y = ct::iota<ct::tile<int, ct::shape<4, 1, 2>>>();   // 4x1x2 (rank 3)
auto z = x + y;                                           // x promoted to
↪1x8x2, then broadcasts to 4x8x2
```

Python

```
x = ct.full((8, 2),    3, dtype=ct.int32)   # 8x2   (rank 2)
y = ct.full((4, 1, 2), 5, dtype=ct.int32)   # 4x1x2 (rank 3)
z = x + y                                    # x promoted to 1x8x2, then
↪broadcasts to 4x8x2
```

2.4.8.2 Arithmetic Operators

All supported arithmetic operators apply element-wise to tiles and produce a new tile of the broadcast shape. A scalar combined with a tile is broadcast across every element. When operand types differ, the type that preserves more information is favored:

- **Tile combined with tile**: the result is a tile of the type with the greater precision or range. For example:
  - `int + float` yields `float`
  - `int16 + int32` yields `int32`
- **Scalar combined with tile**: when the scalar's type is exactly representable in the tile's element type (e.g., the integer literal 2 combined with an `int` tile, or `2.0f` combined with a `float` tile), the operation proceeds in the tile's element type. When the scalar would have to narrow to fit the tile's element type (e.g., the literal `2.5` combined with an `int` tile), the two languages differ:
  - Python promotes the result to a type that can hold both
  - C++ rejects the expression as ill-formed

The snippets below illustrate the divergent scalar-tile case:

C++

```
using i32x8 = ct::tile<int, ct::shape<8>>;
i32x8 x = ct::full<i32x8>(3);

x + 2;       // OK - int literal matches int tile element type
x + 2.5;     // ill-formed - 2.5 would narrow to int
```

Python

```
x = ct.full((8,), 3, dtype=ct.int32)

x + 2          # int32 - int literal matches int32 tile dtype
x + 2.5        # float32 - result promoted to hold both
```

In practice, write scalar literals in the tile's element type when you can and convert explicitly when you want a different precision. The same rules apply inside a kernel when the operands are loaded tiles:

C++

```
__tile_global__ void elementwise(float* __restrict__ a, float* __restrict__ b,
↪ float* __restrict__ out, int N) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;

    a   = ct::assume_aligned(a,   16_ic);
    b   = ct::assume_aligned(b,   16_ic);
    out = ct::assume_aligned(out, 16_ic);

    auto aView = ct::partition_view{ct::tensor_span{a,   ct::extents{N}},
↪ct::shape{8_ic}};
    auto bView = ct::partition_view{ct::tensor_span{b,   ct::extents{N}},
↪ct::shape{8_ic}};
    auto cView = ct::partition_view{ct::tensor_span{out, ct::extents{N}},
↪ct::shape{8_ic}};

    int  bx = ct::bid().x;
    auto x  = aView.load(bx);
    auto y  = bView.load(bx);
    // 2.0f matches the float tiles' element type, so no narrowing conversion
↪is required.
    // The scalar is broadcast across every element; + then runs elementwise.
    auto z  = 2.0f * x + y;
    cView.store(z, bx);
}
```

Python

```
@ct.kernel
def elementwise(a, b, c, TILE: ct.Constant[int]):
    bid = ct.bid(0)
    x = ct.load(a, index=(bid,), shape=(TILE,))
```

(continues on next page)

(continued from previous page)

```
    y = ct.load(b, index=(bid,), shape=(TILE,))
    # 2.0 is a loosely typed float constant; with float tiles, the result
↪stays float.
    # Scalars are broadcast across every element of the tile, then + runs
↪elementwise.
    z = 2.0 * x + y
    ct.store(c, index=(bid,), tile=z)
```

When you need explicit control over rounding mode or subnormal handling, *mathematical functions* (for example, `ct.add`, `ct::add`) that accept those as parameters are provided by the CUDA Tile APIs.

2.4.9. Tile Primitives

Factory functions (*Creating Tiles*), loads and stores (*Tile-Space Loads and Stores*), and element-wise arithmetic (*Element-wise Arithmetic and Broadcasting*) are all *tile primitives*, that is, operations that are part of the language. The programmer writes them at tile granularity and the compiler maps them to hardware, including tensor cores where available. This section covers other primitives that are available in CUDA tile.

2.4.9.1 Matrix Multiply

Matrix multiplication of two tiles is a fundamental operation to implementing a matrix multiplication between two arrays. CUDA Tile provides two forms of matrix multiplication between tiles: a pure matrix multiply (matmul), `a @ b`, and a matrix multiply-accumulate (mma), `a @ b + acc`. In mma, the accumulator carries partial products from one K-tile to the next. This is helpful in the inner loop of a tiled matrix multiplication. Both `matmul` and `mma` support 2D matrix multiplication and 3D batched multiplies as well as mixing the data type (precision) of operands and accumulator. The rank and element type constraints are documented in the API reference for the operations (CUDA Tile C++ matrix multiplication, cuTile Python matmul).

A common pattern, used in the kernels below, is to accumulate in FP32 regardless of input precision and cast to the output element type on store. In Python this is `ct.mma(a, b, acc)` with an FP32-typed `acc`. In C++ it is `ct::mma(a, b, acc)` with an explicit FP32 accumulator type. The K-loop iterates `ceil(K / tk)` times so the right edge of A and the bottom edge of B are covered; partial K-tiles are zero-padded on load (`PaddingMode.ZERO` in Python, `.load_masked()` in C++), and partial M/N edge tiles on the C side are handled by store-side OOB-discard (`ct.store` in Python, `.store_masked()` in C++).

C++

```
__tile_global__ void gemm(const __half* __restrict__ A, const __half* __
↪restrict__ B, float* __restrict__ C,
                          std::size_t M, std::size_t K, std::size_t N) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;
    using f32_acc = ct::tile<float, ct::shape<32, 32>>;

    A = ct::assume_aligned(A, 16_ic);
    B = ct::assume_aligned(B, 16_ic);
```

(continues on next page)

(continued from previous page)

```
    C = ct::assume_aligned(C, 16_ic);

    constexpr auto tm = 32_ic;
    constexpr auto tn = 32_ic;
    constexpr auto tk = 16_ic;

    auto aView = ct::partition_view{ct::tensor_span{A, ct::extents{M, K}},
↪ct::shape{tm, tk}};
    auto bView = ct::partition_view{ct::tensor_span{B, ct::extents{K, N}},
↪ct::shape{tk, tn}};
    auto cView = ct::partition_view{ct::tensor_span{C, ct::extents{M, N}},
↪ct::shape{tm, tn}};

    auto [bx, by, bz] = ct::bid();
    auto acc = ct::full<f32_acc>(0.0f);                // FP32 accumulator

    std::size_t num_k = (K + tk - 1) / tk;
    for (auto k : ct::irange(std::size_t{0}, num_k)) {
        acc = ct::mma(aView.load_masked(bx, k),        // zero-pad partial K-
↪tile
                      bView.load_masked(k, by),
                      acc);                            // acc += a @ b
    }
    cView.store_masked(acc, bx, by);                   // drop OOB edge lanes
}
```

Python

```
@ct.kernel
def gemm(A, B, C,
         tm: ct.Constant[int], tn: ct.Constant[int], tk: ct.Constant[int]):
    bx, by = ct.bid(0), ct.bid(1)
    num_k  = ct.num_tiles(A, axis=1, shape=(tm, tk))    # number of K-tiles

    acc = ct.full((tm, tn), 0, dtype=ct.float32)        # FP32 accumulator
    for k in range(num_k):
        a = ct.load(A, index=(bx, k), shape=(tm, tk),
                    padding_mode=ct.PaddingMode.ZERO)   # zero-pad partial K-
↪tile
        b = ct.load(B, index=(k, by), shape=(tk, tn),
                    padding_mode=ct.PaddingMode.ZERO)
        acc = ct.mma(a, b, acc)                         # acc += a @ b

    ct.store(C, index=(bx, by), tile=acc.astype(C.dtype))  # cast + store
```

2.4.9.2 Reductions and Scans

Reductions are a tool for collapsing a tile into a scalar or a row of scalars. Computing the denominator of a softmax, the mean and variance of a layer norm, or the max in attention scoring all involve reduction operations.

The one point worth internalizing up front is the shape of the result. Python drops the reduced axis by default (pass `keepdims=True` to keep it as length 1); C++ always keeps it, preserving the rank of the tile. The two snippets below both reduce a 2x4 tile along axis 1; the output shape is the visible difference.

C++

```
using namespace ct::literals;
using i32x2x4 = ct::tile<int, ct::shape<2, 4>>;

auto x = ct::iota<i32x2x4>();                          // [[0,1,2,3],[4,5,6,7]]
auto row_sums = ct::sum(x, 1_ic);                      // shape (2, 1) - axis
 ↪kept
// row_sums == [[6], [22]]
```

Python

```
x   = ct.arange(8, dtype=ct.int32).reshape((2, 4))    # [[0,1,2,3],[4,5,6,7]]
s   = ct.sum(x, axis=1)                               # shape (2,)    - axis
 ↪dropped
s_k = ct.sum(x, axis=1, keepdims=True)                # shape (2, 1)  - axis
 ↪kept
# s == [6, 22];  s_k == [[6], [22]]
```

Scans are the running counterpart, producing a cumulative result along an axis. For example, prefix-sum (`cumsum`) produces an output equal in dimension to the input, where the value at a given index is the sum of all the elements up to and including that index along a specified axis. See the API reference for the full set available in each language (CUDA Tile C++ reductions and scans, cuTile Python reductions and scans).

2.4.9.3 Transpose and Permutation

Two related primitives reorder the axes of a tile without touching its data: `transpose` swaps the first two axes, and `permute` does an arbitrary reordering. They show up wherever a tile's logical layout has to change between operation such as materializing the transpose of a matmul operand, swapping rows and columns in an attention block, or lining up axes before a broadcast.

In Python, `ct.transpose(x)` on a rank-2 tile swaps its two axes; on higher-rank tiles it takes explicit `axis0` / `axis1` arguments. `ct.permute(x, axes)` takes a tuple of axis indices. In C++, `ct::transpose(x)` interchanges the first two dimensions (trailing dimensions are preserved), and `ct::permute(x, map)` takes a `ct::dimension_map` describing the new order.

C++

```
using namespace ct::literals;
using t2d = ct::tile<int, ct::shape<2, 4>>;
using t3d = ct::tile<int, ct::shape<2, 2, 2>>;
```

(continues on next page)

(continued from previous page)

```
auto tx = ct::iota<t2d>();
auto ty = ct::transpose(tx);                                          // shape (4,
↪2)

auto tz = ct::iota<t3d>();
auto tw = ct::permute(tz, ct::dimension_map{2_ic, 0_ic, 1_ic});  // axes (0,1,
↪2) -> (2,0,1)
```

Python

```
tx = ct.arange(8, dtype=ct.int32).reshape((2, 4))
ty = ct.transpose(tx)                                                 # shape (4,
↪2)

tz = ct.arange(8, dtype=ct.int32).reshape((2, 2, 2))
tw = ct.permute(tz, (2, 0, 1))                                        # axes (0,1,
↪2) -> (2,0,1)
```

2.4.9.4 Element-wise Selection

Element-wise selection is the tile form of a conditional: given a boolean tile and two operand tiles, each output element is picked from one or the other based on the corresponding boolean. The condition is broadcast to the operand shape; the operand types have to be compatible (see the API reference for the exact rules in each language: CUDA Tile C++ select, cuTile Python selection). Python spells it `ct.where(cond, x, y)`; C++ spells it `ct::select(cond, lhs, rhs)`.

C++

```
using namespace ct::literals;
auto cond = ct::iota<ct::tile<int, ct::shape<4>>>() < 2;   // {T, T, F, F}
auto t    = ct::full<ct::tile<float, ct::shape<4>>>( 1.0f);
auto f    = ct::full<ct::tile<float, ct::shape<4>>>(-1.0f);
auto r    = ct::select(cond, t, f);                        // {1, 1, -1, -1}
```

Python

```
cond    = ct.arange(4, dtype=ct.int32) < 2                # [T, T, F, F]
x_true  = ct.full((4,),  1.0, dtype=ct.float32)
x_false = ct.full((4,), -1.0, dtype=ct.float32)
result  = ct.where(cond, x_true, x_false)                 # [1, 1, -1, -1]
```

2.4.9.5 Mathematical Functions

Common element-wise math operations are available in tile code as functions in the `ct` namespace:

- `add`, `sub`, `mul`
- `truediv`, `floordiv`, `cdiv`
- `mod`

- `pow`
- `exp`, `exp2`, `log`, `log2`
- `sqrt`, `rsqrt`
- `sin`, `cos`, `tan`
- `sinh`, `cosh`, `tanh`
- `minimum`, `maximum`
- `negative`
- `floor`, `ceil`

Each function applies its operation element-wise on input tile(s) and returns a tile of the same shape. These operations also work on scalars within tile code.

For exact details and full lists of supported element-wise operations, refer to the API references:

- cuTile Python Math Operations.
- CUDA Tile C++ Math Operations.

2.4.10. Atomic Memory Operations

There are two situations where use of memory atomics is needed in tile code:

- In *cross-block contention*, each block produces a partial result and uses an atomic operation to merge it with the partial results of other blocks in a global memory location.
- In *intra-block contention*, multiple elements of a tile are written to the same location in memory.

An atomic on a tile performs one atomic update *per element* of the tile. The per-element operation is atomic, but the whole call is not. The order of the per-element atomic operations is unspecified.

In Python, atomics address the target by indices into the array, using the same convention as `ct.gather` and `ct.scatter`. Optional parameters control bounds checking, memory order, and thread scope. The defaults (bounds checking on, `ACQ_REL`, device scope) let the ordinary call pass only the array, the indices, and the update. A `TiledView` also exposes the same atomic operations as instance methods (for example, `TiledView.atomic_add(index, update)`); these address the target by a tile-space index, do not return a value, and lower to an atomic reduction in PTX. When the prior value is not needed, the `TiledView` form is preferred for better performance.

In C++, atomics take a pointer and a corresponding value: a raw pointer and scalar for a single location, or a tile of pointers and tile of values. The memory order is a compile-time type tag at the call site such as `ct::memory_order_relaxed_t{}`. The thread scope is a type tag of the same form and defaults to system-wide visibility when omitted.

2.4.10.1 Cross-block Contention

In the code example below, cross-block contention occurs because different blocks are writing to the same memory location, `out`. Without atomic operations, blocks running in parallel would lead to an incorrect answer. In this example, device thread scope is used (`ct::thread_scope_device_t{}` in C++, thread scope defaults to device-wide scope in Python) because the result of the memory operation must be visible to all blocks running on the device. The Python kernel uses `TiledView.atomic_add` because each block's partial sum is accumulated into `out[0]` and immediately discarded.

C++

```
__tile_global__ void block_sum(int* __restrict__ arr, int* __restrict__ out,
↪std::size_t N) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;
    constexpr auto TILE = 16_ic;

    arr = ct::assume_aligned(arr, 16_ic);
    out = ct::assume_aligned(out, 16_ic);

    auto aView = ct::partition_view{ct::tensor_span{arr, ct::extents{N}},
                                    ct::shape{TILE}};
    int bid = ct::bid().x;
    auto tile    = aView.load_masked(bid);        // partial final tile ->
↪OOB lanes default to 0
    auto partial = ct::sum(tile, 0_ic);           // reduce to a 1-element
↪tile

    ct::atomic_add(out, (int)partial,             // accumulate the scalar
↪into out[0]
                   ct::memory_order_relaxed_t{},  // single-location
↪accumulator -> relaxed suffices
                   ct::thread_scope_device_t{});  // visible across the device
}
```

Python

```
@ct.kernel
def block_sum(arr, out, TILE: ct.Constant[int]):
    bid = ct.bid(0)
    # partial final tile -> OOB lanes default to 0
    tile    = ct.load(arr, index=(bid,), shape=(TILE,),
                      padding_mode=ct.PaddingMode.ZERO)
    partial = ct.sum(tile)                                  # reduce to a scalar
    out.tiled_view((1,)).atomic_add((0,), partial)          # atomically
↪accumulate into out[0]
```

2.4.10.2 Intra-block Contention

In the code fragment below, intra-block contention occurs because all values of a tile are atomically added to a single location in memory.

In this example, each element in the `ptrs` tile points to the same location in memory, `slot`. Each element of the tile created by `ct::iota<i32x16>()` is atomically added to the value stored in that memory location. The order in which multiple atomic operations from a tile to a single memory address are carried out is unspecified. The block thread scope `ct::thread_scope_block_t{}` is used to specify that the result of the atomic operations need only be visible within this thread block.

C++

```
using i32x16 = ct::tile<int, ct::shape<16>>;

int* slot = /* pointer to the contended location */;

// 16 lanes all aim at the same address. Add is commutative, so the
// unspecified ordering doesn't affect this sum; block scope suffices
// since contention stays within one block.
auto ptrs = ct::full<ct::tile<int*, ct::shape<16>>>(slot);
ct::atomic_add(ptrs, ct::iota<i32x16>(),
               ct::memory_order_relaxed_t{},
               ct::thread_scope_block_t{});
```

Note

This is shown for illustrative purposes only. To sum a tile into a scalar within a block, the tile reduction operation shown in Section 2.4.9.2 is the preferred method.

2.4.10.3 Supported Atomic Operations

Tile code supports a variety of atomic memory operations which differ in how the value being written is combined with the value that exists in memory:

- `atomic_and` - performs element-wise atomic bitwise AND between the passed value(s) and the value(s) in memory
- `atomic_or` - performs element-wise atomic bitwise OR between the passed value(s) and the value(s) in memory
- `atomic_xor` - performs element-wise atomic bitwise XOR between the passed value(s) and the value(s) in memory
- `atomic_max` - performs element-wise comparison between the passed value(s) and the value(s) in memory and stores the larger value into memory
- `atomic_min` - performs element-wise comparison between the passed value(s) and the value(s) in memory and stores the smaller value into memory
- `atomic_add` - adds the passed value(s) to the value(s) in memory and stores the result into memory
- `atomic_xchng` - writes the passed value(s) to memory and returns a the value(s) that were in memory before the write
- `atomic_cas` - performs element-wise comparison between the value(s) in memory and expected value(s) passed as arguments. If they match, the value in memory is replaced with the desired value(s)

For complete documentation on all supported atomic memory operations, refer to the memory operations sections of the CUDA Tile C++ API Reference or cuTile Python API Reference.

2.4.11. Optimization Hints

An optimization hint is metadata attached to a source construct (a tile kernel function, a load/store call site, and so on) that guides the compiler's code generation. Hints do not change the semantics of the program: a kernel compiles and runs identically with or without them, so they can be added, removed, or tuned freely without affecting correctness. The compiler may also ignore any hint.

Hints share two general properties:

- **Hints are per-construct.** A hint applies to the specific kernel function or the specific call expression it is attached to, not to the surrounding code.
- **Hints can be specified per architecture.** Each hint may be set to a different value for different GPU architectures, or to a single value that applies to every target.

The two languages expose hints differently:

- C++ uses a C++ attribute that you place on the relevant declaration or statement.
- Python uses keyword arguments on the kernel decorator and on individual memory-operation call sites.

The set of hint kinds, what each hint actually controls, is shared between the two languages and is documented in *Hint Kinds*.

2.4.11.1 C++ – the `cutile::hint` Attribute

In C++, hints are expressed with the C++ attribute `cutile::hint`:

```
[[ cutile::hint(arch, kind1=value1, kind2=value2, ...) ]]
```

The first argument is the target architecture, encoded as an integer using the same convention as the `__CUDA_ARCH__` macro (for example, `900` for `sm_90` and `1000` for `sm_100`). The special value `0` denotes an *architecture-agnostic* hint that applies to every target architecture. Each remaining argument is a `kind=value` pair that specifies a hint kind and its value.

A `cutile::hint` attribute applies to the construct it precedes:

- For tile kernel functions, place the attribute on the function declaration.
- For memory operations such as `ct::load`, `ct::store`, and `ct::partition_view` loads/stores, place it on the expression-statement containing the call.

Other placements have limitations; see the CUDA Tile C++ hint specification for the full set of rules.

The kernel below illustrates both placements: a kernel-level hint that sets a different `num_cta_in_cga` for `sm_90` and `sm_100`, and an expression-statement hint that marks a particular load as bandwidth-heavy.

C++

```
[[ cutile::hint(900,  num_cta_in_cga=4),    // sm_90:  prefer 4 CTAs per
↪cluster
   cutile::hint(1000, num_cta_in_cga=8) ]]  // sm_100: prefer 8 CTAs per
↪cluster
__tile_global__ void optimization_hints(float* __restrict__ in,
                                        float* __restrict__ out) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;
```

(continues on next page)

(continued from previous page)

```
    in  = ct::assume_aligned(in,  16_ic);
    out = ct::assume_aligned(out, 16_ic);

    auto inSpan  = ct::tensor_span{in,  ct::extents{128_ic}};
    auto outSpan = ct::tensor_span{out, ct::extents{128_ic}};
    auto inView  = ct::partition_view{inSpan,  ct::shape{8_ic}};
    auto outView = ct::partition_view{outSpan, ct::shape{8_ic}};

    int bx = ct::bid().x;

    // Expression-statement hint: tag this particular load as bandwidth-heavy.
    ct::tile<float, ct::shape<8>> tile;
    [[ cutile::hint(0, latency=8) ]]
    tile = inView.load(bx);

    outView.store(tile, bx);
}
```

When several hints of the same kind apply to the same construct, an architecture-specific hint overrides an architecture-agnostic one.

2.4.11.2 Python – Decorator Arguments and Call-Site Keywords

Python exposes hints in two ways:

- **Kernel-level hints** are keyword arguments to the `@ct.kernel(...)` decorator. A compiled kernel object also has a `.replace_hints(**hints)` method that returns a new kernel with overridden hints; the new kernel has its own JIT cache, which makes `replace_hints` the natural building block for autotuning loops.
- **Per-call hints** are keyword arguments on memory-operation call sites: `ct.load` / `ct.store`, `TiledView.load` / `TiledView.store`, and `ct.gather` / `ct.scatter`.

For per-architecture values, wrap the value in `cuda.tile.ByTarget(*, default=..., sm_XXX=..., sm_YYY=...)`. Architecture keys must be strings of the form `"sm_<major><minor>"` (for example, `"sm_100"` or `"sm_120"`). A plain (non-`ByTarget`) value applies to every target – it is the Python equivalent of the C++ architecture-agnostic hint with `arch=0`.

The kernel below is the direct Python counterpart of the *C++ example above*: `ByTarget` carries the kernel-level hint, the `latency=8` keyword carries the per-call hint, and `replace_hints` produces a re-tuned kernel without editing the source.

Python

```
@ct.kernel(num_ctas=ByTarget(sm_90=4, sm_100=8))
def optimization_hints(in_, out, TILE: ct.Constant[int]):
    bid = ct.bid(0)

    # Per-call hint: this particular load is bandwidth-heavy.
    tile = ct.load(in_, index=(bid,), shape=(TILE,), latency=8)

    ct.store(out, index=(bid,), tile=tile)
```

(continues on next page)

(continued from previous page)

```
# Autotuning: produce a new kernel with overridden hints without editing the
# source. The new kernel has its own JIT cache.
tuned_kernel = optimization_hints.replace_hints(num_ctas=8)
```

2.4.11.3 Hint Kinds

The following hints are shared between the two languages. Within each hint, the **C++ name** and **Python name** entries are different spellings of the same underlying hint; everything else is the same: where the hint applies, its values, its meaning.

2.4.11.3.1 CTAs per cluster

- **C++ name:** `num_cta_in_cga` (kernel attribute).
- **Python name:** `num_ctas` (`@ct.kernel` decorator argument).
- **Allowed values:** 1, 2, 4, 8, 16. On `sm_80`, only 1 is applicable.
- **Meaning:** the number of cooperative thread arrays (CTAs) that the compiler should prefer per cooperative group array (CGA) when launching the kernel.

2.4.11.3.2 Occupancy

- **C++ name:** `occupancy` (kernel attribute).
- **Python name:** `occupancy` (`@ct.kernel` decorator argument).
- **Allowed values:** any integer in the inclusive range `[1, 32]`.
- **Meaning:** the target number of active CTAs per streaming multiprocessor (SM). The compiler treats the value as a recommendation and will try to honor it during code generation.

2.4.11.3.3 Memory access latency

- **C++ name:** `latency` (attribute on the expression-statement containing the call).
- **Python name:** `latency` (keyword argument on the call site).
- **Applies to:** tile-space loads and stores (`ct::partition_view` in C++; `Array.tiled_view` and `ct.load` / `ct.store` in Python) as well as gather/scatter (`ct::load` / `ct::store` with pointer tiles in C++; `ct.gather` / `ct.scatter` in Python).
- **Allowed values:** any integer in the inclusive range `[1, 10]`, where 1 indicates light DRAM traffic and `10` indicates heavy traffic. Larger values typically cause the compiler to schedule a larger prefetch depth.

2.4.11.3.4 Allow TMA

- **C++ name:** `allow_tma` (attribute on the expression-statement containing the call).
- **Python name:** `allow_tma` (keyword argument on the call site).
- **Applies to:** tile-space loads and stores only (`ct::partition_view` in C++; `Array.tiled_view` and `ct.load` / `ct.store` in Python). Gather and scatter operations do not accept this hint.

- **Allowed values:** `true` / `false` (C++) or `True` / `False` (Python). TMA is allowed by default; setting the hint to `false`/`False` instructs the compiler not to lower this particular load or store to TMA on hardware that supports it.

2.4.12. C++ Performance Tips

The C++ kernels in this guide all use the same handful of annotations and idioms. This section explains what they do and why they matter.

2.4.12.1 Use `__restrict__` Pointers for Arrays in Memory

The `__restrict__` keyword tells the compiler that the region of memory accessed through a pointer will only be accessed through that pointer for the life of the pointer. See Section 5.4.1.4.

In tile C++, using arrays in memory which adhere to these conditions and labeling the pointers to them with the `__restrict__` keyword is essential for good memory-operation performance.

To see why, consider an element-wise copy which uses arrays whose pointers are not `__restrict__`:

C++

```
__tile_global__ void tile_elementwise_copy(float* out, float const* in) {
    namespace ct = cuda::tiles;

    using f32x64 = ct::tile<float, ct::shape<64>>;
    using i32x64 = ct::tile<int, ct::shape<64>>;

    auto inPtrs  = in  + 64 * ct::bid().x + ct::iota<i32x64>();
    auto outPtrs = out + 64 * ct::bid().x + ct::iota<i32x64>();

    auto data = ct::load(inPtrs);   // (1)
    ct::store(outPtrs, data);       // (2)
}
```

How the compiler parallelizes tile operations can generally be ignored in CUDA Tile programs. However, we will consider it here to understand why using non-overlapping arrays enables the compiler to generate better-performing code.

Consider how the compiler will parallelize the `load` and `store` tile operations. If the input and output arrays do not overlap, the `load` can be parallelized into a set of independent memory read operations. Similarly, the `store` can be parallelized into multiple memory write operations, each of which depends only on the load operations for the data element(s) it writes.

If, however, the input and output arrays may overlap, then the compiler must ensure that all memory load operations for the entire tile have completed prior to issuing any of the memory store operations to ensure correct program semantics. Otherwise, a store operations could execute and overwrite an element before it has been read in a load operation, resulting in incorrect program execution. This limits the compiler's ability to interleave reads and writes, since all reads must complete before any writes can be issued.

In short, when the compiler cannot guarantee non-overlapping arrays, it must generate more conservative code. This is why using non-overlapping arrays and informing the compiler of this using the `__restrict__` keyword on their pointers helps achieve best performance.

Labeling a pointer with `__restrict__` when the memory region can be accessed by another pointer will result in undefined behavior.

2.4.12.2 Mark Array Pointers as 16-Byte Aligned

Mark pointers to arrays as 16-byte aligned with `ct::assume_aligned`:

```
__tile_global__ void foo(float* __restrict__ in) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;

    in = ct::assume_aligned(in, 16_ic);

    ct::tensor_span t{in, ct::extents{256_ic, 256_ic}};
    ct::partition_view{t, ct::shape{4_ic, 4_ic}};

    // ...
}
```

This alignment guarantee is necessary for `ct::partition_view` to use the Tensor Memory Accelerator (TMA). You must provide 16-byte-aligned pointers at runtime when using this technique, or behavior is undefined.

Pointers returned by CUDA memory allocators such as `cudaMalloc` are guaranteed to be at least 16-byte aligned.

2.4.12.3 Prefer `ct::partition_view` for Memory Access

Prefer `ct::partition_view` over the gather and scatter forms `ct::load` and `ct::store` for structured memory access. The view-based form can be lowered to the Tensor Memory Accelerator (TMA) on supported hardware, which is significantly faster than per-element gather. See *Gather and Scatter* for the gather/scatter context.

2.4.12.4 Use `ct::irange` for Bounded Loops

Use `ct::irange` instead of a plain `for` loop when iterating over a fixed range. The structured form lets the compiler apply optimizations such as pipelining and vectorization that aren't available when the loop bounds and step are opaque integer expressions (see *Control Flow*):

```
for (auto idx : ct::irange(lowerBound, upperBound, step)) {
    // ...
}
```

2.5. Asynchronous Execution

2.5.1. What is Asynchronous Concurrent Execution?

CUDA allows concurrent, or overlapping, execution of multiple tasks, specifically:

- computation on the host
- computation on the device
- memory transfers from the host to the device
- memory transfers from the device to the host
- memory transfers within the memory of a given device

- memory transfers among devices

The concurrency is expressed via an asynchronous interface, where a dispatching function call or kernel launch returns immediately. Asynchronous calls usually return before the dispatched operation has completed and may return before the asynchronous operation has started. The application is then free to perform other tasks at the same time as the originally dispatched operation. When the final results of the initially dispatched operation are needed, the application must perform some form of synchronization to ensure that the operation in question has completed. A typical example of a concurrent execution pattern is the overlapping of host and device memory transfers with computation and thus reducing or eliminating their overhead.

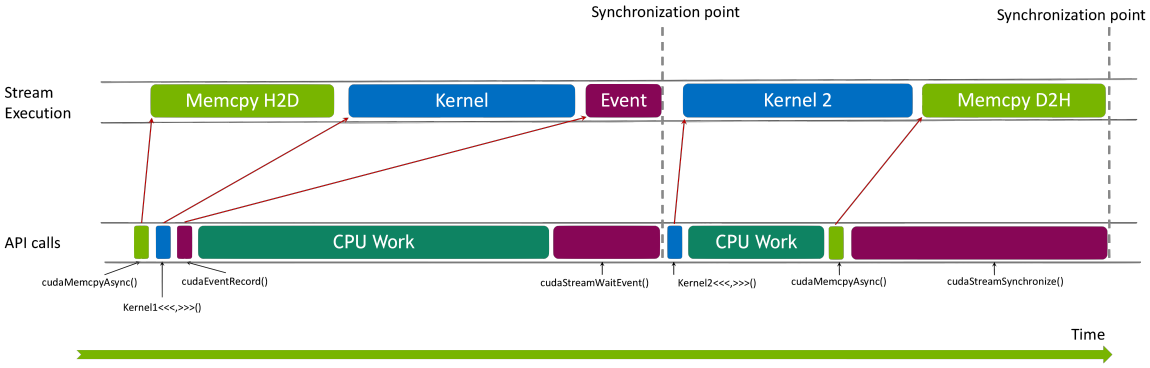


Figure 20: Asynchronous COncurrent Execution with CUDA streams

In general, asynchronous interfaces typically provide three main ways to synchronize with the dispatched operation

- a **blocking approach**, where the application calls a function that blocks, or waits until the operation has completed
- a **non-blocking approach**, or polling approach where the application calls a function that returns immediately and supplies information about the status of the operation
- a **callback approach**, where a pre-registered function is executed when the operation has completed.

While the programming interfaces are asynchronous, the actual ability to carry out various operations concurrently will depend on the version of CUDA and the compute capability of the hardware being used – these details will be left to a later section of this guide (see *Compute Capabilities*).

In *Synchronizing CPU and GPU*, the CUDA runtime function `cudaDeviceSynchronize()` was introduced, which is a blocking call which waits for all previously issued work to complete. The reason the `cudaDeviceSynchronize()` call was needed is because the kernel launch is asynchronous and returns immediately. CUDA provides an API for both blocking and non-blocking approaches to synchronization and even supports the use of host-side callback functions.

The core API components for asynchronous execution in CUDA are **CUDA Streams** and **CUDA Events**. In the rest of this section we will explain how these elements can be used to express asynchronous execution in CUDA.

A related topic is that of **CUDA Graphs**, which allow a graph of asynchronous operations to be defined up front, which can then be executed repeatedly with minimal overhead. We cover CUDA Graphs in a very introductory level in section *2.4.9.2 Introduction to CUDA Graphs with Stream Capture*, and a more comprehensive discussion is provided in section *4.1 CUDA Graphs*.

2.5.2. CUDA Streams

At the most basic level, a CUDA stream is an abstraction which allows the programmer to express a sequence of operations. A stream operates like a work-queue into which programs can add operations, such as memory copies or kernel launches, to be executed in order. Operations at the front of the queue for a given stream are executed and then dequeued allowing the next queued operation to come to the front and to be considered for execution. The order of execution of operations in a stream is sequential and the operations are executed in the order they are enqueued into the stream.

An application may use multiple streams simultaneously. In such cases, the runtime will select a task to execute from the streams that have work available depending on the state of the GPU resources. Streams may be assigned a priority which acts as a hint to the runtime to influence the scheduling, but does not guarantee a specific order of execution.

The API function calls and kernel-launches operating in a stream are asynchronous with respect to the host thread. Applications can synchronize with a stream by waiting for it to be empty of tasks, or they can also synchronize at the device level.

CUDA has a default stream, and operations and kernel launches without a specific stream are queued into this default stream. Code examples which do not specify a stream are using this default stream implicitly. The default stream has some specific semantics which are discussed in subsection *Blocking and non-blocking streams and the default stream*.

2.5.2.1 Creating and Destroying CUDA Streams

CUDA streams can be created using the `cudaStreamCreate()` function. The function call initializes the stream handle which can be used to identify the stream in subsequent function calls.

```
cudaStream_t stream;        // Stream handle
cudaStreamCreate(&stream);  // Create a new stream

// stream based operations ...

cudaStreamDestroy(stream);  // Destroy the stream
```

If the device is still doing work in stream `stream` when the application calls `cudaStreamDestroy()`, the stream will complete all the work in the stream before being destroyed.

2.5.2.2 Launching Kernels in CUDA Streams

The usual triple-chevron syntax for launching a kernel can also be used to launch a kernel into a specific stream. The stream is specified as an extra parameter to the kernel launch. In the following example the kernel named `kernel` is launched into the stream with handle `stream`, which is of type `cudaStream_t` and has been assumed to have been created previously:

```
kernel<<<grid, block, shared_mem_size, stream>>>(...);
```

The kernel launch is asynchronous and the function call returns immediately. Assuming that the kernel launch is successful, the kernel will execute in the stream `stream` and the application is free to perform other tasks on the CPU or in other streams on the GPU while the kernel is executing.

2.5.2.3 Launching Memory Transfers in CUDA Streams

To launch a memory transfer into a stream, we can use the function `cudaMemcpyAsync()`. This function is similar to the `cudaMemcpy()` function, but it takes an additional parameter specifying the stream to use for the memory transfer. The function call in the code block below copies `size` bytes from the host memory pointed to by `src` to the device memory pointed to by `dst` in the stream `stream`.

```
// Copy `size` bytes from `src` to `dst` in stream `stream`
cudaMemcpyAsync(dst, src, size, cudaMemcpyHostToDevice, stream);
```

Like other asynchronous function calls, this function call returns immediately, whereas the `cudaMemcpy()` function blocks until the memory transfer is complete. In order to access the results of the transfer safely, the application must determine that the operation has completed using some form of synchronization.

Other CUDA memory transfer functions such as `cudaMemcpy2D()` also have asynchronous variants.

Note

In order for memory copies involving CPU memory to be carried out asynchronously, the host buffers must be pinned and page-locked. `cudaMemcpyAsync()` will function correctly if host memory which is not pinned and page-locked is used, but it will revert to a synchronous behavior which will not overlap with other work. This can inhibit the performance benefits of using asynchronous memory transfers. It is recommended programs use `cudaMallocHost()` to allocate buffers which will be used to send or receive data from GPUs.

2.5.2.4 Stream Synchronization

The simplest way to synchronize with a stream is to wait for the stream to be empty of tasks. This can be done in two ways, using the `cudaStreamSynchronize()` function or the `cudaStreamQuery()` function.

The `cudaStreamSynchronize()` function will block until all the work in the stream has completed.

```
// Wait for the stream to be empty of tasks
cudaStreamSynchronize(stream);

// At this point the stream is done
// and we can access the results of stream operations safely
```

If we prefer not to block, but just need a quick check to see if the stream is empty we can use the `cudaStreamQuery()` function.

```
// Have a peek at the stream
// returns cudaSuccess if the stream is empty
// returns cudaErrorNotReady if the stream is not empty
cudaError_t status = cudaStreamQuery(stream);

switch (status) {
    case cudaSuccess:
        // The stream is empty
        std::cout << "The stream is empty" << std::endl;
        break;
```

(continues on next page)

(continued from previous page)

```
    case cudaErrorNotReady:
        // The stream is not empty
        std::cout << "The stream is not empty" << std::endl;
        break;
    default:
        // An error occurred - we should handle this
        break;
};
```

2.5.3. CUDA Events

CUDA events are a mechanism for inserting markers into a CUDA stream. They are essentially like tracer particles that can be used to track the progress of tasks in a stream. Imagine launching two kernels into a stream. Without such tracking events, we would only be able to determine whether the stream is empty or not. If we had an operation that was dependent on the output of the first kernel, we would not be able to start that operation safely until we knew the stream was empty by which time both kernels would have completed.

Using CUDA Events we can do better. By enqueuing an event into a stream directly after the first kernel, but before the second kernel, we can wait for this event to come to the front of the stream. Then, we can safely start our dependent operation knowing that the first kernel has completed, but before the second kernel has started. Using CUDA events in this way can build up a graph of dependencies between operations and streams. This graph analogy translates directly into the later discussion of *CUDA graphs*.

CUDA events also keep time information which can be used to time kernel launches and memory transfers.

2.5.3.1 Creating and Destroying CUDA Events

CUDA Events can be created and destroyed using the `cudaEventCreate()` and `cudaEventDestroy()` functions.

```
cudaEvent_t event;

// Create the event
cudaEventCreate(&event);

// do some work involving the event

// Once the work is done and the event is no longer needed
// we can destroy the event
cudaEventDestroy(event);
```

The application is responsible for destroying events when they are no longer needed.

2.5.3.2 Inserting Events into CUDA Streams

CUDA Events can be inserted into a stream using the `cudaEventRecord()` function.

```
cudaEvent_t event;
cudaStream_t stream;
```

(continues on next page)

(continued from previous page)

```
// Create the event
cudaEventCreate(&event);

// Insert the event into the stream
cudaEventRecord(event, stream);
```

2.5.3.3 Timing Operations in CUDA Streams

CUDA events can be used to time the execution of various stream operations including kernels. When an event reaches the front of a stream it records a timestamp. By surrounding a kernel in a stream with two events we can get an accurate timing of the duration of the kernel execution as is shown in the code snippet below:

```
cudaStream_t stream;
cudaStreamCreate(&stream);

cudaEvent_t start;
cudaEvent_t stop;

// create the events
cudaEventCreate(&start);
cudaEventCreate(&stop);

 // record the start event
cudaEventRecord(start, stream);

// launch the kernel
kernel<<<grid, block, 0, stream>>>(...);

// record the stop event
cudaEventRecord(stop, stream);

// wait for the stream to complete
// both events will have been triggered
cudaStreamSynchronize(stream);

// get the timing
float elapsedTime;
cudaEventElapsedTime(&elapsedTime, start, stop);
std::cout << "Kernel execution time: " << elapsedTime << " ms" << std::endl;

// clean up
cudaEventDestroy(start);
cudaEventDestroy(stop);
cudaStreamDestroy(stream);
```

2.5.3.4 Checking the Status of CUDA Events

Like in the case of checking the status of streams, we can check the status of events in either a blocking or a non-blocking way.

The `cudaEventSynchronize()` function will block until the event has completed. In the code snippet below we launch a kernel into a stream, followed by an event and then by a second kernel. We can use the `cudaEventSynchronize()` function to wait for the event after the first kernel to complete and in principle launch a dependent task immediately, potentially before *kernel2* finishes.

```
cudaEvent_t event;
cudaStream_t stream;

// create the stream
cudaStreamCreate(&stream);

// create the event
cudaEventCreate(&event);

// launch a kernel into the stream
kernel<<<grid, block, 0, stream>>>(...);

// Record the event
cudaEventRecord(event, stream);

// launch a kernel into the stream
kernel2<<<grid, block, 0, stream>>>(...);

// Wait for the event to complete
// Kernel 1 will be  guaranteed to have completed
// and we can launch the dependent task.
cudaEventSynchronize(event);
dependentCPUtask();

// Wait for the stream to be empty
// Kernel 2 is guaranteed to have completed
cudaStreamSynchronize(stream);

// destroy the event
cudaEventDestroy(event);

// destroy the stream
cudaStreamDestroy(stream);
```

CUDA Events can be checked for completion in a non-blocking way using the `cudaEventQuery()` function. In the example below we launch 2 kernels into a stream. The first kernel, *kernel1* generates some data which we would like to copy to the host, however we also have some CPU side work to do. In the code below, we enqueue *kernel1* followed by an event (*event*) and then *kernel2* into stream *stream1*. We then go into a CPU work loop, but occasionally take a peek to see if the event has completed indicating that *kernel1* is done. If so, we launch a device to host copy into stream *stream2*. This approach allows the overlap of the CPU work with the GPU kernel execution and the device to host copy.

```
cudaEvent_t event;
cudaStream_t stream1;
```

(continues on next page)

(continued from previous page)

```
cudaStream_t stream2;

size_t size = LARGE_NUMBER;
float* d_data;
float* h_data;

// Create some data
cudaMalloc(&d_data, size);
cudaMallocHost(&h_data, size);

// create the streams
cudaStreamCreate(&stream1);   // Processing stream
cudaStreamCreate(&stream2);   // Copying stream
bool copyStarted = false;

//  create the event
cudaEventCreate(&event);

// launch kernel1 into the stream
kernel1<<<grid, block, 0, stream1>>>(d_data, size);
// enqueue an event following kernel1
cudaEventRecord(event, stream1);

// launch kernel2 into the stream
kernel2<<<grid, block, 0, stream1>>>();

// while the kernels are running do some work on the CPU
// but check if kernel1 has completed because then we will start
// a device to host copy in stream2
while ( not allCPUWorkDone() || not copyStarted ) {
    doNextChunkOfCPUWork();

    // peek to see if kernel 1 has completed
    // if so enqueue a non-blocking copy into stream2
    if ( not copyStarted ) {
        if( cudaEventQuery(event) == cudaSuccess ) {
            cudaMemcpyAsync(h_data, d_data, size, cudaMemcpyDeviceToHost,
↪stream2);
            copyStarted = true;
        }
    }
}

// wait for both streams to be done
cudaStreamSynchronize(stream1);
cudaStreamSynchronize(stream2);

// destroy the event
cudaEventDestroy(event);

// destroy the streams and free the data
cudaStreamDestroy(stream1);
```

(continues on next page)

(continued from previous page)

```
cudaStreamDestroy(stream2);
cudaFree(d_data);
free(h_data);
```

2.5.4. Callback Functions from Streams

CUDA provides a mechanism for launching functions on the host from within a stream. There are currently two functions available for this purpose: `cudaLaunchHostFunc()` and `cudaAddCallback()`. However, `cudaAddCallback()` is slated for deprecation, so applications should use `cudaLaunchHostFunc()`.

Using `cudaLaunchHostFunc()`

The signature of the `cudaLaunchHostFunc()` function is as follows:

```
cudaError_t cudaLaunchHostFunc(cudaStream_t stream, void (*func)(void *),
↪void *data);
```

where

- `stream`: The stream to launch the callback function into.
- `func`: The callback function to launch.
- `data`: A pointer to the data to pass to the callback function.

The host function itself is a simple C function with the signature:

```
void hostFunction(void *data);
```

with the `data` parameter pointing to a user defined data structure which the function can interpret. There are some caveats to keep in mind when using callback functions like this. In particular, the host function may not call any CUDA APIs.

For the purposes of being used with unified memory, the following execution guarantees are provided:

- The stream is considered idle for the duration of the function's execution. Thus, for example, the function may always use memory attached to the stream it was enqueued in.
- The start of execution of the function has the same effect as synchronizing an event recorded in the same stream immediately prior to the function. It thus synchronizes streams which have been "joined" prior to the function.
- Adding device work to any stream does not have the effect of making the stream active until all preceding host functions and stream callbacks have executed. Thus, for example, a function might use global attached memory even if work has been added to another stream, if the work has been ordered behind the function call with an event.
- Completion of the function does not cause a stream to become active except as described above. The stream will remain idle if no device work follows the function, and will remain idle across consecutive host functions or stream callbacks without device work in between. Thus, for example, stream synchronization can be done by signaling from a host function at the end of the stream.

2.5.4.1 Using cudaStreamAddCallback()

Note

The cudaStreamAddCallback() function is slated for deprecation and removal and is discussed here for completeness and because it may still appear in existing code. Applications should use or switch to using cudaLaunchHostFunc().

The signature of the cudaStreamAddCallback() function is as follows:

```
cudaError_t cudaStreamAddCallback(cudaStream_t stream, cudaStreamCallback_t
↪callback, void* userData, unsigned int flags);
```

where

- stream: The stream to launch the callback function into.
- callback: The callback function to launch.
- userData: A pointer to the data to pass to the callback function.
- flags: Currently, this parameter must be 0 for future compatibility.

The signature of the callback function is a little different from the case when we used the cudaLaunchHostFunc() function. In this case the callback function is a C function with the signature:

```
void callbackFunction(cudaStream_t stream, cudaError_t status, void
↪*userData);
```

where the function is now passed

- stream: The stream handle from which the callback function was launched.
- status: The status of the stream operation that triggered the callback.
- userData: A pointer to the data that was passed to the callback function.

In particular the status parameter will contain the current error status of the stream, which may have been set by previous operations. Similarly to the cudaLaunchHostFunc() func case, the stream will not be active and advance to tasks until the host-function has completed, and no CUDA functions may be called from within the callback function.

2.5.4.2 Asynchronous Error Handling

In a cuda stream, errors may originate from any operation in the stream, including for kernel launches and memory transfers. These errors may not be propagated back to the user at run-time until the stream is synchronized, for example, by waiting for an event or calling cudaStreamSynchronize(). There are two ways to find out about errors which may have occurred in a stream.

- Using the function cudaGetLastError() - this function returns and clears the last error encountered in any stream in the current context. An immediate second call to cudaGetLastError() would return cudaSuccess if no other error occurred between the two calls.
- Using the function cudaPeekAtLastError() - this function returns the last error in the current context, but does not clear it.

Both of these functions return the error as a value of type cudaError_t. Printable names names of the errors can be generated using the functions *cudaGetErrorName()* and *cudaGetErrorString()*.

An example of using these functions is shown below:

Listing 1: Example of using cudaGetLastError() and cudaPeekAtLastError()

```
// Some work occurs in streams.
cudaStreamSynchronize(stream);

// Look at the last error but do not clear it
cudaError_t err = cudaPeekAtLastError();
if (err != cudaSuccess) {
    printf("Error with name: %s\n", cudaGetErrorName(err));
    printf("Error description: %s\n", cudaGetErrorString(err));
}

// Look at the last error and clear it
cudaError_t err2 = cudaGetLastError();
if (err2 != cudaSuccess) {
    printf("Error with name: %s\n", cudaGetErrorName(err2));
    printf("Error description: %s\n", cudaGetErrorString(err2));
}

if (err2 != err) {
    printf("As expected, cudaPeekAtLastError() did not clear the error\n");
}

// Check again
cudaError_t err3 = cudaGetLastError();
if (err3 == cudaSuccess) {
    printf("As expected, cudaGetLastError() cleared the error\n");
}
```

Tip

When an error appears at a synchronization, especially in a stream with many operations, it is often difficult to pinpoint exactly where in the stream the error may have occurred. To debug such a situation a useful trick may be to set the environment variable `CUDA_LAUNCH_BLOCKING=1` and then run the application. The effect of this environment variable is to synchronize after every single kernel launch. This can aid in tracking down which kernel, or transfer caused the error. Synchronization can be expensive; applications may run substantially slower when this environment variable is set.

2.5.5. CUDA Stream Ordering

Now that we have discussed the basic mechanisms of streams, events and callback functions it is important to consider the ordering semantics of asynchronous operations in a stream. These semantics are to allow application programmers to think about the ordering of operations in a stream in a safe way. There are some special cases where these semantics may be relaxed for purposes of performance optimization such as in the case of a *programmatic dependent kernel launch*, which allows the overlap of two kernels through the use of special attributes and kernel launch mechanisms, or in the case of batching memory transfers using the *asynchronous batched memory copy functions*, when the runtime can perform non-overlapping batch copies concurrently.

Most importantly CUDA streams are what are known as in-order streams. This means that the order

of execution of the operations in a stream is the same as the order in which those operations were enqueued. An operation in a stream cannot leap-frog other operations. Memory operations (such as copies) are tracked by the runtime and will always complete before the next operation in order to allow dependent kernels safe access to the data being transferred.

2.5.6. Blocking and non-blocking streams and the default stream

In CUDA there are two types of streams: blocking and non-blocking. The name can be a little misleading as the blocking and non-blocking semantics refer only to how the streams synchronize with the default stream. By default, streams created with `cudaStreamCreate()` are blocking streams. In order to create a non-blocking stream, the `cudaStreamCreateWithFlags()` function must be used with the `cudaStreamNonBlocking` flag:

```
cudaStream_t stream;
cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
```

and non-blocking streams can be destroyed in the usual way with `cudaStreamDestroy()`.

2.5.6.1 Legacy Default Stream

The key difference between the blocking and non-blocking streams is how they synchronize with the **default stream**. CUDA provides a legacy default stream (also known as the NULL stream or the stream with stream ID 0) which is used when no stream is specified in kernel launches or in blocking `cudaMemcpy()` calls. This default stream, which was shared amongst all host threads, is a blocking stream. When an operation is launched into this default stream, it will synchronize with all other blocking streams, in other words it will wait for all other blocking streams to complete before it can execute.

```
cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

kernel1<<<grid, block, 0, stream1>>>(...);
kernel2<<<grid, block>>>(...);
kernel3<<<grid, block, 0, stream2>>>(...);

cudaDeviceSynchronize();
```

The default stream behavior means that in the above code snippet above, *kernel2* will wait for *kernel1* to complete, and *kernel3* will wait for *kernel2* to complete, even if in principle all three kernels could execute concurrently. By creating a non-blocking stream we can avoid this synchronization behavior. In the code snippet below we create two non-blocking streams. The default stream will no longer synchronize with these streams and in principle all three kernels could execute concurrently. As such we cannot assume any ordering of execution of the kernels and should perform explicit synchronization (such as with the rather heavy handed `cudaDeviceSynchronize()` call) in order to ensure that the kernels have completed.

```
cudaStream_t stream1, stream2;
cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&stream2, cudaStreamNonBlocking);

kernel1<<<grid, block, 0, stream1>>>(...);
```

(continues on next page)

(continued from previous page)

```
kernel2<<<grid, block>>>(...);
kernel3<<<grid, block, 0, stream2>>>(...);

cudaDeviceSynchronize();
```

2.5.6.2 Per-thread Default Stream

Starting in CUDA-7, CUDA allows for each host thread to have its own independent default stream, rather than the shared legacy default stream. In order to enable this behavior one must either use the *nvcc* compiler option `--default-stream per-thread` or define the `CUDA_API_PER_THREAD_DEFAULT_STREAM` preprocessor macro. When this behavior is enabled, each host thread will have its own independent default stream which will not synchronize with other streams in the same way the legacy default stream does. In such a situation the *legacy default stream example* will now exhibit the same synchronization behavior as the *non-blocking stream example*.

2.5.7. Explicit Synchronization

There are various ways to explicitly synchronize streams with each other.

`cudaDeviceSynchronize()` waits until all preceding commands in all streams of all host threads have completed.

`cudaStreamSynchronize()`takes a stream as a parameter and waits until all preceding commands in the given stream have completed. It can be used to synchronize the host with a specific stream, allowing other streams to continue executing on the device.

`cudaStreamWaitEvent()`takes a stream and an event as parameters (see *CUDA Events* for a description of events)and makes all the commands added to the given stream after the call to `cudaStreamWaitEvent()`delay their execution until the given event has completed.

`cudaStreamQuery()`provides applications with a way to know if all preceding commands in a stream have completed.

2.5.8. Implicit Synchronization

Two operations from different streams cannot run concurrently if any CUDA operation on the NULL stream is submitted in-between them, unless the streams are non-blocking streams (created with the `cudaStreamNonBlocking` flag).

Applications should follow these guidelines to improve their potential for concurrent kernel execution:

- All independent operations should be issued before dependent operations,
- Synchronization of any kind should be delayed as long as possible.

2.5.9. Miscellaneous and Advanced topics

2.5.9.1 Stream Prioritization

As mentioned previously, developers can assign priorities to CUDA streams. Prioritized streams need to be created using the `cudaStreamCreateWithPriority()` function. The function takes two parameters: the stream handle and the priority level. The general scheme is that lower numbers corre-

spond to higher priorities. The given priority range for a given device and context can be queried using the `cudaDeviceGetStreamPriorityRange()` function. The default priority of a stream is 0.

```
int minPriority, maxPriority;

// Query the priority range for the device
cudaDeviceGetStreamPriorityRange(&minPriority, &maxPriority);

// Create two streams with different priorities
// cudaStreamDefault indicates the stream should be created with default flags
// in other words they will be blocking streams with respect to the legacy
 ↪default stream
// One could also use the option `cudaStreamNonBlocking` here to create a non-
 ↪blocking streams
cudaStream_t stream1, stream2;
cudaStreamCreateWithPriority(&stream1, cudaStreamDefault, minPriority);  //
 ↪Lowest priority
cudaStreamCreateWithPriority(&stream2, cudaStreamDefault, maxPriority);  //
 ↪Highest priority
```

We should note that a priority of a stream is only a hint to the runtime and generally applies primarily to kernel launches, and may not be respected for memory transfers. Stream priorities will not preempt already executing work, or guarantee any specific execution order.

2.5.9.2 Introduction to CUDA Graphs with Stream Capture

CUDA streams allow programs to specify a sequence of operations, kernels or memory copies, in order. Using multiple streams and cross-stream dependencies with `cudaStreamWaitEvent`, an application can specify a full directed acyclic graph (DAG) of operations. Some applications may have a sequence or DAG of operations that needs to be run many times throughout execution.

For this situation, CUDA provides a feature known as CUDA graphs. This section introduces CUDA graphs and one mechanism of creating them called *stream capture*. A more detailed discussion of CUDA graphs is presented in *CUDA Graphs*. Capturing or creating a graph can help reduce latency and CPU overhead of repeatedly invoking the same chain of API calls from the host thread. Instead, the APIs to specify the graph operations can be called once, and then the resulting graph executed many times.

CUDA Graphs work in the following way:

i) The graph is *captured* by the application. This step is done once the first time the graph is executed. The graph can also be manually composed using the CUDA graph API.

ii) The graph is *instantiated*. This step is done one time, after the graph is captured. This step can set up all the various runtime structures needed to execute the graph, in order to make launching its components as fast as possible.

iii) In the remaining steps, the pre-instantiated graph is executed as many times as required. Since all the runtime structures needed to execute the graph operations are already in place, the CPU overheads of the graph execution are minimized.

Listing 2: The stages of capturing, instantiating and executing a simple linear graph using CUDA Graphs (from CUDA Developer Technical Blog, A. Gray, 2019)

```
#define N 500000 // tuned such that kernel takes a few microseconds

// A very lightweight kernel
__global__ void shortKernel(float * out_d, float * in_d){
    int idx=blockIdx.x*blockDim.x+threadIdx.x;
    if(idx<N) out_d[idx]=1.23*in_d[idx];
}

bool graphCreated=false;
cudaGraph_t graph;
cudaGraphExec_t instance;

// The graph will be executed NSTEP times
for(int istep=0; istep<NSTEP; istep++){
    if(!graphCreated){
        // Capture the graph
        cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);

        // Launch NKERNEL kernels
        for(int ikrnl=0; ikrnl<NKERNEL; ikrnl++){
            shortKernel<<<blocks, threads, 0, stream>>>(out_d, in_d);
        }

        // End the capture
        cudaStreamEndCapture(stream, &graph);

        // Instantiate the graph
        cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
        graphCreated=true;
    }

    // Launch the graph
    cudaGraphLaunch(instance, stream);

    // Synchronize the stream
    cudaStreamSynchronize(stream);
}
```

Much more detail on CUDA graph is provided in *CUDA Graphs*.

2.5.10. Summary of Asynchronous Execution

The key points of this section are:

- Asynchronous APIs allow us to express concurrent execution of tasks providing the way to express overlapping of various operations. The actual concurrency achieved is dependent on available hardware resources and compute-capabilities.
- The key abstractions in CUDA for asynchronous execution are streams, events and callback functions.

- Synchronization is possible at the event, stream and device level
- The default stream is a blocking stream which synchronizes with all other blocking streams, but does not synchronize with non-blocking streams
- The default stream behavior can be avoided using per-thread default streams via the `--default-stream per-thread` compiler option or the CUDA_API_PER_THREAD_DEFAULT_STREAM preprocessor macro.
- Streams can be created with different priorities, which are hints to the runtime and may not be respected for memory transfers.
- CUDA provides API functions to reduce, or overlap overheads of kernel launches and memory transfers such as CUDA Graphs, Batched Memory Transfers and Programmatic Dependent Kernel Launch.

2.6. Unified and System Memory

Heterogeneous systems have multiple physical memories where data can be stored. The host CPU has attached DRAM, and every GPU in a system has its own attached DRAM. Performance is best when data is resident in the memory of the processor accessing it. CUDA provides APIs to *explicitly manage memory placement*, but this can be verbose and complicate software design. CUDA provides features and capabilities aimed at easing allocation, placement, and migration of data between different physical memories.

The purpose of this chapter is to introduce and explain these features and what they mean to application developers for both functionality and performance. Unified memory has several different manifestations which depend upon the OS, driver version, and GPU used. This chapter will show how to determine which unified memory paradigm applies and how the features of unified memory behave in each. The later *chapter on unified memory* explains unified memory in more detail.

The following concepts will be defined and explained in this chapter:

- *Unified Virtual Address Space* - CPU memory and each GPU's memory have a distinct range within a single virtual address space
- *Unified Memory* - A CUDA feature that enables managed memory which can be automatically migrated between CPU and GPUs
  - *Limited Unified Memory* - A unified memory paradigm with some limitations
  - *Full Unified Memory* - Full support for unified memory features
  - *Full Unified Memory with Hardware Coherency* - Full support for unified memory using hardware capabilities
  - *Unified memory hints* - APIs to guide unified memory behavior for specific allocations
- *Page-locked Host Memory* - Non-pageable system memory, which is necessary for some CUDA operations
  - *Mapped memory* - A mechanism (different from unified memory) for accessing host memory directly from a kernel

Additionally, the following terms used when discussing unified and system memory are introduced here:

- *Heterogeneous Managed Memory* (HMM) - A feature of the Linux kernel that enables software coherency for full unified memory

- *Address Translation Services* (ATS) - A hardware feature, available when GPUs are connected to the CPU by the NVLink Chip-to-Chip (C2C) interconnect, which provides hardware coherency for full unified memory

2.6.1. Unified Virtual Address Space

A single virtual address space is used for all host memory and all global memory on all GPUs in the system within a single OS process. All memory allocations on the host and on all devices lie in this virtual address space. This is true whether allocations are made with CUDA APIs (e.g. `cudaMalloc`, `cudaMallocHost`) or with system allocation APIs (e.g. `new`, `malloc`, `mmap`). The CPU and each GPU has a unique range within the unified virtual address space.

This means:

- The location of any memory (that is, CPU or which GPU's memory it lies in) can be determined from the value of a pointer using `cudaPointerGetAttributes()`
- The `cudaMemcpyKind` parameter of `cudaMemcpy*()` can be set to `cudaMemcpyDefault` to automatically determine the copy type from the pointers

2.6.2. Unified Memory

Unified memory is a CUDA memory feature which allows memory allocations called *managed memory* to be accessed from code running on either the CPU or the GPU. Unified memory was shown in *the intro to CUDA in C++*. Unified memory is available on all systems supported by CUDA.

On some systems, managed memory must be explicitly allocated. Managed memory can be explicitly allocated in CUDA in a few different ways:

- The CUDA API `cudaMallocManaged`
- The CUDA API `cudaMallocFromPoolAsync` with a pool created with `allocType` set to `cudaMemAllocationTypeManaged`
- Global variables with the `__managed__` specifier (see *Memory Space Specifiers*)

On systems with *HMM* or *ATS*, all system memory is implicitly managed memory, regardless of how it is allocated. No special allocation is needed.

2.6.2.1 Unified Memory Paradigms

The features and behavior of unified memory vary between operating systems, kernel versions on Linux, GPU hardware, and the GPU-CPU interconnect. The form of unified memory available can be determined by using `cudaDeviceGetAttribute` to query a few attributes:

- `cudaDevAttrConcurrentManagedAccess` - 1 for full unified memory support, 0 for limited support
- `cudaDevAttrPageableMemoryAccess` - 1 means all system memory is fully-supported unified memory, 0 means only memory explicitly allocated as managed memory is fully-supported unified memory
- `cudaDevAttrPageableMemoryAccessUsesHostPageTables` - Indicates the mechanism of CPU/GPU coherence: 1 is hardware, 0 is software.

Figure 21 illustrates how to determine the unified memory paradigm visually and is followed by a *code sample* implementing the same logic.

There are four paradigms of unified memory operation:

- *Full support for explicit managed memory allocations*
- *Full support for all allocations with software coherence*
- *Full support for all allocations with hardware coherence*
- *Limited unified memory support*

When full support is available, it can either require explicit allocations, or all system memory may implicitly be unified memory. When all memory is implicitly unified, the coherence mechanism can either be software or hardware. Windows and some Tegra devices have limited support for unified memory.

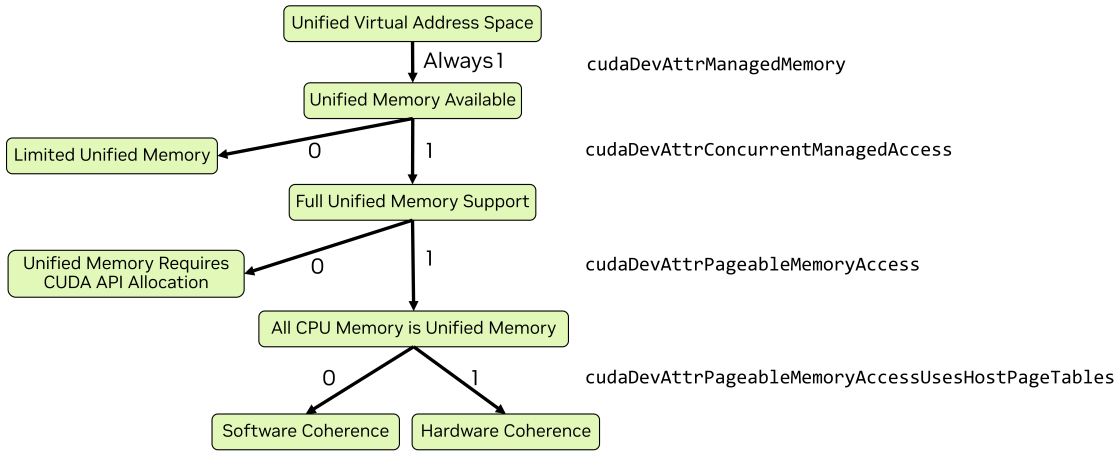


Figure 21: All current GPUs use a unified virtual address space and have unified memory available. When `cudaDevAttrConcurrentManagedAccess` is 1, full unified memory support is available, otherwise only limited support is available. When full support is available, if `cudaDevAttrPageableMemoryAccess` is also 1, then all system memory is unified memory. Otherwise, only memory allocated with CUDA APIs (such as `cudaMallocManaged`) is unified memory. When all system memory is unified, `cudaDevAttrPageableMemoryAccessUsesHostPageTables` indicates whether coherence is provided by hardware (when value is 1) or software (when value is 0).

Table 3 shows the same information as Figure 21 as a table with links to the relevant sections of this chapter and more complete documentation in a later section of this guide.

Table 3: Overview of Unified Memory Paradigms

Unified Memory Paradigm	Device Attributes	Full Documentation
Limited unified memory support	`cudaDevAttrConcurrentMana` is 0	*Unified Memory on Windows, WSL, and Tegra* CUDA for Tegra Memory Management Unified memory on Tegra
Full support for explicit managed memory allocations	`cudaDevAttrPageableMemory` is 0 and `cudaDevAttrConcurrentManagedAccess` is 1	*Unified Memory on Devices with only CUDA Managed Memory Support*
Full support for all allocations with software coherence	`cudaDevAttrPageableMemory` is 0 and `cudaDevAttrPageableMemoryAccess` is 1 and `cudaDevAttrConcurrentManagedAccess` is 1	*Unified Memory on Devices with Full CUDA Unified Memory Support*
Full support for all allocations with hardware coherence	`cudaDevAttrPageableMemory` is 1 and `cudaDevAttrPageableMemoryAccess` is 1 and `cudaDevAttrConcurrentManagedAccess` is 1	*Unified Memory on Devices with Full CUDA Unified Memory Support*

2.6.2.1.1 Unified Memory Paradigm: Code Example

The following code example demonstrates querying the device attributes and determining the unified memory paradigm, following the logic of Figure 21, for each GPU in a system.

```
void queryDevices()
{
    int numDevices = 0;
    cudaGetDeviceCount(&numDevices);
    for(int i=0; i<numDevices; i++)
    {
        cudaSetDevice(i);
```

(continues on next page)

(continued from previous page)

```
        cudaInitDevice(0, 0, 0);
        int deviceId = i;

        int concurrentManagedAccess = -1;
        cudaDeviceGetAttribute (&concurrentManagedAccess,
↪cudaDevAttrConcurrentManagedAccess, deviceId);
        int pageableMemoryAccess = -1;
        cudaDeviceGetAttribute (&pageableMemoryAccess,
↪cudaDevAttrPageableMemoryAccess, deviceId);
        int pageableMemoryAccessUsesHostPageTables = -1;
        cudaDeviceGetAttribute (&pageableMemoryAccessUsesHostPageTables,
↪cudaDevAttrPageableMemoryAccessUsesHostPageTables, deviceId);

        printf("Device %d has ", deviceId);
        if(concurrentManagedAccess){
            if(pageableMemoryAccess){
                printf("full unified memory support");
                if( pageableMemoryAccessUsesHostPageTables)
                    { printf(" with hardware coherency\n");  }
                else
                    { printf(" with software coherency\n"); }
            }
            else
                { printf("full unified memory support for CUDA-made managed
↪allocations\n"); }
        }
        else
        {   printf("limited unified memory support: Windows, WSL, or Tegra\n
↪");  }
    }
}
```

2.6.2.2 Full Unified Memory Feature Support

Most Linux systems have full unified memory support. If device attribute `cudaDevAttrPageableMemoryAccess` is 1, then all system memory, whether allocated by CUDA APIs or system APIs, operates as unified memory with full feature support. This includes file-backed memory allocations created with `mmap`.

If `cudaDevAttrPageableMemoryAccess` is 0, then only memory allocated as managed memory by CUDA behaves as unified memory. Memory allocated with system APIs is not managed and is not necessarily accessible from GPU kernels.

In general, for unified allocations with full support:

- Managed memory is usually allocated in the memory space of the processor where it is first touched
- Managed memory is usually migrated when it is used by a processor other than the processor where it currently resides
- Managed memory is migrated or accessed at the granularity of memory pages (software coherence) or cache lines (hardware coherence)

- Oversubscription is allowed: an application may allocate more managed memory than is physically available on the GPU

Allocation and migration behavior can deviate from the above. This can by influenced the programmer using *hints and prefetches*. Full coverage of full unified memory support can be found in *Unified Memory on Devices with Full CUDA Unified Memory Support*.

2.6.2.2.1 Full Unified Memory with Hardware Coherency

On hardware such as Grace Hopper and Grace Blackwell, where an NVIDIA CPU is used and the interconnect between the CPU and GPU is NVLink Chip-to-Chip (C2C), address translation services (ATS) are available. `cudaDevAttrPageableMemoryAccessUsesHostPageTables` is 1 when ATS is available.

With ATS, in addition to full unified memory support for all host allocations:

- Managed allocations resident on the GPU (e.g. `cudaMallocManaged`) can be accessed from the CPU without migration(`cudaDevAttrDirectManagedMemAccessFromHost` will be 1)
- The link between CPU and GPU supports native atomics (`cudaDevAttrHostNativeAtomicSupported` will be 1)
- Hardware support for coherence can improve performance compared to software coherence

ATS provides all capabilities of *HMM*. When ATS is available, HMM is automatically disabled. Further discussion of hardware vs. software coherency is found in *CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency*.

Note

Hardware coherency does not enable access to GPU-only allocations such as those made with `cudaMalloc` from the host.

2.6.2.2.2 HMM - Full Unified Memory with Software Coherency

Heterogeneous Memory Management (HMM) is a feature available on Linux operating systems (with appropriate kernel versions) which enables software-coherent *full unified memory support*. Heterogeneous memory management brings some of the capabilities and convenience provided by ATS to PCIe-connected GPUs.

On Linux with at least Linux Kernel 6.1.24, 6.2.11, or 6.3 or later, heterogeneous memory management (HMM) may be available. The following command can be used to find if the addressing mode is `HMM`.

```
$ nvidia-smi -q | grep Addressing
Addressing Mode  : HMM
```

When HMM is available, *full unified memory* is supported and all system allocations are implicitly unified memory. If a system also has *ATS*, HMM is disabled and ATS is used, since ATS provides all the capabilities of HMM and more.

2.6.2.3 Limited Unified Memory Support

On Windows, including Windows Subsystem for Linux (WSL), and on some Tegra systems, a limited subset of unified memory functionality is available. On these systems, managed memory is available, but migration between CPU and GPUs behaves differently.

- Managed memory is first allocated in the CPU's physical memory

- Managed memory is migrated in larger granularity than virtual memory pages
- Managed memory is migrated to the GPU when the GPU begins executing
- The CPU must not access managed memory while the GPU is active
- Managed memory is migrated back to the CPU when the GPU is synchronized
- Oversubscription of GPU memory is not allowed
- Only memory explicitly allocated by CUDA as managed memory is unified

Full coverage of this paradigm can be found in *Unified Memory on Windows, WSL, and Tegra*.

2.6.2.4 Memory Advise and Prefetch

The programmer can provide hints to the NVIDIA Driver managing unified memory to help it maximize application performance. The CUDA API `cudaMemAdvise` allows the programmer to specify properties of allocations that affect where they are placed and whether or not the memory is migrated when accessed from another device.

`cudaMemPrefetchAsync` allows the programmer to suggest an asynchronous migration of a specific allocation to a different location be started. A common use is starting the transfer of data a kernel will use before the kernel is launched. This enables the copy of data to occur while other GPU kernels are executing.

The section on *Performance Hints* covers the different hints that can be passed to `cudaMemAdvise` and shows examples of using `cudaMemPrefetchAsync`.

2.6.3. Page-Locked Host Memory

In *introductory code examples*, `cudaMallocHost` was used to allocate memory on the CPU. This allocates *page-locked* memory (also known as *pinned* memory) on the host. Host allocations made through traditional allocation mechanisms like `malloc`, `new`, or `mmap` are not page-locked, which means they may be swapped to disk or physically relocated by the operating system.

Page-locked host memory is required for *asynchronous copies between the CPU and GPU*. Page-locked host memory also improves performance of synchronous copies. Page-locked memory can be *mapped* to the GPU for direct access from GPU kernels.

The CUDA runtime provides APIs to allocate page-locked host memory or to page-lock existing allocations:

- `cudaMallocHost` allocates page-locked host memory
- `cudaHostAlloc` defaults to the same behavior as `cudaMallocHost`, but also takes flags to specify other memory parameters
- `cudaFreeHost` frees memory allocated with `cudaMallocHost` or `cudaHostAlloc`
- `cudaHostRegister` page-locks a range of existing memory allocated outside the CUDA API, such as with `malloc` or `mmap`

`cudaHostRegister` enables host memory allocated by 3rd party libraries or other code outside of a developer's control to be page-locked so that it can be used in asynchronous copies or mapped.

Note

Page-locked host memory can be used for asynchronous copies and mapped-memory by all GPUs in the system.

Page-locked host memory is not cached on non I/O coherent Tegra devices. Also, `cudaHostRegister()` is not supported on non I/O coherent Tegra devices.

2.6.3.1 Mapped Memory

On systems with *HMM* or *ATS*, all host memory is directly accessible from the GPU using the host pointers. When ATS or HMM are not available, host allocations can be made accessible to the GPU by *mapping* the memory into the GPU's memory space. Mapped memory is always page-locked.

The code examples which follow will illustrate the following array copy kernel operating directly on mapped host memory.

```
__global__ void copyKernel(float* a, float* b)
{
        int idx = threadIdx.x + blockDim.x * blockIdx.x;
        a[idx] = b[idx];
}
```

While mapped memory may be useful in some cases where certain data which is not copied to the GPU needs to be accessed from a kernel, accessing mapped memory in a kernel requires transactions across the CPU-GPU interconnect, PCIe, or NVLink C2C. These operations have higher latency and lower bandwidth compared to accessing device memory. Mapped memory should not be considered a performant alternative to *unified memory* or *explicit memory management* for the majority of a kernel's memory needs.

2.6.3.1.1 cudaMallocHost and cudaHostAlloc

Host memory allocated with `cudaMallocHost` or `cudaHostAlloc` is automatically mapped. The pointers returned by these APIs can be directly used in kernel code to access the memory on the host. The host memory is accessed over the CPU-GPU interconnect.

cudaMallocHost

```
void usingMallocHost() {
  float* a = nullptr;
  float* b = nullptr;

  CUDA_CHECK(cudaMallocHost(&a, vLen*sizeof(float)));
  CUDA_CHECK(cudaMallocHost(&b, vLen*sizeof(float)));

  initVector(b, vLen);
  memset(a, 0, vLen*sizeof(float));

  int threads = 256;
  int blocks = vLen/threads;
  copyKernel<<<blocks, threads>>>(a, b);
  CUDA_CHECK(cudaGetLastError());
  CUDA_CHECK(cudaDeviceSynchronize());

  printf("Using cudaMallocHost: ");
```

(continues on next page)

(continued from previous page)

```
  checkAnswer(a,b);
}
```

cudaHostAlloc

```
void usingCudaHostAlloc() {
  float* a = nullptr;
  float* b = nullptr;

  CUDA_CHECK(cudaHostAlloc(&a, vLen*sizeof(float), cudaHostAllocMapped));
  CUDA_CHECK(cudaHostAlloc(&b, vLen*sizeof(float), cudaHostAllocMapped));

  initVector(b, vLen);
  memset(a, 0, vLen*sizeof(float));

  int threads = 256;
  int blocks = vLen/threads;
  copyKernel<<<blocks, threads>>>(a, b);
  CUDA_CHECK(cudaGetLastError());
  CUDA_CHECK(cudaDeviceSynchronize());

  printf("Using cudaHostAlloc: ");
  checkAnswer(a, b);
}
```

2.6.3.1.2 cudaHostRegister

When ATS and HMM are not available, allocations made by system allocators can still be mapped for access directly from GPU kernels using `cudaHostRegister`. Unlike memory created with CUDA APIs, however, the memory cannot be accessed from the kernel using the host pointer. A pointer in the device's memory region must be obtained using `cudaHostGetDevicePointer()`, and that pointer must be used for accesses in kernel code.

```
void usingRegister() {
  float* a = nullptr;
  float* b = nullptr;
  float* devA = nullptr;
  float* devB = nullptr;

  a = (float*)malloc(vLen*sizeof(float));
  b = (float*)malloc(vLen*sizeof(float));
  CUDA_CHECK(cudaHostRegister(a, vLen*sizeof(float), 0 ));
  CUDA_CHECK(cudaHostRegister(b, vLen*sizeof(float), 0  ));

  CUDA_CHECK(cudaHostGetDevicePointer((void**)&devA, (void*)a, 0));
  CUDA_CHECK(cudaHostGetDevicePointer((void**)&devB, (void*)b, 0));

  initVector(b, vLen);
  memset(a, 0, vLen*sizeof(float));

  int threads = 256;
```

(continues on next page)

(continued from previous page)

```
  int blocks = vLen/threads;
  copyKernel<<<blocks, threads>>>(devA, devB);
  CUDA_CHECK(cudaGetLastError());
  CUDA_CHECK(cudaDeviceSynchronize());

  printf("Using cudaHostRegister: ");
  checkAnswer(a, b);
}
```

2.6.3.1.3 Comparing Unified Memory and Mapped Memory

Mapped memory makes CPU memory accessible from the GPU, but does not guarantee that all types of access, for example atomics, are supported on all systems. Unified memory guarantees that all access types are supported.

Mapped memory remains in CPU memory, which means all GPU accesses must go through the connection between the CPU and GPU: PCIe or NVLink. Latency of accesses made across these links are significantly higher than access to GPU memory, and total available bandwidth is lower. As such, using mapped memory for all kernel memory accesses is unlikely to fully utilize GPU computing resources.

Unified memory is most often migrated to the physical memory of the processor accessing it. After the first migration, repeated access to the same memory page or cache line by a kernel can utilize the full GPU memory bandwidth.

Note

Mapped memory has also been referred to as *zero-copy* memory in previous documents.

Prior to all CUDA applications using a *unified virtual address space*, additional APIs were needed to enable memory mapping (`cudaSetDeviceFlags` with `cudaDeviceMapHost`). These APIs are no longer needed.

Atomic functions (see *Atomic Functions*) operating on mapped host memory are not atomic from the point of view of the host or other GPUs.

CUDA runtime requires that 1-byte, 2-byte, 4-byte, 8-byte, and 16-byte naturally aligned loads and stores to host memory initiated from the device are preserved as single accesses from the point of view of the host and other devices. On some platforms, atomics to memory may be broken by the hardware into separate load and store operations. These component load and store operations have the same requirements on preservation of naturally aligned accesses. The CUDA runtime does not support a PCI Express bus topology where a PCI Express bridge splits 8-byte naturally aligned operations and NVIDIA is not aware of any topology that splits 16-byte naturally aligned operations.

2.6.4. Summary

- On Linux platforms with heterogeneous memory management (HMM) or address translation services (ATS), all system-allocated memory is managed memory
- On Linux platforms without HMM or ATS, on Tegra processors, and on all Windows platforms, managed memory must be allocated using CUDA:
  - `cudaMallocManaged` or

- `cudaMallocFromPoolAsync` with a pool created with `allocType=cudaMemAllocationTypeManaged`
- Global variables with `__managed__` specifier

- On Windows and Tegra processors, unified memory has limitations
- On NVLINK C2C connected systems with ATS, device memory allocated with `cudaMallocManaged` can be directly accessed from the CPU or other GPUs

2.7. NVCC: The NVIDIA CUDA Compiler

The NVIDIA CUDA Compiler `nvcc` is a toolchain from NVIDIA for compiling CUDA C/C++ as well as PTX code. The toolchain is part of the CUDA Toolkit and consists of several tools, including the compiler, linker, and the PTX and *Cubin* assemblers. The top-level `nvcc` tool coordinates the compilation process, invoking the appropriate tool for each stage of compilation.

`nvcc` drives offline compilation of CUDA code, in contrast to online or Just-in-Time (JIT) compilation driven by the CUDA runtime compiler nvrtc.

This chapter covers the most common uses and details of `nvcc` needed for building applications. Full coverage of `nvcc` is found in the nvcc documentation.

2.7.1. CUDA Source Files and Headers

Source files compiled with `nvcc` may contain a combination of host code, which executes on the CPU, and device code that executes on the GPU. `nvcc` accepts the common C/C++ source file extensions `.c`, `.cpp`, `.cc`, `.cxx` for host-only code and `.cu` for files that contain device code or a mix of host and device code. Headers containing device code typically adopt the `.cuh` extension to distinguish them from host-only code headers `.h`, `.hpp`, `.hh`, `.hxx`, etc.

File Extension	Description	Content
`.c`	C source file	Host-only code
`.cpp`, `.cc`, `.cxx`	C++ source file	Host-only code
`.h`, `.hpp`, `.hh`, `.hxx`	C/C++ header file	Device code, host code, mix of host/device code
`.cu`	CUDA source file	Device code, host code, mix of host/device code
`.cuh`	CUDA header file	Device code, host code, mix of host/device code

2.7.2. NVCC Compilation Workflow

In the initial phase, `nvcc` separates the device code from the host code and dispatches their compilation to the GPU and the host compilers, respectively.

To compile the host code, the CUDA compiler `nvcc` requires a compatible host compiler to be available. The CUDA Toolkit defines the host compiler support policy for Linux and Windows platforms.

Files containing only host code can be built using either `nvcc` or the host compiler directly. The resulting object files can be combined with object files from `nvcc` which contain GPU code at link-time.

The GPU compiler compiles the C/C++ device code to PTX assembly code. The GPU compiler is run for each virtual machine instruction set architecture (e.g. `compute_90`) specified in the compilation command line.
Individual PTX code is then passed to the `ptxas` tool, which generates *Cubin* for the target hardware ISAs. The hardware ISA is identified by its *SM version*.
It is possible to embed multiple PTX and Cubin targets into a single binary *Fatbin* container within an application or library so that a single binary can support multiple virtual and target hardware ISAs.

The invocation and coordination of the tools described above are done automatically by `nvcc`. The `-v` option can be used to display the full compilation workflow and tool invocation. The `-keep` option can be used to save the intermediate files generated during the compilation in the current directory or in the directory specified by `--keep-dir` instead.

The following example illustrates the compilation workflow for a CUDA source file `example.cu`:

```
// ----- example.cu -----
#include <stdio.h>
__global__ void kernel() {
    printf("Hello from kernel\n");
}

void kernel_launcher() {
    kernel<<<1, 1>>>();
    cudaDeviceSynchronize();
}

int main() {
    kernel_launcher();
    return 0;
}
```

`nvcc` basic compilation workflow:

`nvcc` compilation workflow with multiple PTX and Cubin architectures:

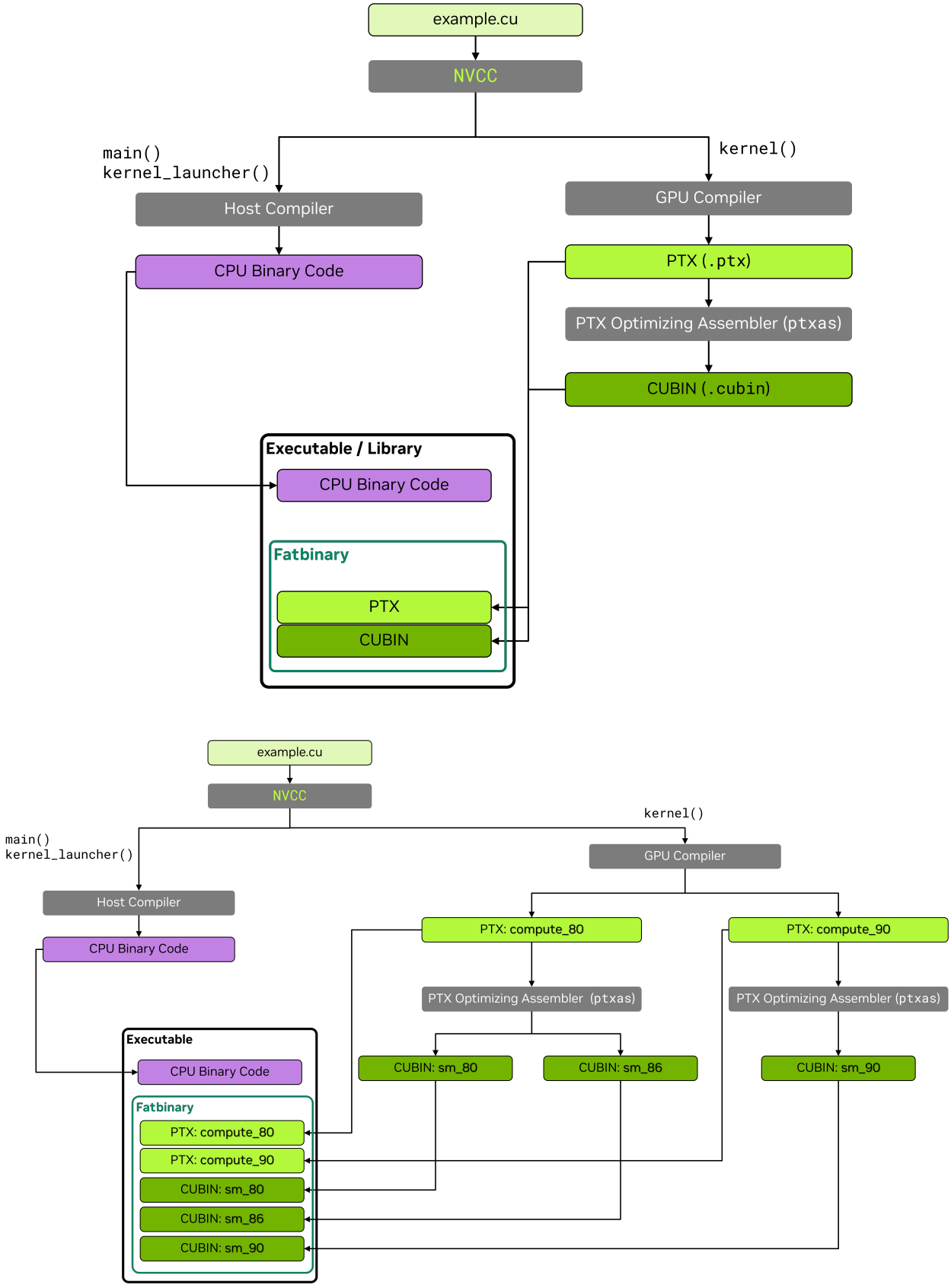
A more detailed description of the `nvcc` compilation workflow can be found in the compiler documentation.

2.7.3. NVCC Basic Usage

The basic command to compile a CUDA source file with `nvcc` is:

```
nvcc <source_file>.cu -o <output_file>
```

`nvcc` accepts common compiler flags used for specifying include directories `-I <path>` and library paths `-L <path>`, linking against other libraries `-l<library>`, and defining macros `-D<macro>=<value>`.


example.cu
NVCC
main()
kernel_launcher()
kernel()
Host Compiler
GPU Compiler
CPU Binary Code
PTX (.ptx)
PTX Optimizing Assembler (ptxas)
CUBIN (.cubin)
Executable / Library
CPU Binary Code
Fatbinary
PTX
CUBIN
example.cu
NVCC
kernel()
main()
kernel_launcher()
GPU Compiler
Host Compiler
PTX: compute_80
PTX: compute_90
CPU Binary Code
PTX Optimizing Assembler (ptxas)
PTX Optimizing Assembler (ptxas)
Executable
CUBIN: sm_80
CUBIN: sm_86
CUBIN: sm_90
CPU Binary Code
Fatbinary
PTX: compute_80
PTX: compute_90
CUBIN: sm_80
CUBIN: sm_86
CUBIN: sm_90

```
nvcc example.cu -I path_to_include/ -L path_to_library/ -lcublas -o <output_
↪file>
```

2.7.3.1 NVCC PTX and Cubin Generation

By default, nvcc generates PTX and Cubin for the earliest GPU architecture (lowest compute_XY and sm_XY version) supported by the CUDA Toolkit to maximize compatibility.

- The -arch option can be used to generate PTX and Cubin for a specific GPU architecture.
- The -gencode option can be used to generate PTX and Cubin for multiple GPU architectures.

The complete list of supported virtual and real GPU architectures can be obtained by passing the --list-gpu-code and --list-gpu-arch flags respectively, or by referring to the Virtual Architecture List and the GPU Architecture List sections within the nvcc documentation.

```
nvcc --list-gpu-code # list all supported real GPU architectures
nvcc --list-gpu-arch # list all supported virtual GPU architectures
```

```
nvcc example.cu -arch=compute_<XY> # e.g. -arch=compute_80 for NVIDIA Ampere
↪GPUs and later
                                   # PTX-only, GPU forward compatible

nvcc example.cu -arch=sm_<XY>      # e.g. -arch=sm_80 for NVIDIA Ampere GPUs
↪and later
                                   # PTX and Cubin, GPU forward compatible

nvcc example.cu -arch=native       # automatically detects and generates Cubin
↪for the current GPU
                                   # no PTX, no GPU forward compatibility

nvcc example.cu -arch=all          # generate Cubin for all supported GPU
↪architectures
                                   # also includes the latest PTX for GPU
↪forward compatibility

nvcc example.cu -arch=all-major    # generate Cubin for all major supported
↪GPU architectures, e.g. sm_80, sm_90,
                                   # also includes the latest PTX for GPU
↪forward compatibility
```

More advanced usage allows PTX and Cubin targets to be specified individually:

```
# generate PTX for virtual architecture compute_80 and compile it to Cubin for
↪real architecture sm_86, keep compute_80 PTX
nvcc example.cu -arch=compute_80 -gpu-code=sm_86,compute_80 # (PTX and Cubin)

# generate PTX for virtual architecture compute_80 and compile it to Cubin for
↪real architecture sm_86, sm_89
nvcc example.cu -arch=compute_80 -gpu-code=sm_86,sm_89    # (no PTX)
nvcc example.cu -gencode=arch=compute_80,code=sm_86,sm_89 # same as above

# (1) generate PTX for virtual architecture compute_80 and compile it to Cubin
↪for real architecture sm_86, sm_89
```

(continues on next page)

(continued from previous page)

```
# (2) generate PTX for virtual architecture compute_90 and compile it to Cubin
 ↪for real architecture sm_90
nvcc example.cu -gencode=arch=compute_80,code=sm_86,sm_89 -
 ↪gencode=arch=compute_90,code=sm_90
```

The full reference of nvcc command-line options for steering GPU code generation can be found in the nvcc documentation.

2.7.3.2 Host Code Compilation Notes

Compilation units, namely a source file and its headers, that do not contain device code or symbols can be compiled directly with a host compiler. If any compilation unit uses CUDA runtime API functions, the application must be linked with the CUDA runtime library. The CUDA runtime is available as both a static and a shared library, `libcudart_static` and `libcudart`, respectively. By default, nvcc links against the static CUDA runtime library. To use the shared library version of the CUDA runtime, pass the flag `--cudart=shared` to nvcc on the compile or link command.

nvcc allows the host compiler used for host functions to be specified via the `-ccbin <compiler>` argument. The environment variable `NVCC_CCBIN` can also be defined to specify the host compiler used by nvcc. The `-Xcompiler` argument to nvcc passes through arguments to the host compiler. For example, in the example below, the `-O3` argument is passed to the host compiler by nvcc.

```
nvcc example.cu -ccbin=clang++

export NVCC_CCBIN='gcc'
nvcc example.cu -Xcompiler=-O3
```

2.7.3.3 Separate Compilation of GPU Code

nvcc defaults to *whole-program compilation*, which expects all GPU code and symbols to be present in the compilation unit that uses them. CUDA device functions may call device functions or access device variables defined in other compilation units, but either the `-rdc=true` or its alias the `-dc` flag must be specified on the nvcc command line to enable linking of device code from different compilation units. The ability to link device code and symbols from different compilation units is called *separate compilation*.

Separate compilation allows more flexible code organization, can improve compile time, and can lead to smaller binaries. Separate compilation may involve some build-time complexity compared to whole-program compilation. Performance can be affected by the use of device code linking, which is why it is not used by default. *Link-Time Optimization (LTO)* can help reduce the performance overhead of separate compilation.

Separate compilation requires the following conditions:

- Non-`const` device variables defined in one compilation unit must be referred to with the `extern` keyword in other compilation units.
- All `const` device variables must be defined and referred to with the `extern` keyword.
- All CUDA source files `.cu` must be compiled with the `-dc` or `-rdc=true` flags.

Host and device functions have external linkage by default and do not require the `extern` keyword. Note that starting from CUDA 13, `__global__` functions and `__managed__`/`__device__`/`__constant__` variables have internal linkage by default.

In the following example, `definition.cu` defines a variable and a function, while `example.cu` refers to them. Both files are compiled separately and linked into the final binary.

```
// ----- definition.cu -----
extern __device__ int device_variable = 5;
__device__        int device_function() { return 10; }
```

```
// ----- example.cu -----
extern __device__ int  device_variable;
__device__        int device_function();

__global__ void kernel(int* ptr) {
    device_variable = 0;
    *ptr            = device_function();
}
```

```
nvcc -dc definition.cu -o definition.o
nvcc -dc example.cu    -o example.o
nvcc definition.o example.o -o program
```

2.7.4. Common Compiler Options

This section presents the most relevant compiler options that can be used with `nvcc`, covering language features, optimization, debugging, profiling, and build aspects. The full description of all options can be found in the nvcc documentation.

2.7.4.1 Language Features

`nvcc` supports the C++ core language features, from C++03 to *C++23 Language Features*. The `-std` flag can be used to specify the language standard to use:

- `--std={c++03|c++11|c++14|c++17|c++20|c++23}`

In addition, `nvcc` supports the following language extensions:

- `-restrict`: Assert that all kernel pointer parameters are *restrict* pointers.
- `-extended-lambda`: Allow `__host__`, `__device__` annotations in lambda declarations.
- `-expt-relaxed-constexpr`: (Experimental flag) Allow host code to invoke `__device__ constexpr` functions, and device code to invoke `__host__ constexpr` functions.

More detail on these features can be found in the *extended lambda* and *constexpr* sections.

2.7.4.2 Debugging Options

`nvcc` supports the following options to generate debug information:

- `-g`: Generate debug information for host code. `gdb/lldb` and similar tools rely on such information for host code debugging.
- `-G`: Generate debug information for device code. cuda-gdb relies on such information for device-code debugging. The flag also defines the `__CUDACC_DEBUG__` macro.
- `-lineinfo`: Generate line-number information for device code. This option does not affect execution performance and is useful in conjunction with the compute-sanitizer tool to trace the kernel execution.

nvcc uses the highest optimization level `-O3` for GPU code by default. The debug flag `-G` prevents some compiler optimizations, and so debug code is expected to have lower performance than non-debug code. The `-DNDEBUG` flag can be defined to disable runtime assertions, as these can also slow down execution.

2.7.4.3 Optimization Options

nvcc provides many options for optimizing performance. This section aims to provide a brief survey of some of the options available that developers may find useful, as well as links to further information. Complete coverage can be found in the nvcc documentation.

- `-Xptxas` passes arguments to the PTX assembler tool `ptxas`. The `nvcc` documentation provides a list of useful arguments for `ptxas`. For example, `-Xptxas=-maxrregcount=N` specifies the maximum number of registers to use, per thread.
- `-extra-device-vectorization`: Enables more aggressive device code vectorization.
- `--apply-controls=/path/to/file` passes an advanced controls file (ACF) into nvcc and ptxas. This file changes the default compilation behavior, and makes it more targeted for a specific workload. Using an advanced controls file may cause compilation failure or incorrect runtime execution. Use at your own risk. See the CompileIQ Github page for more information on how to generate advanced controls files.
- Additional flags which provide fine-grained control over floating point behavior are covered in the *Floating-Point Computation* section and in the nvcc documentation.

The following flags get output from the compiler which can be useful in more advanced code optimization:

- `-res-usage`: Print resource usage report after compilation. It includes the number of registers, shared memory, constant memory, and local memory allocated for each kernel function.
- `-opt-info=inline`: Print information about inlined functions.
- `-Xptxas=-warn-lmem-usage`: Warn if local memory is used.
- `-Xptxas=-warn-spills`: Warn if registers are spilled to local memory.

2.7.4.4 Link-Time Optimization (LTO)

Separate compilation can result in lower performance than whole-program compilation due to limited cross-file optimization opportunities. Link-Time Optimization (LTO) addresses this by performing optimizations across separately compiled files at link time, at the cost of increased compilation time. LTO can recover much of the performance of whole-program compilation while maintaining the flexibility of separate compilation.

nvcc requires the `-dlto` flag or `lto_<SM version>` link-time optimization targets to enable LTO:

```
nvcc -dc -dlto -arch=sm_100 definition.cu -o definition.o
nvcc -dc -dlto -arch=sm_100 example.cu    -o example.o
nvcc -dlto definition.o example.o -o program
```

```
nvcc -dc -arch=lto_100 definition.cu -o definition.o
nvcc -dc -arch=lto_100 example.cu    -o example.o
nvcc -dlto definition.o example.o -o program
```

2.7.4.5 Profiling Options

It is possible to directly profile a CUDA application using the Nsight Compute and Nsight Systems tools without the need for additional flags during the compilation process. However, additional information which can be generated by `nvcc` can assist profiling by correlating source files with the generated code:

- `-lineinfo`: Generate line-number information for device code; this allows viewing the source code in the profiling tools. Profiling tools require the original source code to be available in the same location where the code was compiled.
- `-src-in-ptx`: Keep the original source code in the PTX, avoiding the limitations of `-lineinfo` mentioned above. Requires `-lineinfo`.

2.7.4.6 Fatbin Compression

`nvcc` compresses the *fatbins* stored in application or library binaries by default. Fatbin compression can be controlled using the following options:

- `-no-compress`: Disable the compression of the fatbin.
- `--compress-mode={default|size|speed|balance|none}`: Set the compression mode. `speed` focuses on fast decompression time, while `size` aims at reducing the fatbin size. `balance` provides a trade-off between speed and size. The default mode is `speed`. `none` disables compression.

2.7.4.7 Compiler Performance Controls

`nvcc` provides options to analyze and accelerate the compilation process itself:

- `-t <N>`: The number of CPU threads used to parallelize the compilation of a single compilation unit for multiple GPU architectures.
- `-split-compile <N>`: The number of CPU threads used to parallelize the optimization phase.
- `-split-compile-extended <N>`: More aggressive form of split compilation. Requires link-time optimization.
- `-Ofc <N>`: Level of device code compilation speed.
- `-time <filename>`: Generate a comma-separated value (CSV) table with the time taken by each compilation phase.
- `-fdevice-time-trace`: Generate a time trace for device code compilation.

Chapter 3. Advanced CUDA

3.1. Advanced CUDA APIs and Features

This section will cover use of more advanced CUDA APIs and features. These topics cover techniques or features that do not usually require CUDA kernel modifications, but can still influence, from the host-side, application-level behavior, both in terms of GPU work execution and performance as well as CPU-side performance.

3.1.1. cudaLaunchKernelEx

When the *triple chevron notation* was introduced in first versions of, the *Kernel Configuration* of a kernel had only four programmable parameters:

- thread block dimensions
- grid dimensions
- dynamic shared-memory (optional, 0 if unspecified)
- stream (default stream used if unspecified)

Some CUDA features can benefit from additional attributes and hints provided with a kernel launch. The `cudaLaunchKernelEx` enables a program to set the above mentioned execution configuration parameters via the `cudaLaunchConfig_t` structure. In addition, the `cudaLaunchConfig_t` structure allows the program to pass in zero or more `cudaLaunchAttributes` to control or suggest other parameters for the kernel launch. For example, the `cudaLaunchAttributePreferredSharedMemoryCarveout` discussed later in this chapter (see *Configuring L1/Shared Memory Balance*) is specified using `cudaLaunchKernelEx`. The `cudaLaunchAttributeClusterDimension` attribute, discussed later in this chapter, is used to specify the desired cluster size for the kernel launch.

The complete list of supported attributes and their meaning is captured in the CUDA Runtime API Reference Documentation.

3.1.2. Launching Clusters:

Thread block clusters, introduced in previous sections, are an optional level of thread block organization available in compute capability 9.0 and higher which enable applications to guarantee that thread blocks of a cluster are simultaneously executed on single GPC. This enables larger groups of threads than those that fit in a single SM to exchange data and synchronize with each other.

Section Section 2.1.10.1 showed how a kernel which uses clusters can be specified and launched using triple chevron notation. In this section, the `__cluster_dims__` annotation was used to specify the

dimensions of the cluster which must be used to launch the kernel. When using triple chevron notation, the size of the clusters is determined implicitly.

3.1.2.1 Launching with Clusters using cudaLaunchKernelEx

Unlike *launching kernels using clusters with triple chevron notation*, the size of the thread block cluster can be configured on a per-launch basis. The code example below shows how to launch a cluster kernel using `cudaLaunchKernelEx`.

```
// Kernel definition
// No compile time attribute attached to the kernel
__global__ void cluster_kernel(float *input, float* output)
{

}

int main()
{
    float *input, *output;
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks(N / threadsPerBlock.x, N / threadsPerBlock.y);

    // Kernel invocation with runtime cluster size
    {
        cudaLaunchConfig_t config = {0};
        // The grid dimension is not affected by cluster launch, and is still
↪enumerated
        // using number of blocks.
        // The grid dimension should be a multiple of cluster size.
        config.gridDim = numBlocks;
        config.blockDim = threadsPerBlock;

        cudaLaunchAttribute attribute[1];
        attribute[0].id = cudaLaunchAttributeClusterDimension;
        attribute[0].val.clusterDim.x = 2; // Cluster size in X-dimension
        attribute[0].val.clusterDim.y = 1;
        attribute[0].val.clusterDim.z = 1;
        config.attrs = attribute;
        config.numAttrs = 1;

        cudaLaunchKernelEx(&config, cluster_kernel, input, output);
    }
}
```

There are two `cudaLaunchAttribute` types which are relevant to thread block clusters clusters: `cudaLaunchAttributeClusterDimension` and `cudaLaunchAttributePreferredClusterDimension`.

The attribute id `cudaLaunchAttributeClusterDimension` specifies the required dimensions with which to execute the cluster. The value for this attribute, `clusterDim`, is a 3-dimensional value. The corresponding dimensions of the grid (x, y, and z) must be divisible by the respective dimensions of the specified cluster dimension. Setting this is similar to using the `__cluster_dims__` attribute on the kernel definition at compile time as shown in *Launching with Clusters in Triple Chevron Notation*, but can be changed at runtime for different launches of the same kernel.

On GPUs with compute capability of 10.0 and higher, another attribute id `cudaLaunchAttributePreferredClusterDimension` allows the application to additionally specify a preferred dimension for the cluster. The preferred dimension must be an integer multiple of the minimum cluster dimensions specified by the `__cluster_dims__` attribute on the kernel or the `cudaLaunchAttributeClusterDimension` attribute to `cudaLaunchKernelEx`. That is, a minimum cluster dimension must be specified in addition to the preferred cluster dimension. The corresponding dimensions of the grid (x, y, and z) must be divisible by the respective dimension of the specified preferred cluster dimension.

All thread blocks will execute in clusters of at least the minimum cluster dimension. Where possible, clusters of the preferred dimension will be used, but not all clusters are guaranteed to execute with the preferred dimensions. All thread blocks will execute in clusters with either the minimum or preferred cluster dimension. Kernels which use a preferred cluster dimension must be written to operate correctly in either the minimum or the preferred cluster dimension.

3.1.2.2 Blocks as Clusters

When a kernel is defined with the `__cluster_dims__` annotation, the number of clusters in the grid is implicit and can be calculated from the size of the grid divided into the specified cluster size.

```
__cluster_dims__((2, 2, 2)) __global__ void foo();

// 8x8x8 clusters each with 2x2x2 thread blocks.
foo<<<dim3(16, 16, 16), dim3(1024, 1, 1)>>>();
```

In the above example, the kernel is launched as a grid of 16x16x16 thread blocks, which means a grid of of 8x8x8 clusters is used.

A kernel can alternatively use the `__block_size__` annotation, which specifies both the required block size and cluster size at the time the kernel is defined. When this annotation is used, the triple chevron launch becomes the grid dimension in terms of clusters rather than thread blocks, as shown below.

```
// Implementation detail of how many threads per block and blocks per cluster
// is handled as an attribute of the kernel.
__block_size__((1024, 1, 1), (2, 2, 2)) __global__ void foo();

// 8x8x8 clusters.
foo<<<dim3(8, 8, 8)>>>();
```

`__block_size__` requires two fields each being a tuple of 3 elements. The first tuple denotes block dimension and second cluster size. The second tuple is assumed to be `(1,1,1)` if it's not passed. When specifying a dynamic shared memory size and/or a stream during kernel launch, the second argument in `<<<>>>` must be the placeholder 1. Any other value results in undefined behavior.

Note that it is illegal for the second tuple of `__block_size__` and `__cluster_dims__` to be specified at the same time. It's also illegal to use `__block_size__` with an empty `__cluster_dims__`. When the second tuple of `__block_size__` is specified, it implies the "Blocks as Clusters" being enabled and the compiler would recognize the first argument inside `<<<>>>` as the number of clusters instead of thread blocks.

3.1.3. More on Streams and Events

CUDA Streams introduced the basics of CUDA streams. By default, operations submitted on a given CUDA stream are serialized: one cannot start executing until the previous one has completed. The only exception is the recently added *Programmatic Dependent Launch and Synchronization* feature. Having multiple CUDA streams is a way to enable concurrent execution; another way is using *CUDA Graphs*. The two approaches can also be combined.

Work submitted on different CUDA streams *may* execute concurrently under specific circumstances, e.g., if there are no event dependencies, if there is no implicit synchronization, if there are sufficient resources, etc.

Independent operations from different CUDA streams cannot run concurrently if any CUDA operation on the NULL stream is submitted in between them, unless the streams are non-blocking CUDA streams. These are streams created with `cudaStreamCreateWithFlags()` runtime API with the `cudaStreamNonBlocking` flag. To improve potential for concurrent GPU work execution, it is recommended that the user creates non-blocking CUDA streams.

It is also recommended that the user selects the least general synchronization option that is sufficient for their problem. For example, if the requirement is for the CPU to wait (block) for all work on a specific CUDA stream to complete, using `cudaStreamSynchronize()` for that stream would be preferable to `cudaDeviceSynchronize()`, as the latter would unnecessarily wait for GPU work on all CUDA streams of the device to complete. And if the requirement is for the CPU to wait without blocking, then using `cudaStreamQuery()` and checking its return value, in a polling loop, may be preferable.

A similar synchronization effect can also be achieved with CUDA events (*CUDA Events*), e.g., by recording an event on that stream and calling `cudaEventSynchronize()` to wait, in a blocking manner, for the work captured in that event to complete. Again, this would be preferable and more focused than using `cudaDeviceSynchronize()`. Calling `cudaEventQuery()` and checking its return value, e.g., in a polling loop, would be a non-blocking alternative.

The choice of the explicit synchronization method is particularly important if this operation happens in the application's critical path. Table 4 provides a high-level summary of various synchronization options with the host.

Table 4: Summary of explicit synchronization options with the host

	Wait for specific stream	**Wait for specific event**	**Wait for everything on the device**
Non-blocking (would need a polling loop)	cudaStream-Query()	cudaEvent-Query()	N/A
Blocking	cudaStreamSyn-chronize()	cudaEventSyn-chronize()	cudaDeviceSynchronize()

For synchronization, i.e., to express dependencies, between CUDA streams, use of non-timing CUDA events is recommended, as described in *CUDA Events*. A user can call `cudaStreamWaitEvent()` to force future submitted operations on a specific stream to wait for the completion of a previously recorded event (e.g., on another stream). Note that for any CUDA API waiting or querying an event, it is the responsibility of the user to ensure the cudaEventRecord API has been already called, as a non-recorded event will always return success.

CUDA events carry, by default, timing information, as they can be used in `cudaEventElapsedTime()` API calls. However, a CUDA event that is solely used to express dependencies across streams does not

need timing information. For such cases, it is recommended to create events with timing information disabled for improved performance. This is possible using `cudaEventCreateWithFlags()` API with the `cudaEventDisableTiming` flag.

3.1.3.1 Stream Priorities

The relative priorities of streams can be specified at creation time using `cudaStreamCreateWithPriority()`. The range of allowable priorities, ordered as [greatest priority, least priority] can be obtained using the `cudaDeviceGetStreamPriorityRange()` function. At runtime, the GPU scheduler utilizes stream priorities to determine task execution order, but these priorities serve as hints rather than guarantees. When selecting work to launch, pending tasks in higher-priority streams take precedence over those in lower-priority streams. Higher-priority tasks do not preempt already running lower-priority tasks. The GPU does not reassess work queues during task execution, and increasing a stream's priority will not interrupt ongoing work. Stream priorities influence task execution without enforcing strict ordering, so users can leverage stream priorities to influence task execution without relying on strict ordering guarantees.

The following code sample obtains the allowable range of priorities for the current device, and creates two non-blocking CUDA streams with the highest and lowest available priorities.

```
// get the range of stream priorities for this device
int leastPriority, greatestPriority;
cudaDeviceGetStreamPriorityRange(&leastPriority, &greatestPriority);

// create streams with highest and lowest available priorities
cudaStream_t st_high, st_low;
cudaStreamCreateWithPriority(&st_high, cudaStreamNonBlocking,
↪greatestPriority));
cudaStreamCreateWithPriority(&st_low, cudaStreamNonBlocking, leastPriority);
```

3.1.3.2 Explicit Synchronization

As previously outlined, there are a number of ways that streams can synchronize with other streams. The following provides common methods at different levels of granularity:

- `cudaDeviceSynchronize()` waits until all preceding commands in all streams of all host threads have completed.
- `cudaStreamSynchronize()`takes a stream as a parameter and waits until all preceding commands in the given stream have completed. It can be used to synchronize the host with a specific stream, allowing other streams to continue executing on the device.
- `cudaStreamWaitEvent()`takes a stream and an event as parameters (see *CUDA Events* for a description of events) and makes all the commands added to the given stream after the call to `cudaStreamWaitEvent()`delay their execution until the given event has completed.
- `cudaStreamQuery()`provides applications with a way to know if all preceding commands in a stream have completed.

3.1.3.3 Implicit Synchronization

Two commands from different streams cannot run concurrently if any one of the following operations is issued in-between them by the host thread:

- a page-locked host memory allocation
- a device memory allocation

- a device memory set
- a memory copy between two addresses to the same device memory
- any CUDA command to the NULL stream
- a switch between the L1/shared memory configurations

Operations that require a dependency check include any other commands within the same stream as the launch being checked and any call to `cudaStreamQuery()` on that stream. Therefore, applications should follow these guidelines to improve their potential for concurrent kernel execution:

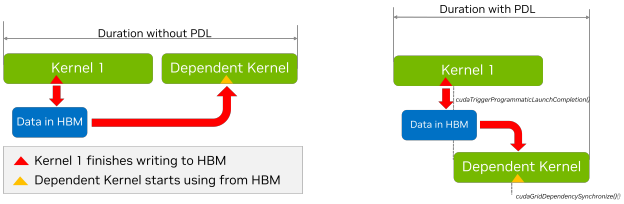
- All independent operations should be issued before dependent operations,
- Synchronization of any kind should be delayed as long as possible.

3.1.4. Programmatic Dependent Kernel Launch

As we have discussed earlier, the semantics of CUDA Streams are such that kernels execute in order. This is so that if we have two successive kernels, where the second kernel depends on results from the first one, the programmer can be safe in the knowledge that by the time the second kernel starts executing the dependent data will be available. However, it may be the case that the first kernel can have the data on which a subsequent kernel depends already written to global memory and it still has more work to do. Likewise the dependent second kernel may have some independent work before it needs the data from the first kernel. In such a situation it is possible to partially overlap the execution of the two kernels (assuming that hardware resources are available). The overlapping can also overlap the launch overheads of the second kernel too. Other than the availability of hardware resources,the degree of overlap which can be achieved is dependent on the specific structure of the kernels, such as

- when in its execution does the first kernel finish the work on which the second kernel depends?
- when in its execution does the second kernel start working on the data from the first kernel?

since this is very much dependent on the specific kernels in question it is difficult to automate completely and hence CUDA provides a mechanism to allow the application developer to specify the synchronization point between the two kernels. This is done via a technique known as Programmatic Dependent Kernel Launch. The situation is depicted in the figure below.



PDL has three main components.

i) The first kernel (the so called *primary kernel*) needs to call a special function to indicate that it is done with the everything that the subsequent dependent kernels (also called *secondary kernel*) will need. This is done by calling the function `cudaTriggerProgrammaticLaunchCompletion()`.

ii) In turn, the dependent secondary kernel needs to indicate that it has reached the portion of the its work which is independent of the primary kernel and that it is now waiting on the primary kernel to finish the work on which it depends. This is done with the function `cudaGridDependencySynchronize()`.

iii) THe second kernel needs to be launched with a special attribute *cudaLaunchAttributeProgrammaticStreamSerialization* with its *programmaticStreamSerializationAllowed* field set to '1'.

The following code snippet shows an example of how this can be done.

Listing 3: Example of Programmatic Dependent Kernel Launch with two Kernels

```
__global__ void primary_kernel() {
    // Initial work that should finish before starting secondary kernel

    // Trigger the secondary kernel
    cudaTriggerProgrammaticLaunchCompletion();

    // Work that can coincide with the secondary kernel
}

__global__ void secondary_kernel()
{
    // Initialization, Independent work, etc.

    // Will block until all primary kernels the secondary kernel is dependent
↪on have
    // completed and flushed results to global memory
    cudaGridDependencySynchronize();

    // Dependent work
}

// Launch the secondary kernel with the special attribute

// Set Up the attribute
cudaLaunchAttribute attribute[1];
attribute[0].id = cudaLaunchAttributeProgrammaticStreamSerialization;
attribute[0].val.programmaticStreamSerializationAllowed = 1;

// Set the attribute in a kernel launch configuration
 cudaLaunchConfig_t config = {0};

// Base launch configuration
config.gridDim = grid_dim;
config.blockDim = block_dim;
config.dynamicSmemBytes= 0;
config.stream = stream;

// Add special attribute for PDL
config.attrs = attribute;
config.numAttrs = 1;

// Launch primary kernel
primary_kernel<<<grid_dim, block_dim, 0, stream>>>();

// Launch secondary (dependent) kernel using the configuration with
// the attribute
```

(continues on next page)

(continued from previous page)

```
cudaLaunchKernelEx(&config, secondary_kernel);
```

3.1.5. Batched Memory Transfers

A common pattern in CUDA development is to use a technique of batching. By batching we loosely mean that we have several (typically small)tasks grouped together into a single (typically bigger) operation. The components of the batch do not necessarily all have to be identical although they often are. An example of this idea is the batch matrix multiplication operation provided by cuBLAS.

Generally as with CUDA Graphs, and PDL, the purpose of batching is to reduce overheads associated with dispatching the individual batch tasks separately. In terms of memory transfers launching a memory transfer can incur some CPU and driver overheads. Further, the regular `cudaMemcpyAsync()` function in its current form does not necessarily provide enough information for the driver to optimize the transfer, for example, in terms of hints about the source and destination. On Tegra platforms one has the choice of using SMs or Copy Engines (CEs)o perform transfers. The choice of which is currently specified by a heuristic in the driver. This can be important because using the SMs may result in a faster transfer, however it ties down some of the available compute power. On the other hand, using the CEs may result in a slower transfer but overall higher application performance, since it leaves the SMs free to perform other work.

These considerations motivated the design of the `cudaMemcpyBatchAsync()` function (and its relative `cudaMemcpyBatch3DAsync()`). These functions allow batched memory transfers to be optimized. Apart from the lists of source and destination pointers, the API uses memory copy attributes to specify expectations of orderings, with hints for source and destination locations, as well as for whether one prefers to overlap the transfer with compute (something that is currently only supported on Tegra platforms with CEs).

Let us first consider the simplest case of a simple batch transfer of data from pinned host memory to pinned device memory

Listing 4: Example of Homogeneous Batched Memory Transfer from Pinned Host Memory to Pinned Device Memory

```
std::vector<void *> srcs(batch_size);
std::vector<void *> dsts(batch_size);
std::vector<size_t> sizes(batch_size);

// Allocate the source and destination buffers
// initialize with the stream number
for (size_t i = 0; i < batch_size; i++) {
    cudaMallocHost(&srcs[i], sizes[i]);
    cudaMalloc(&dsts[i], sizes[i]);
    cudaMemsetAsync(srcs[i], sizes[i], stream);
}

// Setup attributes for this batch of copies
cudaMemcpyAttributes attrs = {};
attrs.srcAccessOrder = cudaMemcpySrcAccessOrderStream;

// All copies in the batch have same copy attributes.
size_t attrsIdxs = 0;  // Index of the attributes
```

(continues on next page)

(continued from previous page)

```
// Launch the batched memory transfer
cudaMemcpyBatchAsync(&dsts[0], &srcs[0], &sizes[0], batch_size,
    &attrs, &attrsIdxs, 1 /*numAttrs*/, nullptr /*failIdx*/, stream);
```

The first few parameters to the `cudaMemcpyBatchAsync()` function seem immediately sensible. The are comprised of arrays containing the source and destination pointers, as well as the transfer sizes. Each array has to have ``batch_size`` elements. The new information comes from the attributes. The function needs a pointer to an array of attributes, and a corresponding array of attribute indices. In principle it is also possible to pass an array of `size_t` and in this array the indices of an failed transfers can be recorded, however it is safe to pass a `nullptr` here, in this case the indices of failures will simply not be recorded.

Turning to the attributes, in this instance the transfers are homogeneous. So we use only one attribute, which will apply to all the transfers. This is controlled by the *attrIndex* parameter. In principle this can be an array. Element *i* of the array contains the index of the first transfer to which the *i*-th element of the attribute array applies. In this case, *attrIndex* is treated as a single element array, with the value '0' meaning that `attribute[0]` will apply to all transfers with index 0 and up, in other words all the transfers.

Finally, we note that we have set the `srcAccessOrder` attribute to `cudaMemcpySrcAccessOrderStream`. This means that the source data will be accessed in regular stream order. In other words, the memcpy will block until previous kernels dealing with the data from any of these source and destination pointers are completed.

In the next example we will consider a more complex case of a heterogeneous batch transfer.

Listing 5: Example of Heterogeneous Batched Memory Transfer using some Ephemeral Host Memory to Pinned Device Memory

```
std::vector<void *> srcs(batch_size);
std::vector<void *> dsts(batch_size);
std::vector<size_t> sizes(batch_size);

// Allocate the src and dst buffers
for (size_t i = 0; i < batch_size - 10; i++) {
    cudaMallocHost(&srcs[i], sizes[i]);
    cudaMalloc(&dsts[i], sizes[i]);
}

int buffer[10];

for (size_t i = batch_size - 10; i < batch_size; i++) {
    srcs[i] = &buffer[10 - (batch_size - i];
    cudaMalloc(&dsts[i], sizes[i]);
}

// Setup attributes for this batch of copies
cudaMemcpyAttributes attrs[2] = {};
attrs[0].srcAccessOrder = cudaMemcpySrcAccessOrderStream;
attrs[1].srcAccessOrder = cudaMemcpySrcAccessOrderDuringApiCall;

size_t attrsIdxs[2];
attrsIdxs[0] = 0;
```

(continues on next page)

(continued from previous page)

```
attrsIdxs[1] = batch_size - 10;

// Launch the batched memory transfer
cudaMemcpyBatchAsync(&dsts[0], &srcs[0], &sizes[0], batch_size,
    &attrs, &attrsIdxs, 2 /*numAttrs*/, nullptr /*failIdx*/, stream);
```

Here we have two kinds of transfers: `batch_size-10` transfer from pinned host memory to pinned device memory, and 10 transfers from a host array to pinned device memory. Further, the *buffer* array is not only on the host but is only in existence in the current scope – its address is what is known as an *ephemeral pointer*. This pointer may not be valid after the API call completes (it is asynchronous). To perform the copies with such ephemeral pointers, the *srcAccessOrder* in the attribute must be set to *cudaMemcpySrcAccessOrderDuringApiCall*.

We now have two attributes, the first one applies to all transfers with indices starting at 0, and less than `batch_size-10`. The second one applies to all transfers with indices starting at `batch_size-10` and less than `batch_size`.

If instead of allocating the *buffer* array from the stack, we had allocated it from the heap using *malloc* the data would not be ephemeral any more. It would be valid until the pointer was explicitly freed. In such a case the best option for how to stage the copies would depend on whether the system had hardware managed memory or coherent GPU access to host memory via address translation in which case it would be best to use stream ordering, or whether it did not in which case staging the transfers immediately would make most sense. In this situation, one should use the value `cudaMemcpyAccessOrderAny` for the `srcAccessOrder` of the attribute.

The `cudaMemcpyBatchAsync` function also allows the programmer to provide hints about the source and destination locations. This is done by setting the `srcLocation` and `dstLocation` fields of the `cudaMemcpyAttributes` structure. The `srcLocation``and ``dstLocation` fields are both of type `cudaMemLocation` which is a structure that contains the type of the location and the ID of the location. This is the same `cudaMemLocation` structure that can be used to give prefetching hints to the runtime when using `cudaMemPrefetchAsync()`. We illustrate how to set up the hints for a transfer from the device, to a specific NUMA node of the host in the code example below:

Listing 6: Example of Setting Source and Destination Location Hints

```
// Allocate the source and destination buffers
std::vector<void *> srcs(batch_size);
std::vector<void *> dsts(batch_size);
std::vector<size_t> sizes(batch_size);

// cudaMemLocation structures we will use tp provide location hints
// Device device_id
cudaMemLocation srcLoc = {cudaMemLocationTypeDevice, dev_id};

// Host with numa Node numa_id
cudaMemLocation dstLoc = {cudaMemLocationTypeHostNuma, numa_id};

// Allocate the src and dst buffers
for (size_t i = 0; i < batch_size; i++) {
    cudaMallocManaged(&srcs[i], sizes[i]);
    cudaMallocManaged(&dsts[i], sizes[i]);

    cudaMemPrefetchAsync(srcs[i], sizes[i], srcLoc, 0, stream);
```

(continues on next page)

(continued from previous page)

```
    cudaMemPrefetchAsync(dsts[i], sizes[i], dstLoc, 0, stream);
    cudaMemsetAsync(srcs[i], sizes[i], stream);
}

// Setup attributes for this batch of copies
cudaMemcpyAttributes attrs = {};

// These are managed memory pointers so Stream Order is appropriate
attrs.srcAccessOrder = cudaMemcpySrcAccessOrderStream;

// Now we can specify the location hints here.
attrs.srcLocHint = srcLoc;
attrs.dstlocHint = dstLoc;

// All copies in the batch have same copy attributes.
size_t attrsIdxs = 0;

// Launch the batched memory transfer
cudaMemcpyBatchAsync(&dsts[0], &srcs[0], &sizes[0], batch_size,
    &attrs, &attrsIdxs, 1 /*numAttrs*/, nullptr /*failIdx*/, stream);
```

THe last thing to cover is the flag for hinting whether we want to use SM's or CEs for the transfers. The field for this is the `cudaMemcpyAttributesflags::flags` and the possible values are:

- `cudaMemcpyFlagDefault` – default behavior
- `cudaMemcpyFlagPreferOverlapWithCompute` – this hints that the system should prefer to use CEs for the transfers overlapping the transfer with computations. This flag is ignored on non-Tegra platforms

In summary, the main points regarding "cudaMemcpyBatchAsync" are as follows:

- The `cudaMemcpyBatchAsync` function (and its 3D variant) allows the programmer to specify a batch of memory transfers, allowing the amortization of transfer setup overheads.
- Other than the source and destination pointers and the transfer sizes, the function can take one or more memory copy attributes providing information about the kind of memory being transferred and the corresponding stream ordering behavior of the source pointers, hints about the source and destination locations, and hints as to whether to prefer to overlap the transfer with compute (if possible) or whether to use SMs for the transfer.
- Given the above information the runtime can attempt to optimize the transfer to the maximum degree possible..

3.1.6. Environment Variables

CUDA provides various environment variables (see Section 5.2), which can affect execution and performance. If they are not explicitly set, CUDA uses reasonable default values for them, but special handling may be required on a per-case basis, e.g., for debugging purposes or to get improved performance.

For example, increasing the value of the `CUDA_DEVICE_MAX_CONNECTIONS` environment variable may be necessary to reduce the possibility that independent work from different CUDA streams gets serialized due to false dependencies. Such false dependencies may be introduced when the same underlying resource(s) are used. It is recommended to start by using the default value and only explore the

impact of this environment variable in case of performance issues (e.g., unexpected serialization of independent work across CUDA streams that cannot be attributed to other factors like lack of available SM resources). Worth noting that this environment variable has a different (lower) default value in case of MPS.

Similarly, setting the `CUDA_MODULE_LOADING` environment variable to `EAGER` may be preferable for latency-sensitive applications, in order to move all overhead due to module loading to the application initialization phase and outside its critical phase. The current default mode is lazy module loading. In this default mode, a similar effect to eager module loading could be achieved by adding "warm-up" calls of the various kernels during the application's initialization phase, to force module loading to happen sooner.

Please refer to *CUDA Environment Variables* for more details about the various CUDA environment variables. It is recommended that you set the environment variables to new values *before* you launch the application; attempting to set them within your application may have no effect.

3.2. Advanced Kernel Programming

This chapter will first take a deeper dive into the hardware model of NVIDIA GPUs, and then introduce some of the more advanced features available in CUDA kernel code aimed at improving kernel performance. This chapter will introduce some concepts related to thread scopes, asynchronous execution, and the associated synchronization primitives. These conceptual discussions provide a necessary foundation for some of the advanced performance features available within kernel code.

Detailed descriptions for some of these features are contained in chapters dedicated to the features in the next part of this programming guide.

- *Advanced synchronization primitives* introduced in this chapter, are covered completely in Section 4.9 and Section 4.10.
- *Asynchronous data copies*, including the tensor memory accelerator (TMA), are introduced in this chapter and covered completely in Section 4.11.

3.2.1. Using PTX

Parallel Thread Execution (PTX), the virtual machine instruction set architecture (ISA) that CUDA uses to abstract hardware ISAs, was introduced in Section 1.3.3. Writing code in PTX directly is a highly advanced optimization technique that is not necessary for most developers and should be considered a tool of last resort. Nevertheless, there are situations where the fine-grained control enabled by writing PTX directly enables performance improvements in specific applications. These situations are typically in very performance-sensitive portions of an application where every fraction of a percent of performance improvement has significant benefits. All of the available PTX instructions are in the PTX ISA document.

`cuda::ptx` namespace

One way to use PTX directly in your code is to use the `cuda::ptx` namespace from libcu++. This namespace provides C++ functions that map directly to PTX instructions, simplifying their use within a C++ application. For more information, please refer to the cuda::ptx namespace documentation.

Inline PTX

Another way to include PTX in your code is to use inline PTX. This method is described in detail in the corresponding documentation. This is very similar to writing assembly code on a CPU.

3.2.2. Hardware Implementation

A streaming multiprocessor or SM (see *GPU Hardware Model*) is designed to execute hundreds of threads concurrently. To manage such a large number of threads, it employs a unique parallel computing model called *Single-Instruction, Multiple-Thread*, or *SIMT*, that is described in *SIMT Execution Model*. The instructions are pipelined, leveraging instruction-level parallelism within a single thread, as well as extensive thread-level parallelism through simultaneous hardware multithreading as detailed in *Hardware Multithreading*. Unlike CPU cores, SMs issue instructions in order and do not perform branch prediction or speculative execution.

Sections *SIMT Execution Model* and *Hardware Multithreading* describe the architectural features of the SM that are common to all devices. Section *Compute Capabilities* provides the specifics for devices of different compute capabilities.

The NVIDIA GPU architecture uses a little-endian representation.

3.2.2.1 SIMT Execution Model

Each SM creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*. Individual threads composing a warp start together at the same program address, but they have their own instruction address counter and register state and are therefore free to branch and execute independently. The term *warp* originates from weaving, the first parallel thread technology. A *half-warp* is either the first or second half of a warp. A *quarter-warp* is either the first, second, third, or fourth quarter of a warp.

A warp executes one common instruction at a time, so full efficiency is realized when all 32 threads of a warp agree on their execution path. If threads of a warp diverge via a data-dependent conditional branch, the warp executes each branch path taken, disabling threads that are not on that path. Branch divergence occurs only within a warp; different warps execute independently regardless of whether they are executing common or disjoint code paths.

The SIMT architecture is akin to SIMD (Single Instruction, Multiple Data) vector organizations in that a single instruction controls multiple processing elements. A key difference is that SIMD vector organizations expose the SIMD width to the software, whereas SIMT instructions specify the execution and branching behavior of a single thread. In contrast with SIMD vector machines, SIMT enables programmers to write thread-level parallel code for independent, scalar threads, as well as data-parallel code for coordinated threads. For the purposes of correctness, the programmer can essentially ignore the SIMT behavior; however, substantial performance improvements can be realized by taking care that the code seldom requires threads in a warp to diverge. In practice, this is analogous to the role of cache lines: the cache line size can be safely ignored when designing for correctness but must be considered in the code structure when designing for peak performance. Vector architectures, on the other hand, require the software to coalesce loads into vectors and manage divergence manually.

3.2.2.1.1 Independent Thread Scheduling

On GPUs with compute capability lower than 7.0, warps used a single program counter shared amongst all 32 threads in the warp together with an active mask specifying the active threads of the warp. As a result, threads from the same warp in divergent regions or different states of execution cannot signal each other or exchange data, and algorithms requiring fine-grained sharing of data guarded by locks or mutexes can lead to deadlock, depending on which warp the contending threads come from.

In GPUs of compute capability 7.0 and later, *independent thread scheduling* allows full concurrency between threads, regardless of warp. With independent thread scheduling, the GPU maintains execution state per thread, including a program counter and call stack, and can yield execution at a per-thread granularity, either to make better use of execution resources or to allow one thread to wait for data to be produced by another. A schedule optimizer determines how to group active threads from the

same warp together into SIMT units. This retains the high throughput of SIMT execution as in prior NVIDIA GPUs, but with much more flexibility: threads can now diverge and reconverge at sub-warp granularity.

Independent thread scheduling can break code that relies on implicit warp-synchronous behavior from previous GPU architectures. *Warp-synchronous* code assumes that threads in the same warp execute in lockstep at every instruction, but the ability for threads to diverge and reconverge at sub-warp granularity makes such assumptions invalid. This can lead to a different set of threads participating in the executed code than intended. Any warp-synchronous code developed for GPUs prior to CC 7.0 (such as synchronization-free intra-warp reductions) should be revisited to ensure compatibility. Developers should explicitly synchronize such code using `__syncwarp()` to ensure correct behavior across all GPU generations.

Note

The threads of a warp that are participating in the current instruction are called the *active* threads, whereas threads not on the current instruction are *inactive* (disabled). Threads can be inactive for a variety of reasons including having exited earlier than other threads of their warp, having taken a different branch path than the branch path currently executed by the warp, or being the last threads of a block whose number of threads is not a multiple of the warp size.

If a non-atomic instruction executed by a warp writes to the same location in global or shared memory from more than one of the threads of the warp, the number of serialized writes that occur to that location may vary depending on the compute capability of the device. However, for all compute capabilities, which thread performs the final write is undefined.

If an *atomic* instruction executed by a warp reads, modifies, and writes to the same location in global memory for more than one of the threads of the warp, each read/modify/write to that location occurs and they are all serialized, but the order in which they occur is undefined.

3.2.2.2 Hardware Multithreading

When an SM is given one or more thread blocks to execute, it partitions them into warps and each warp gets scheduled for execution by a *warp scheduler*. The way a block is partitioned into warps is always the same; each warp contains threads of consecutive, increasing thread IDs with the first warp containing thread 0. *Thread Hierarchy* describes how thread IDs relate to thread indices in the block.

The total number of warps in a block is defined as follows:

ceil $\left(\frac{T}{W_{size}}, 1\right)$

- T is the number of threads per block,
- *Wsize* is the warp size, which is equal to 32,
- ceil(x, y) is equal to x rounded up to the nearest multiple of y.

The execution context (program counters, registers, etc.) for each warp processed by an SM is maintained on-chip throughout the warp's lifetime. Therefore, switching between warps incurs no cost. At each instruction issue cycle, a warp scheduler selects a warp with threads ready to execute its next instruction (the *active threads* of the warp) and issues the instruction to those threads.

Each SM has a set of 32-bit registers that are partitioned among the warps, and a *shared memory* that is partitioned among the thread blocks. The number of blocks and warps that can reside and be processed concurrently on the SM for a given kernel depends on the amount of registers and shared memory used by the kernel, as well as the amount of registers and shared memory available on the SM. There are also a maximum number of resident blocks and warps per SM. These limits, as well the

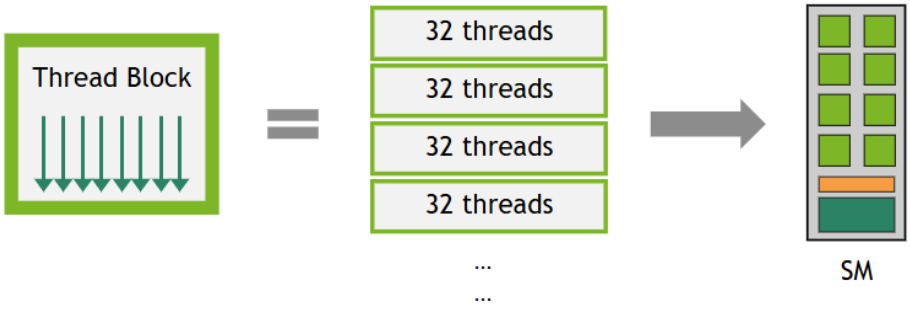


Figure 22: A thread block is partitioned into warps of 32 threads.

amount of registers and shared memory available on the SM, depend on the compute capability of the device and are specified in *Compute Capabilities*. If there are not enough resources available per SM to process at least one block, the kernel will fail to launch. The total number of registers and shared memory allocated for a block can be determined in several ways documented in the *Occupancy* section.

3.2.2.3 Asynchronous Execution Features

Recent NVIDIA GPU generations have included asynchronous execution capabilities to allow more overlap of data movement, computation, and synchronization within the GPU. These capabilities enable certain operations invoked from GPU code to execute asynchronously to other GPU code in the same thread block. This asynchronous execution should not be confused with asynchronous CUDA APIs discussed in Section 2.5, which enable GPU kernel launches or memory operations to operate asynchronously to each other or to the CPU.

Compute capability 8.0 (The NVIDIA Ampere GPU Architecture) introduced hardware-accelerated asynchronous data copies from global to shared memory and asynchronous barriers (see NVIDIA A100 Tensor Core GPU Architecture).

Compute capability 9.0 (The NVIDIA Hopper GPU architecture) extended the asynchronous execution features with the *Tensor Memory Accelerator (TMA)* unit, which can transfer large blocks of data and multidimensional tensors from global memory to shared memory and vice versa, asynchronous transaction barriers, and asynchronous matrix multiply-accumulate operations (see Hopper Architecture in Depth blog post for details.).

CUDA provides APIs which can be called by threads from device code to use these features. The asynchronous programming model defines the behavior of asynchronous operations with respect to CUDA threads.

An asynchronous operation is an operation initiated by a CUDA thread, but executed asynchronously as if by another thread, which we will refer to as an *async thread*. In a well-formed program, one or more CUDA threads synchronize with the asynchronous operation. The CUDA thread that initiated the asynchronous operation is not required to be among the synchronizing threads. The async thread is always associated with the CUDA thread that initiated the operation.

An asynchronous operation uses a synchronization object to signal its completion, which could be a barrier or a pipeline. These synchronization objects are explained in detail in *Advanced Synchronization Primitives*, and their role in performing asynchronous memory operations is demonstrated in *Asynchronous Data Copies*.

3.2.2.3.1 Async Thread and Async Proxy

Asynchronous operations may access memory differently than regular operations. To distinguish between these different memory access methods, CUDA introduces the concepts of an *async thread*, a *generic proxy*, and an *async proxy*. Normal operations (loads and stores) go through the generic proxy. Some asynchronous instructions, such as *LDGSTS* and *STAS/REDAS*, are modeled using an async thread operating in the generic proxy. Other asynchronous instructions, such as bulk-asynchronous copies with TMA and some tensor core operations (tcgen05.*, wgmma.mma_async.*), are modeled using an async thread operating in the async proxy.

Async thread operating in generic proxy. When an asynchronous operation is initiated, it is associated with an async thread, which is different from the CUDA thread that initiated the operation. *Preceding* generic proxy (normal) loads and stores to the same address are guaranteed to be ordered before the asynchronous operation. However, *subsequent* normal loads and stores to the same address are not guaranteed to maintain their ordering, potentially incurring a race condition until the async thread completes.

Async thread operating in async proxy. When an asynchronous operation is initiated, it is associated with an async thread, which is different from the CUDA thread that initiated the operation. *Prior and subsequent* normal loads and stores to the same address are not guaranteed to maintain their ordering. A proxy fence is required to synchronize them across the different proxies to ensure proper memory ordering. Section *Using the Tensor Memory Accelerator (TMA)* demonstrates use of proxy fences to ensure correctness when performing asynchronous copies with TMA.

For more details on these concepts, see the PTX ISA documentation.

3.2.3. Thread Scopes

CUDA threads form a *Thread Hierarchy*, and using this hierarchy is essential for writing both correct and performant CUDA kernels. Within this hierarchy, the visibility and synchronization scope of memory operations can vary. To account for this non-uniformity, the CUDA programming model introduces the concept of *thread scopes*. A thread scope defines which threads can observe a thread's loads and stores and specifies which threads can synchronize with each other using synchronization primitives such as atomic operations and barriers. Each scope has an associated point of coherency in the memory hierarchy.

Thread scopes are exposed in CUDA PTX and are also available as extensions in the libcu++ library. The following table defines the thread scopes available:

CUDA C++ Thread Scope	CUDA PTX Thread Scope	Description	Point of Coherency in Memory Hierarchy
cuda::thread_s		Memory operations are visible only to the local thread.	–
cuda::thread_s	.cta	Memory operations are visible to other threads in the same thread block.	L1
	.cluster	Memory operations are visible to other threads in the same thread block cluster.	L2
cuda::thread_s	.gpu	Memory operations are visible to other threads in the same GPU device.	L2
cuda::thread_s	.sys	Memory operations are visible to other threads in the same system (CPU, other GPUs).	L2 + connected caches

Sections *Advanced Synchronization Primitives* and *Asynchronous Data Copies* demonstrate use of thread scopes.

3.2.4. Advanced Synchronization Primitives

This section introduces three families of synchronization primitives:

- *Scoped Atomics*, which pair C++ memory ordering with CUDA thread scopes to safely communicate across threads at block, cluster, device, or system scope (see *Thread Scopes*).
- *Asynchronous Barriers*, which split synchronization into arrival and wait phases, and can be used to track the progress of asynchronous operations.
- *Pipelines*, which stage work and coordinate multi-buffer producer–consumer patterns, commonly used to overlap compute with *asynchronous data copies*.

3.2.4.1 Scoped Atomics

Section 5.4.5 gives an overview of atomic functions available in CUDA. In this section, we will focus on *scoped* atomics that support C++ standard atomic memory semantics, available through the libcu++ library or through compiler built-in functions. Scoped atomics provide the tools for efficient synchronization at the appropriate level of the CUDA thread hierarchy, enabling both correctness and performance in complex parallel algorithms.

3.2.4.1.1 Thread Scope and Memory Ordering

Scoped atomics combine two key concepts:

- **Thread Scope**: defines which threads can observe the effect of the atomic operation (see *Thread Scopes*).
- **Memory Ordering**: defines the ordering constraints relative to other memory operations (see C++ standard atomic memory semantics).

CUDA C++ `cuda::atomic`

```
#include <cuda/atomic>

__global__ void block_scoped_counter() {
    // Shared atomic counter visible only within this block
    __shared__ cuda::atomic<int, cuda::thread_scope_block> counter;

    // Initialize counter (only one thread should do this)
    if (threadIdx.x == 0) {
        counter.store(0, cuda::memory_order_relaxed);
    }
    __syncthreads();

    // All threads in block atomically increment
    int old_value = counter.fetch_add(1, cuda::memory_order_relaxed);

    // Use old_value...
}
```

Built-in Atomic Functions

```
__global__ void block_scoped_counter() {
    // Shared counter visible only within this block
    __shared__ int counter;

    // Initialize counter (only one thread should do this)
    if (threadIdx.x == 0) {
        __nv_atomic_store_n(&counter, 0,
                            __NV_ATOMIC_RELAXED,
                            __NV_THREAD_SCOPE_BLOCK);
    }
    __syncthreads();

    // All threads in block atomically increment
    int old_value = __nv_atomic_fetch_add(&counter, 1,
                                          __NV_ATOMIC_RELAXED,
                                          __NV_THREAD_SCOPE_BLOCK);

    // Use old_value...
}
```

This example implements a *block-scoped atomic counter* that demonstrates the fundamental concepts of scoped atomics:

- **Shared Variable**: a single counter is shared among all threads in the block using `__shared__` memory.
- **Atomic Type Declaration**: `cuda::atomic<int, cuda::thread_scope_block>` creates an atomic integer with block-level visibility.
- **Single Initialization**: only thread 0 initializes the counter to prevent race conditions during setup.
- **Block Synchronization**: `__syncthreads()` ensures all threads see the initialized counter before proceeding.

- **Atomic Increment**: each thread atomically increments the counter and receives the previous value.

`cuda::memory_order_relaxed` is chosen here because we only need atomicity (indivisible read-modify-write) without ordering constraints between different memory locations. Since this is a straightforward counting operation, the order of increments doesn't matter for correctness.

For producer-consumer patterns, acquire-release semantics ensure proper ordering:

CUDA C++ `cuda::atomic`

```
__global__ void producer_consumer() {
    __shared__ int data;
    __shared__ cuda::atomic<bool, cuda::thread_scope_block> ready;

    if (threadIdx.x == 0) {
        // Producer: write data then signal ready
        data = 42;
        ready.store(true, cuda::memory_order_release);  // Release ensures
↪data write is visible
    } else {
        // Consumer: wait for ready signal then read data
        while (!ready.load(cuda::memory_order_acquire)) {  // Acquire ensures
↪data read sees the write
            // spin wait
        }
        int value = data;
        // Process value...
    }
}
```

Built-in Atomic Functions

```
__global__ void producer_consumer() {
    __shared__ int data;
    __shared__ bool ready; // Only ready flag needs atomic operations

    if (threadIdx.x == 0) {
        // Producer: write data then signal ready
        data = 42;
        __nv_atomic_store_n(&ready, true,
                            __NV_ATOMIC_RELEASE,
                            __NV_THREAD_SCOPE_BLOCK);  // Release ensures
↪data write is visible
    } else {
        // Consumer: wait for ready signal then read data
        while (!__nv_atomic_load_n(&ready,
                                   __NV_ATOMIC_ACQUIRE,
                                   __NV_THREAD_SCOPE_BLOCK)) {  // Acquire
↪ensures data read sees the write
            // spin wait
        }
        int value = data;
```

(continues on next page)

(continued from previous page)

```
        // Process value...
    }
}
```

3.2.4.1.2 Performance Considerations

- *Use the narrowest scope possible*: block-scoped atomics are much faster than system-scoped atomics.
- *Prefer weaker orderings*: use stronger orderings only when necessary for correctness.
- *Consider memory location*: shared memory atomics are faster than global memory atomics.

3.2.4.2 Asynchronous Barriers

An asynchronous barrier differs from a typical single-stage barrier (`__syncthreads()`) in that the notification by a thread that it has reached the barrier (the "arrival") is separated from the operation of waiting for other threads to arrive at the barrier (the "wait"). This separation increases execution efficiency by allowing a thread to perform additional operations unrelated to the barrier, making more efficient use of the wait time. Asynchronous barriers can be used to implement producer-consumer patterns with CUDA threads or enable asynchronous data copies within the memory hierarchy by having the copy operation signal ("arrive on") a barrier upon completion.

Asynchronous barriers are available on devices of compute capability 7.0 or higher. Devices of compute capability 8.0 or higher provide hardware acceleration for asynchronous barriers in shared-memory and a significant advancement in synchronization granularity, by allowing hardware-accelerated synchronization of any subset of CUDA threads within the block. Previous architectures only accelerate synchronization at a whole-warp (`__syncwarp()`) or whole-block (`__syncthreads()`) level.

The CUDA programming model provides asynchronous barriers via `cuda::std::barrier`, an ISO C++-conforming barrier available in the libcu++ library. In addition to implementing std::barrier, the library offers CUDA-specific extensions to select a barrier's thread scope to improve performance and exposes a lower-level cuda::ptx API. A `cuda::barrier` can interoperate with `cuda::ptx` by using the `friend` function `cuda::device::barrier_native_handle()` to retrieve the barrier's native handle and pass it to `cuda::ptx` functions. CUDA also provides a *primitives API* for asynchronous barriers in shared memory at thread-block scope.

The following table gives an overview of asynchronous barriers available for synchronizing at different thread scopes.

Thread Scope	Memory Location	Arrive on Barrier	Wait on Barrier	Hardware-accelerated	CUDA APIs
block	local shared memory	allowed	allowed	yes (8.0+)	`cuda::barrier`, `cuda::ptx`, primitives
cluster	local shared memory	allowed	allowed	yes (9.0+)	`cuda::barrier`, `cuda::ptx`
cluster	remote shared memory	allowed	not allowed	yes (9.0+)	`cuda::barrier`, `cuda::ptx`
device	global memory	allowed	allowed	no	`cuda::barrier`
system	global/unified memory	allowed	allowed	no	`cuda::barrier`

Temporal Splitting of Synchronization

Without the asynchronous arrive-wait barriers, synchronization within a thread block is achieved using `__syncthreads()` or `block.sync()` when using *Cooperative Groups*.

```
#include <cooperative_groups.h>

__global__ void simple_sync(int iteration_count) {
    auto block = cooperative_groups::this_thread_block();

    for (int i = 0; i < iteration_count; ++i) {
        /* code before arrive */

         // Wait for all threads to arrive here.
        block.sync();

        /* code after wait */
    }
}
```

Threads are blocked at the synchronization point (`block.sync()`) until all threads have reached the synchronization point. In addition, memory updates that happened before the synchronization point are guaranteed to be visible to all threads in the block after the synchronization point.

This pattern has three stages:

- Code **before** the sync performs memory updates that will be read **after** the sync.
- Synchronization point.
- Code **after** the sync, with visibility of memory updates that happened **before** the sync.

Using asynchronous barriers instead, the temporally-split synchronization pattern is as follows.

CUDA C++ cuda::barrier

```
#include <cuda/barrier>
#include <cooperative_groups.h>

__device__ void compute(float *data, int iteration);

__global__ void split_arrive_wait(int iteration_count, float *data)
{
  using barrier_t = cuda::barrier<cuda::thread_scope_block>;
  __shared__ barrier_t bar;
  auto block = cooperative_groups::this_thread_block();

  if (block.thread_rank() == 0)
  {
    // Initialize barrier with expected arrival count.
    init(&bar, block.size());
  }
  block.sync();

  for (int i = 0; i < iteration_count; ++i)
  {
    /* code before arrive */

    // This thread arrives. Arrival does not block a thread.
    barrier_t::arrival_token token = bar.arrive();

    compute(data, i);

    // Wait for all threads participating in the barrier to complete bar.
↪arrive().
    bar.wait(std::move(token));

    /* code after wait */
  }
}
```

CUDA C++ cuda::ptx

```
#include <cuda/ptx>
#include <cooperative_groups.h>

__device__ void compute(float *data, int iteration);

__global__ void split_arrive_wait(int iteration_count, float *data)
{
  __shared__ uint64_t bar;
  auto block = cooperative_groups::this_thread_block();

  if (block.thread_rank() == 0)
  {
    // Initialize barrier with expected arrival count.
    cuda::ptx::mbarrier_init(&bar, block.size());
  }
  block.sync();

  for (int i = 0; i < iteration_count; ++i)
  {
    /* code before arrive */

    // This thread arrives. Arrival does not block a thread.
    uint64_t token = cuda::ptx::mbarrier_arrive(&bar);

    compute(data, i);

    // Wait for all threads participating in the barrier to complete mbarrier_
↪arrive().
    while(!cuda::ptx::mbarrier_try_wait(&bar, token)) {}

    /* code after wait */
  }
}
```

CUDA C primitives

```
#include <cuda_awbarrier_primitives.h>
#include <cooperative_groups.h>

__device__ void compute(float *data, int iteration);

__global__ void split_arrive_wait(int iteration_count, float *data)
{
  __shared__ __mbarrier_t bar;
  auto block = cooperative_groups::this_thread_block();

  if (block.thread_rank() == 0)
  {
    // Initialize barrier with expected arrival count.
    __mbarrier_init(&bar, block.size());
  }
  block.sync();

  for (int i = 0; i < iteration_count; ++i)
  {
    /* code before arrive */

    // This thread arrives. Arrival does not block a thread.
    __mbarrier_token_t token = __mbarrier_arrive(&bar);

    compute(data, i);

    // Wait for all threads participating in the barrier to complete __
↪mbarrier_arrive().
    while(!__mbarrier_try_wait(&bar, token, 1000)) {}

    /* code after wait */
  }
}
```

In this pattern, the synchronization point is split into an arrive point (`bar.arrive()`) and a wait point (`bar.wait(std::move(token))`). A thread begins participating in a `cuda::barrier` with its first call to `bar.arrive()`. When a thread calls `bar.wait(std::move(token))` it will be blocked until participating threads have completed `bar.arrive()` the expected number of times, which is the expected arrival count argument passed to `init()`. Memory updates that happen before participating threads' call to `bar.arrive()` are guaranteed to be visible to participating threads after their call to `bar.wait(std::move(token))`. Note that the call to `bar.arrive()` does not block a thread, it can proceed with other work that does not depend upon memory updates that happen before other participating threads' call to `bar.arrive()`.

The *arrive and wait* pattern has five stages:

- Code **before** the arrive performs memory updates that will be read **after** the wait.
- Arrive point with implicit memory fence (i.e., equivalent to `cuda::atomic_thread_fence(cuda::memory_orde`
  `cuda::thread_scope_block)`).

- Code **between** arrive and wait.
- Wait point.
- Code **after** the wait, with visibility of updates that were performed **before** the arrive.

For a comprehensive guide on how to use asynchronous barriers, see *Asynchronous Barriers*.

3.2.4.3 Pipelines

The CUDA programming model provides the pipeline synchronization object as a coordination mechanism to sequence asynchronous memory copies into multiple stages, facilitating the implementation of double- or multi-buffering producer-consumer patterns. A pipeline is a double-ended queue with a *head* and a *tail* that processes work in a first-in first-out (FIFO) order. Producer threads commit work to the pipeline's head, while consumer threads pull work from the pipeline's tail.

Pipelines are exposed through the `cuda::pipeline` API in the libcu++ library, as well as through a *primitives API*. The following tables describe the main functionality of the two APIs.

`cuda::pipeline` API	**Description**
`producer_acquire`	Acquires an available stage in the pipeline's internal queue.
`producer_commit`	Commits the asynchronous operations issued after the `producer_acquire` call on the currently acquired stage of the pipeline.
`consumer_wait`	Waits for completion of asynchronous operations in the oldest stage of the pipeline.
`consumer_release`	Releases the oldest stage of the pipeline to the pipeline object for reuse. The released stage can be then acquired by a producer.

Primitives API	**Description**
`__pipeline_memcpy_async`	Request a memory copy from global to shared memory to be submitted for asynchronous evaluation.
`__pipeline_commit`	Commits the asynchronous operations issued before the call on the current stage of the pipeline.
`__pipeline_wait_prior(N`	Waits for completion of asynchronous operations in all but the last N commits to the pipeline.

The `cuda::pipeline` API has a richer interface with less restrictions, while the primitives API only supports tracking asynchronous copies from global memory to shared memory with specific size and alignment requirements. The primitives API provides equivalent functionality to a `cuda::pipeline` object with `cuda::thread_scope_thread`.

For detailed usage patterns and examples, see *Pipelines*.

3.2.5. Asynchronous Data Copies

Efficient data movement within the memory hierarchy is fundamental to achieving high performance in GPU computing. Traditional synchronous memory operations force threads to wait idle during data

transfers. GPUs inherently hide memory latency through parallelism. That is, the SM switches to execute another warp while memory operations complete. Even with this latency hiding through parallelism, it is still possible for memory latency to be a bottleneck on both memory bandwidth utilization and compute resource efficiency. To address these bottlenecks, modern GPU architectures provide hardware-accelerated asynchronous data copy mechanisms that allow memory transfers to proceed independently while threads continue executing other work.

Asynchronous data copies enable overlapping of computation with data movement, by decoupling the initiation of a memory transfer from waiting for its completion. This way, threads can perform useful work during memory latency periods, leading to improved overall throughput and resource utilization.

Note

While concepts and principles underlying this section are similar to those discussed in the earlier chapter on *Asynchronous Execution*, that chapter covered asynchronous execution of kernels and memory transfers such as those invoked by `cudaMemcpyAsync`. That can be considered asynchrony of different components of the application.

The asynchrony described in this section refers to enabling transfer of data between the GPU's DRAM, i.e. global memory, and on-SM memory such as shared memory or tensor memory without blocking the GPU threads. This is an asynchrony within the execution of a single kernel launch.

To understand how asynchronous copies can improve performance, it is helpful to examine a common GPU computing pattern. CUDA applications often employ a *copy and compute* pattern that:

- fetches data from global memory,
- stores data to shared memory, and
- performs computations on shared memory data, and potentially writes results back to global memory.

The *copy* phase of this pattern is typically expressed as `shared[local_idx] = global[global_idx]`. This global to shared memory copy is expanded by the compiler to a read from global memory into a register followed by a write to shared memory from the register.

When this pattern occurs within an iterative algorithm, each thread block needs to synchronize after the `shared[local_idx] = global[global_idx]` assignment, to ensure all writes to shared memory have completed before the compute phase can begin. The thread block also needs to synchronize again after the compute phase, to prevent overwriting shared memory before all threads have completed their computations. This pattern is illustrated in the following code snippet.

```
#include <cooperative_groups.h>

__device__ void compute(int* global_out, int const* shared_in) {
    // Computes using all values of current batch from shared memory.
    // Stores this thread's result back to global memory.
}

__global__ void without_async_copy(int* global_out, int const* global_in,
↪size_t size, size_t batch_sz) {
  auto grid = cooperative_groups::this_grid();
  auto block = cooperative_groups::this_thread_block();
  assert(size == batch_sz * grid.size()); // Exposition: input size fits batch_
↪sz * grid_size
```

(continues on next page)

(continued from previous page)

```
  extern __shared__ int shared[]; // block.size() * sizeof(int) bytes

  size_t local_idx = block.thread_rank();

  for (size_t batch = 0; batch < batch_sz; ++batch) {
    // Compute the index of the current batch for this block in global memory.
    size_t block_batch_idx = block.group_index().x * block.size() + grid.
↪size() * batch;
    size_t global_idx = block_batch_idx + threadIdx.x;
    shared[local_idx] = global_in[global_idx];

    // Wait for all copies to complete.
    block.sync();

    // Compute and write result to global memory.
    compute(global_out + block_batch_idx, shared);

    // Wait for compute using shared memory to finish.
    block.sync();
  }
}
```

With asynchronous data copies, data movement from global memory to shared memory can be done asynchronously to enable more efficient use of the SM while waiting for data to arrive.

```
#include <cooperative_groups.h>
#include <cooperative_groups/memcpy_async.h>

__device__ void compute(int* global_out, int const* shared_in) {
    // Computes using all values of current batch from shared memory.
    // Stores this thread's result back to global memory.
}

__global__ void with_async_copy(int* global_out, int const* global_in, size_t
↪size, size_t batch_sz) {
  auto grid = cooperative_groups::this_grid();
  auto block = cooperative_groups::this_thread_block();
  assert(size == batch_sz * grid.size()); // Exposition: input size fits batch_
↪sz * grid_size

  extern __shared__ int shared[]; // block.size() * sizeof(int) bytes

  size_t local_idx = block.thread_rank();

  for (size_t batch = 0; batch < batch_sz; ++batch) {
    // Compute the index of the current batch for this block in global memory.
    size_t block_batch_idx = block.group_index().x * block.size() + grid.
↪size() * batch;

    // Whole thread-group cooperatively copies whole batch to shared memory.
    cooperative_groups::memcpy_async(block, shared, global_in + block_batch_
↪idx, block.size());
```

(continues on next page)

(continued from previous page)

```
    // Compute on different data while waiting.

    // Wait for all copies to complete.
    cooperative_groups::wait(block);

    // Compute and write result to global memory.
    compute(global_out + block_batch_idx, shared);

    // Wait for compute using shared memory to finish.
    block.sync();
  }
}
```

The *cooperative_groups::memcpy_async* function copies `block.size()` elements from global memory to the `shared` data. This operation happens as-if performed by another thread, which synchronizes with the current thread's call to *cooperative_groups::wait* after the copy has completed. Until the copy operation completes, modifying the global data or reading or writing the shared data introduces a data race.

This example illustrates the fundamental concept behind all asynchronous copy operations: they decouple memory transfer initiation from completion, allowing threads to perform other work while data moves in the background. The CUDA programming model provides several APIs to access these capabilities, including `memcpy_async` functions available in *Cooperative Groups* and the libcu++ library, as well as lower-level `cuda::ptx` and primitives APIs. These APIs share similar semantics: they copy objects from source to destination as-if performed by another thread which, on completion of the copy, can be synchronized using different completion mechanisms.

Modern GPU architectures provide multiple hardware mechanisms for asynchronous data movement.

- LDGSTS (compute capability 8.0+) allows for efficient small-scale asynchronous transfers from global to shared memory.
- The tensor memory accelerator (TMA, compute capability 9.0+) extends these capabilities, providing bulk-asynchronous copy operations optimized for large multi-dimensional data transfers
- STAS instructions (compute capability 9.0+) enable small-scale asynchronous transfers from registers to distributed shared memory within a cluster.

These mechanisms support different data paths, transfer sizes, and alignment requirements, allowing developers to choose the most appropriate approach for their specific data access patterns. The following table gives an overview of the supported data paths for asynchronous copies within the GPU.

Table 5: Asynchronous copies with possible source and destination memory spaces. An empty cell indicates that a source-destination pair is not supported.

Direction		Copy Mechanism	
Source	**Destination**	**Asynchronous Copy**	**Bulk-Asynchronous Copy**
global	global		
shared::cta	global		supported (TMA, 9.0+)
global	shared::cta	supported (LDGSTS, 8.0+)	supported (TMA, 9.0+)
global	shared::cluster		supported (TMA, 9.0+)
shared::cluster	shared::cta		supported (TMA, 9.0+)
shared::cta	shared::cta		
registers	shared::cluster	supported (STAS, 9.0+)	

Sections *Using LDGSTS*, *Using the Tensor Memory Accelerator (TMA)* and *Using STAS* will go into more details about each mechanism.

3.2.6. Configuring L1/Shared Memory Balance

As mentioned in *L1 data cache*, the L1 and shared memory on an SM use the same physical resource, known as the unified data cache. On most architectures, if a kernel uses little or no shared memory, the unified data cache can be configured to provide the maximal amount of L1 cache allowed by the architecture.

The unified data cache reserved for shared memory is configurable on a per-kernel basis. An application can set the `carveout`, or preferred shared memory capacity, with the cudaFuncSetAttribute function called before the kernel is launched.

```
cudaFuncSetAttribute(kernel_name,
↪cudaFuncAttributePreferredSharedMemoryCarveout, carveout);
```

The application can set the `carveout` as an integer percentage of the maximum supported shared memory capacity of that architecture. In addition to an integer percentage, three convenience enums are provided as carveout values.

- `cudaSharedmemCarveoutDefault`
- `cudaSharedmemCarveoutMaxL1`
- `cudaSharedmemCarveoutMaxShared`

The maximum supported shared memory and the supported carveout sizes vary by architecture; see *Shared Memory Capacity per Compute Capability* for details.

Where a chosen integer percentage carveout does not map exactly to a supported shared memory capacity, the next larger capacity is used. For example, for devices of compute capability 12.0, which have a maximum shared memory capacity of 100KB, setting the carveout to 50% will result in 64KB of shared memory, not 50KB, because devices of compute capability 12.0 support shared memory sizes of 0, 8, 16, 32, 64, and 100.

The function passed to `cudaFuncSetAttribute` must be declared with the `__global__` specifier. `cudaFuncSetAttribute` is interpreted by the driver as a hint, and the driver may choose a different carveout size if required to execute the kernel.

Note

Another CUDA API, `cudaFuncSetCacheConfig`, also allows an application to adjust the balance between L1 and shared memory for a kernel. However, this API set a hard requirements on shared/L1 balance for kernel launch. As a result, interleaving kernels with different shared memory configurations would needlessly *serialize launches* behind shared memory reconfigurations. `cudaFuncSetAttribute` is preferred because driver may choose a different configuration if required to execute the function or to avoid thrashing.

Kernels relying on shared memory allocations over 48 KB per block are architecture-specific. As such they must use *dynamic shared memory* rather than statically-sized arrays and require an explicit opt-in using `cudaFuncSetAttribute` as follows.

```
// Device code
__global__ void MyKernel(...)
{
  extern __shared__ float buffer[];
  ...
}

// Host code
int maxbytes = 98304; // 96 KB
cudaFuncSetAttribute(MyKernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
 ↪maxbytes);
MyKernel <<<gridDim, blockDim, maxbytes>>>(...);
```

3.3. The CUDA Driver API

Previous sections of this guide have covered the CUDA runtime. As mentioned in *CUDA Runtime API and CUDA Driver API*, the CUDA runtime is written on top of the lower level CUDA driver API. This section covers some of the differences between the CUDA runtime and the driver APIs, as well has how to intermix them. Most applications can operate at full performance without ever needing to interact with the CUDA driver API. However, new interfaces are sometimes available in the driver API earlier than the runtime API, and some advanced interfaces, such as *Virtual Memory Management*, are only exposed in the driver API.

The driver API is implemented in the `cuda` dynamic library (`cuda.dll` or `cuda.so`) which is copied on the system during the installation of the device driver. All its entry points are prefixed with cu.

It is a handle-based, imperative API: Most objects are referenced by opaque handles that may be specified to functions to manipulate the objects.

The objects available in the driver API are summarized in Table 6.

Table 6: Objects Available in the CUDA Driver API

Object	Handle	Description
Device	CUdevice	CUDA-enabled device
Context	CUcontext	Roughly equivalent to a CPU process
Module	CUmodule	Roughly equivalent to a dynamic library
Function	CUfunction	Kernel
Heap memory	CUdeviceptr	Pointer to device memory
CUDA array	CUarray	Opaque container for one-dimensional or two-dimensional data on the device, readable via texture or surface references
Texture object	CUtexref	Object that describes how to interpret texture memory data
Surface reference	CUsurfref	Object that describes how to read or write CUDA arrays
Stream	CUstream	Object that describes a CUDA stream
Event	CUevent	Object that describes a CUDA event

The driver API must be initialized with `cuInit()` before any function from the driver API is called. A CUDA context must then be created that is attached to a specific device and made current to the calling host thread as detailed in *Context*.

Within a CUDA context, kernels are explicitly loaded as PTX or binary objects by the host code as described in *Module*. Kernels written in C++ must therefore be compiled separately into *PTX* or binary objects. Kernels are launched using API entry points as described in *Kernel Execution*.

Any application that wants to run on future device architectures must load *PTX*, not binary code. This is because binary code is architecture-specific and therefore incompatible with future architectures, whereas *PTX* code is compiled to binary code at load time by the device driver.

Here is the host code of the sample from *Kernels* written using the driver API:

```
int main()
{
    int N = ...;
    size_t size = N * sizeof(float);

    // Allocate input vectors h_A and h_B in host memory
    float* h_A = (float*)malloc(size);
    float* h_B = (float*)malloc(size);

    // Initialize input vectors
    ...
```

(continues on next page)

(continued from previous page)

```
    // Initialize
    cuInit(0);

    // Get number of devices supporting CUDA
    int deviceCount = 0;
    cuDeviceGetCount(&deviceCount);
    if (deviceCount == 0) {
        printf("There is no device supporting CUDA.\n");
        exit (0);
    }

    // Get handle for device 0
    CUdevice cuDevice;
    cuDeviceGet(&cuDevice, 0);

    // Create context
    CUcontext cuContext;
    cuCtxCreate(&cuContext, 0, cuDevice);

    // Create module from binary file
    CUmodule cuModule;
    cuModuleLoad(&cuModule, "VecAdd.ptx");

    // Allocate vectors in device memory
    CUdeviceptr d_A;
    cuMemAlloc(&d_A, size);
    CUdeviceptr d_B;
    cuMemAlloc(&d_B, size);
    CUdeviceptr d_C;
    cuMemAlloc(&d_C, size);

    // Copy vectors from host memory to device memory
    cuMemcpyHtoD(d_A, h_A, size);
    cuMemcpyHtoD(d_B, h_B, size);

    // Get function handle from module
    CUfunction vecAdd;
    cuModuleGetFunction(&vecAdd, cuModule, "VecAdd");

    // Invoke kernel
    int threadsPerBlock = 256;
    int blocksPerGrid =
            (N + threadsPerBlock - 1) / threadsPerBlock;
    void* args[] = { &d_A, &d_B, &d_C, &N };
    cuLaunchKernel(vecAdd,
                   blocksPerGrid, 1, 1, threadsPerBlock, 1, 1,
                   0, 0, args, 0);

    ...
}
```

Full code can be found in the `vectorAddDrv` CUDA sample.

3.3.1. Context

A CUDA context is analogous to a CPU process. All resources and actions performed within the driver API are encapsulated inside a CUDA context, and the system automatically cleans up these resources when the context is destroyed. Besides objects such as modules and texture or surface references, each context has its own distinct address space. As a result, `CUdeviceptr` values from different contexts reference different memory locations.

A host thread may have only one device context current at a time. When a context is created with `cuCtxCreate()`, it is made current to the calling host thread. CUDA functions that operate in a context (most functions that do not involve device enumeration or context management) will return `CUDA_ERROR_INVALID_CONTEXT` if a valid context is not current to the thread.

Each host thread has a stack of current contexts. `cuCtxCreate()` pushes the new context onto the top of the stack. `cuCtxPopCurrent()` may be called to detach the context from the host thread. The context is then "floating" and may be pushed as the current context for any host thread. `cuCtxPopCurrent()` also restores the previous current context, if any.

A usage count is also maintained for each context. `cuCtxCreate()` creates a context with a usage count of 1. `cuCtxAttach()` increments the usage count and `cuCtxDetach()` decrements it. A context is destroyed when the usage count goes to 0 when calling `cuCtxDetach()` or `cuCtxDestroy()`.

The driver API is interoperable with the runtime and it is possible to access the primary context (see *Runtime Initialization*) managed by the runtime from the driver API via `cuDevicePrimaryCtxRetain()`.

Usage count facilitates interoperability between third party authored code operating in the same context. For example, if three libraries are loaded to use the same context, each library would call `cuCtxAttach()` to increment the usage count and `cuCtxDetach()` to decrement the usage count when the library is done using the context. For most libraries, it is expected that the application will have created a context before loading or initializing the library; that way, the application can create the context using its own heuristics, and the library simply operates on the context handed to it. Libraries that wish to create their own contexts - unbeknownst to their API clients who may or may not have created contexts of their own - would use `cuCtxPushCurrent()` and `cuCtxPopCurrent()` as illustrated in the following figure.

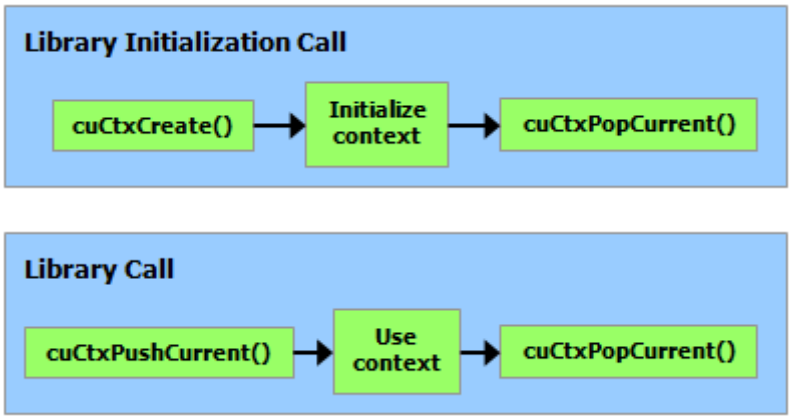


Figure 23: Library Context Management

3.3.2. Module

Modules are dynamically loadable packages of device code and data, akin to DLLs in Windows, that are output by nvcc (see *Compilation with NVCC*). The names for all symbols, including functions, global variables, and texture or surface references, are maintained at module scope so that modules written by independent third parties may interoperate in the same CUDA context.

This code sample loads a module and retrieves a handle to some kernel:

```
CUmodule cuModule;
cuModuleLoad(&cuModule, "myModule.ptx");
CUfunction myKernel;
cuModuleGetFunction(&myKernel, cuModule, "MyKernel");
```

This code sample compiles and loads a new module from PTX code and parses compilation errors:

```
#define BUFFER_SIZE 8192
CUmodule cuModule;
CUjit_option options[3];
void* values[3];
char* PTXCode = "some PTX code";
char error_log[BUFFER_SIZE];
int err;
options[0] = CU_JIT_ERROR_LOG_BUFFER;
values[0]  = (void*)error_log;
options[1] = CU_JIT_ERROR_LOG_BUFFER_SIZE_BYTES;
values[1]  = (void*)BUFFER_SIZE;
options[2] = CU_JIT_TARGET_FROM_CUCONTEXT;
values[2]  = 0;
err = cuModuleLoadDataEx(&cuModule, PTXCode, 3, options, values);
if (err != CUDA_SUCCESS)
    printf("Link error:\n%s\n", error_log);
```

This code sample compiles, links, and loads a new module from multiple PTX codes and parses link and compilation errors:

```
#define BUFFER_SIZE 8192
CUmodule cuModule;
CUjit_option options[6];
void* values[6];
float walltime;
char error_log[BUFFER_SIZE], info_log[BUFFER_SIZE];
char* PTXCode0 = "some PTX code";
char* PTXCode1 = "some other PTX code";
CUlinkState linkState;
int err;
void* cubin;
size_t cubinSize;
options[0] = CU_JIT_WALL_TIME;
values[0] = (void*)&walltime;
options[1] = CU_JIT_INFO_LOG_BUFFER;
values[1] = (void*)info_log;
options[2] = CU_JIT_INFO_LOG_BUFFER_SIZE_BYTES;
values[2] = (void*)BUFFER_SIZE;
```

(continues on next page)

(continued from previous page)

```
options[3] = CU_JIT_ERROR_LOG_BUFFER;
values[3] = (void*)error_log;
options[4] = CU_JIT_ERROR_LOG_BUFFER_SIZE_BYTES;
values[4] = (void*)BUFFER_SIZE;
options[5] = CU_JIT_LOG_VERBOSE;
values[5] = (void*)1;
cuLinkCreate(6, options, values, &linkState);
err = cuLinkAddData(linkState, CU_JIT_INPUT_PTX,
                    (void*)PTXCode0, strlen(PTXCode0) + 1, 0, 0, 0, 0);
if (err != CUDA_SUCCESS)
    printf("Link error:\n%s\n", error_log);
err = cuLinkAddData(linkState, CU_JIT_INPUT_PTX,
                    (void*)PTXCode1, strlen(PTXCode1) + 1, 0, 0, 0, 0);
if (err != CUDA_SUCCESS)
    printf("Link error:\n%s\n", error_log);
cuLinkComplete(linkState, &cubin, &cubinSize);
printf("Link completed in %fms. Linker Output:\n%s\n", walltime, info_log);
cuModuleLoadData(cuModule, cubin);
cuLinkDestroy(linkState);
```

It's possible to accelerate some parts of the module linking/loading process by using multiple threads, including when loading a cubin. This code sample uses `CU_JIT_BINARY_LOADER_THREAD_COUNT` to speed up module loading.

```
#define BUFFER_SIZE 8192
CUmodule cuModule;
CUjit_option options[3];
void* values[3];
char* cubinCode = "some cubin code";
char error_log[BUFFER_SIZE];
int err;
options[0] = CU_JIT_ERROR_LOG_BUFFER;
values[0]  = (void*)error_log;
options[1] = CU_JIT_ERROR_LOG_BUFFER_SIZE_BYTES;
values[1]  = (void*)BUFFER_SIZE;
options[2] = CU_JIT_BINARY_LOADER_THREAD_COUNT;
values[2]  = 0; // Use as many threads as CPUs on the machine
err = cuModuleLoadDataEx(&cuModule, cubinCode, 3, options, values);
if (err != CUDA_SUCCESS)
    printf("Link error:\n%s\n", error_log);
```

Full code can be found in the `ptxjit` CUDA sample.

3.3.3. Kernel Execution

`cuLaunchKernel()` launches a kernel with a given execution configuration.

Parameters are passed either as an array of pointers (next to last parameter of `cuLaunchKernel()`) where the nth pointer corresponds to the nth parameter and points to a region of memory from which the parameter is copied, or as one of the extra options (last parameter of `cuLaunchKernel()`).

When parameters are passed as an extra option (the `CU_LAUNCH_PARAM_BUFFER_POINTER` option), they are passed as a pointer to a single buffer where parameters are assumed to be properly offset

with respect to each other by matching the alignment requirement for each parameter type in device code.

Alignment requirements in device code for the built-in vector types are listed in Table 43. For all other basic types, the alignment requirement in device code matches the alignment requirement in host code and can therefore be obtained using `__alignof()`. The only exception is when the host compiler aligns `double` and `long long` (and `long` on a 64-bit system) on a one-word boundary instead of a two-word boundary (for example, using `gcc`'s compilation flag `-mno-align-double`) since in device code these types are always aligned on a two-word boundary.

`CUdeviceptr` is an integer, but represents a pointer, so its alignment requirement is `__alignof(void*)`.

The following code sample uses a macro (`ALIGN_UP()`) to adjust the offset of each parameter to meet its alignment requirement and another macro (`ADD_TO_PARAM_BUFFER()`) to add each parameter to the parameter buffer passed to the `CU_LAUNCH_PARAM_BUFFER_POINTER` option.

```
#define ALIGN_UP(offset, alignment) \
      (offset) = ((offset) + (alignment) - 1) & ~((alignment) - 1)

char paramBuffer[1024];
size_t paramBufferSize = 0;

#define ADD_TO_PARAM_BUFFER(value, alignment)                   \
    do {                                                        \
        paramBufferSize = ALIGN_UP(paramBufferSize, alignment); \
        memcpy(paramBuffer + paramBufferSize,                   \
               &(value), sizeof(value));                        \
        paramBufferSize += sizeof(value);                       \
    } while (0)

int i;
ADD_TO_PARAM_BUFFER(i, __alignof(i));
float4 f4;
ADD_TO_PARAM_BUFFER(f4, 16); // float4's alignment is 16
char c;
ADD_TO_PARAM_BUFFER(c, __alignof(c));
float f;
ADD_TO_PARAM_BUFFER(f, __alignof(f));
CUdeviceptr devPtr;
ADD_TO_PARAM_BUFFER(devPtr, __alignof(devPtr));
float2 f2;
ADD_TO_PARAM_BUFFER(f2, 8); // float2's alignment is 8

void* extra[] = {
    CU_LAUNCH_PARAM_BUFFER_POINTER, paramBuffer,
    CU_LAUNCH_PARAM_BUFFER_SIZE,    &paramBufferSize,
    CU_LAUNCH_PARAM_END
};
cuLaunchKernel(cuFunction,
               blockWidth, blockHeight, blockDepth,
               gridWidth, gridHeight, gridDepth,
               0, 0, 0, extra);
```

The alignment requirement of a structure is equal to the maximum of the alignment requirements of its fields. The alignment requirement of a structure that contains built-in vector types, `CUdeviceptr`,

or non-aligned `double` and `long long`, might therefore differ between device code and host code. Such a structure might also be padded differently. The following structure, for example, is not padded at all in host code, but it is padded in device code with 12 bytes after field `f` since the alignment requirement for field `f4` is 16.

```
typedef struct {
    float  f;
    float4 f4;
} myStruct;
```

3.3.4. Interoperability between Runtime and Driver APIs

An application can mix runtime API code with driver API code.

If a context is created and made current via the driver API, subsequent runtime calls will use this context instead of creating a new one.

If the runtime is initialized, `cuCtxGetCurrent()` can be used to retrieve the context created during initialization. This context can be used by subsequent driver API calls.

The implicitly created context from the runtime is called the primary context (see *Runtime Initialization*). It can be managed from the driver API with the Primary Context Management functions.

Device memory can be allocated and freed using either API. `CUdeviceptr` can be cast to regular pointers and vice-versa:

```
CUdeviceptr devPtr;
float* d_data;

// Allocation using driver API
cuMemAlloc(&devPtr, size);
d_data = (float*)devPtr;

// Allocation using runtime API
cudaMalloc(&d_data, size);
devPtr = (CUdeviceptr)d_data;
```

In particular, this means that applications written using the driver API can invoke libraries written using the runtime API (such as cuFFT, cuBLAS, ...).

All functions from the device and version management sections of the reference manual can be used interchangeably.

3.4. Programming Systems with Multiple GPUs

Multi-GPU programming allows an application to address problem sizes and achieve performance levels beyond what is possible with a single GPU by exploiting the larger aggregate arithmetic performance, memory capacity, and memory bandwidth provided by multi-GPU systems.

CUDA enables multi-GPU programming through host APIs, driver infrastructure, and supporting GPU hardware technologies:

- Host thread CUDA context management
- Unified memory addressing for all processors in the system

- Peer-to-peer bulk memory transfers between GPUs
- Fine-grained peer-to-peer GPU load/store memory access
- Higher level abstractions and supporting system software such as CUDA interprocess communication, parallel reductions using NCCL, and communication using NVLink and/or GPU-Direct RDMA with APIs such as NVSHMEM and MPI

At the most basic level, multi-GPU programming requires the application to manage multiple active CUDA contexts concurrently, distribute data to the GPUs, launch kernels on the GPUs to complete their work, and to communicate or collect the results so that they can be acted upon by the application. The details of how this is done differ depending on the most effective mapping of an application's algorithms, available parallelism, and existing code structure to a suitable multi-GPU programming approach. Some of the most common multi-GPU programming approaches include:

- A single host thread driving multiple GPUs
- Multiple host threads, each driving their own GPU
- Multiple single-threaded host processes, each driving their own GPU
- Multiple host processes containing multiple threads, each driving their own GPU
- Multi-node NVLink-connected clusters, with GPUs driven by threads and processes running within multiple operating system instances across the cluster nodes

GPUs can communicate with each other through memory transfers and peer accesses between device memories, covering each of the multi-device work distribution approaches listed above. High performance, low-latency GPU communications are supported by querying for and enabling the use of peer-to-peer GPU memory access, and leveraging NVLink to achieve high bandwidth transfers and finer-grained load/store operations between devices.

CUDA unified virtual addressing permits communication between multiple GPUs within the same host process with minimal additional steps to query and enable the use of high performance peer-to-peer memory access and transfers, e.g., via NVLink.

Communication between multiple GPUs managed by different host processes is supported through the use of interprocess communication (IPC) and Virtual memory Management (VMM) APIs. An introduction to high level IPC concepts and intra-node CUDA IPC APIs are discussed in the *Interprocess Communication* section. AdvancedVirtual Memory Management (VMM) APIs support both intra-node and multi-node IPC, are usable on both Linux and Windows operating systems, and allow per-allocation granularity control over IPC sharing of memory buffers as described in *Virtual Memory Management*.

CUDA itself provides the APIs needed to implement collective operations within a group of GPUs, potentially including the host, but it does not provide high level multi-GPU collective APIs itself. Multi-GPU collectives are provided by higher abstraction CUDA communication libraries such as NCCL and NVSHMEM.

3.4.1. Multi-Device Context and Execution Management

The first steps that are required to for an application to use multiple GPUs are to enumerate the available GPU devices, select among the available devices as appropriate based on their hardware properties, CPU affinity, and connectivity to peers, and to create CUDA contexts for each device that the application will use.

3.4.1.1 Device Enumeration

The following code sample shows how to query number of CUDA-enabled devices, enumerate each of the devices, and query their properties.

```
int deviceCount;
cudaGetDeviceCount(&deviceCount);
int device;
for (device = 0; device < deviceCount; ++device) {
    cudaDeviceProp deviceProp;
    cudaGetDeviceProperties(&deviceProp, device);
    printf("Device %d has compute capability %d.%d.\n",
           device, deviceProp.major, deviceProp.minor);
}
```

3.4.1.2 Device Selection

A host thread can set the device it is currently operating on at any time by calling `cudaSetDevice()`. Device memory allocations and kernel launches are made on the current device; streams and events are created in association with the currently set device. Until a call to `cudaSetDevice()` is made by the host thread, the current device defaults to device 0.

The following code sample illustrates how setting the current device affects subsequent memory allocation and kernel execution operations.

```
size_t size = 1024 * sizeof(float);
cudaSetDevice(0);            // Set device 0 as current
float* p0;
cudaMalloc(&p0, size);       // Allocate memory on device 0
MyKernel<<<1000, 128>>>(p0); // Launch kernel on device 0

cudaSetDevice(1);            // Set device 1 as current
float* p1;
cudaMalloc(&p1, size);       // Allocate memory on device 1
MyKernel<<<1000, 128>>>(p1); // Launch kernel on device 1
```

3.4.1.3 Multi-Device Stream, Event, and Memory Copy Behavior

A kernel launch will fail if it is issued to a stream that is not associated to the current device as illustrated in the following code sample.

```
cudaSetDevice(0);               // Set device 0 as current
cudaStream_t s0;
cudaStreamCreate(&s0);          // Create stream s0 on device 0
MyKernel<<<100, 64, 0, s0>>>(); // Launch kernel on device 0 in s0

cudaSetDevice(1);               // Set device 1 as current
cudaStream_t s1;
cudaStreamCreate(&s1);          // Create stream s1 on device 1
MyKernel<<<100, 64, 0, s1>>>(); // Launch kernel on device 1 in s1

// This kernel launch will fail, since stream s0 is not associated to device 1:
MyKernel<<<100, 64, 0, s0>>>(); // Launch kernel on device 1 in s0
```

A memory copy will succeed even if it is issued to a stream that is not associated to the current device.

`cudaEventRecord()` will fail if the input event and input stream are associated to different devices.

`cudaEventElapsedTime()` will fail if the two input events are associated to different devices.

`cudaEventSynchronize()` and `cudaEventQuery()` will succeed even if the input event is associated to a device that is different from the current device.

`cudaStreamWaitEvent()` will succeed even if the input stream and input event are associated to different devices. `cudaStreamWaitEvent()` can therefore be used to synchronize multiple devices with each other.

Each device has its own *default stream*, so commands issued to the default stream of a device may execute out of order or concurrently with respect to commands issued to the default stream of any other device.

3.4.2. Multi-Device Peer-to-Peer Transfers and Memory Access

3.4.2.1 Peer-to-Peer Memory Transfers

CUDA can perform memory transfers between devices and will take advantage of dedicated copy engines and NVLink hardware to maximize performance when peer-to-peer memory access is possible.

`cudaMemcpy` can be used with the copy type `cudaMemcpyDeviceToDevice` or `cudaMemcpyDefault`.

Otherwise, copies must be performed using `cudaMemcpyPeer()`, `cudaMemcpyPeerAsync()`, `cudaMemcpy3DPeer()`, or `cudaMemcpy3DPeerAsync()` as illustrated in the following code sample.

```
cudaSetDevice(0);                   // Set device 0 as current
float* p0;
size_t size = 1024 * sizeof(float);
cudaMalloc(&p0, size);              // Allocate memory on device 0

cudaSetDevice(1);                   // Set device 1 as current
float* p1;
cudaMalloc(&p1, size);              // Allocate memory on device 1

cudaSetDevice(0);                   // Set device 0 as current
MyKernel<<<1000, 128>>>(p0);        // Launch kernel on device 0

cudaSetDevice(1);                   // Set device 1 as current
cudaMemcpyPeer(p1, 1, p0, 0, size); // Copy p0 to p1
MyKernel<<<1000, 128>>>(p1);        // Launch kernel on device 1
```

A copy (in the implicit *NULL* stream) between the memories of two different devices:

- does not start until all commands previously issued to either device have completed and
- runs to completion before any commands (see *Asynchronous Execution*) issued after the copy to either device can start.

Consistent with the normal behavior of streams, an asynchronous copy between the memories of two devices may overlap with copies or kernels in another stream.

If peer-to-peer access is enabled between two devices, e.g., as described in *Peer-to-Peer Memory Access*, peer-to-peer memory copies between these two devices no longer need to be staged through the host and are therefore faster.

3.4.2.2 Peer-to-Peer Memory Access

Depending on the system properties, specifically the PCIe and/or NVLink topology, devices are able to address each other's memory (i.e., a kernel executing on one device can dereference a pointer to the memory of the other device). Peer-to-peer memory access is supported between two devices if `cudaDeviceCanAccessPeer()` returns true for the specified devices.

Peer-to-peer memory access must be enabled between two devices by calling `cudaDeviceEnablePeerAccess()` as illustrated in the following code sample. On non-NVSwitch enabled systems, each device can support a system-wide maximum of eight peer connections.

A unified virtual address space is used for both devices (see *Unified Virtual Address Space*), so the same pointer can be used to address memory from both devices as shown in the code sample below.

```
cudaSetDevice(0);                   // Set device 0 as current
float* p0;
size_t size = 1024 * sizeof(float);
cudaMalloc(&p0, size);              // Allocate memory on device 0
MyKernel<<<1000, 128>>>(p0);        // Launch kernel on device 0

cudaSetDevice(1);                   // Set device 1 as current
cudaDeviceEnablePeerAccess(0, 0);   // Enable peer-to-peer access
                                    // with device 0

// Launch kernel on device 1
// This kernel launch can access memory on device 0 at address p0
MyKernel<<<1000, 128>>>(p0);
```

Note

The use of `cudaDeviceEnablePeerAccess()` to enable peer memory access operates globally on all previous and subsequent GPU memory allocations on the peer device. Enabling peer access to a device via `cudaDeviceEnablePeerAccess()` adds runtime cost to device memory allocation operations on that peer due to the need make the allocations immediately accessible to the current device and any other peers that also have access, adding multiplicative overhead that scales with the number of peer devices.

A more scalable alternative to enabling peer memory access for all device memory allocations is to make use of CUDA Virtual Memory Management APIs to explicitly allocate peer-accessible memory regions only as-needed, at allocation time. By requesting peer-accessibility explicitly during memory allocation, the runtime cost of memory allocations are unharmed for allocations not accessible to peers, and peer-accessible data structures are correctly scoped for improved software debugging and reliability (see ref::*virtual-memory-management*).

3.4.2.3 Peer-to-Peer Memory Consistency

Synchronization operations must be used to enforce the ordering and correctness of memory accesses by concurrently executing threads in grids distributed across multiple devices. Threads synchronizing across devices operate at the `thread_scope_system` synchronization scope. Similarly, memory operations fall within the `thread_scope_system` memory synchronization domain.

CUDA ref::*atomic-functions* can perform read-modify-write operations on an object in peer device memory when only a single GPU is accessing that object. The requirements and limitations for peer atomicity are described in the CUDA memory model atomicity requirements discussion.

3.4.2.4 Multi-Device Managed Memory

Managed memory can be used on multi-GPU systems with peer-to-peer support. The detailed requirements for concurrent multi-device managed memory access and APIs for GPU-exclusive access to managed memory are described in *Multi-GPU*.

3.4.2.5 Host IOMMU Hardware, PCI Access Control Services, and VMs

On Linux specifically, CUDA and the display driver do not support IOMMU-enabled bare-metal PCIe peer-to-peer memory transfer. However, CUDA and the display driver do support IOMMU via virtual machine pass through. The IOMMU must be disabled when running Linux on a bare metal system to prevent silent device memory corruption. Conversely, the IOMMU should be enabled and the VFIO driver be used for PCIe pass through for virtual machines.

On Windows the IOMMU limitation above does not exist.

See also Allocating DMA Buffers on 64-bit Platforms.

Additionally, PCI Access Control Services (ACS) can be enabled on systems that support IOMMU. The PCI ACS feature redirects all PCI point-to-point traffic through the CPU root complex, which can cause significant performance loss due to the reduction in overall bisection bandwidth.

3.5. A Tour of CUDA Features

Sections 1-3 of this programming guide have introduced CUDA and GPU programming, covering foundational topics both conceptually and in simple code examples. The sections describing specific CUDA features in part 4 of this guide assume knowledge of the concepts covered in sections 1-3 of this guide.

CUDA has many features which apply to different problems. Not all of them will be applicable to every use case. This chapter serves to introduce each of these features and describe its intended use and the problems it may help solve. Features are coarsely sorted into categories by the type of problem they are intended to solve. Some features, such as CUDA graphs, could fit into more than one category.

Section 4 covers these CUDA features in more complete detail.

3.5.1. Improving Kernel Performance

The features outlined in this section are all intended to aid kernel developers to maximize the performance of their kernels.

3.5.1.1 Asynchronous Barriers

Asynchronous barriers were introduced in Section 3.2.4.2 and allow for more nuanced control over synchronization between threads. Asynchronous barriers separate the arrival and the wait of a barrier. This allows applications to perform work that does not depend on the barrier while waiting for other threads to arrive. Asynchronous barriers can be specified for different *thread scopes*. Full details of asynchronous barriers are found in Section 4.9.

3.5.1.2 Asynchronous Data Copies and the Tensor Memory Accelerator (TMA)

Asynchronous data copies in the context of CUDA kernel code refers to the ability to move data between shared memory and GPU DRAM while still carrying out computations. This should not be confused with asynchronous memory copies between the CPU and GPU. This feature makes use of asynchronous barriers. Section 4.11 covers the use of asynchronous copies in detail.

3.5.1.3 Pipelines

Pipelines are a mechanism for staging work and coordinating multi-buffer producer–consumer patterns, commonly used to overlap compute with *asynchronous data copies*. Section 4.10 has details and examples of using pipelines in CUDA.

3.5.1.4 Work Stealing with Cluster Launch Control

Work stealing is a technique for maintaining utilization in uneven workloads where workers that have completed their work can 'steal' tasks from other workers. Cluster launch control, a feature introduced in compute capability 10.0 (Blackwell), gives kernels direct control over in-flight block scheduling so they can adapt to uneven workloads in real time. A thread block can cancel the launch of another thread block or cluster that has not yet started, claim its index, and immediately begin executing the stolen work. This work-stealing flow keeps SMs busy and cuts idle time under irregular data or runtime variation—delivering finer-grained load balancing without relying on the hardware scheduler alone.

Section 4.12 provides details on how to use this feature.

3.5.2. Improving Latencies

The features outlined in this section share a common theme of aiming to reduce some type of latency, though the type of latency being addressed differs between the different features. By and large they are focused on latencies at the kernel launch level or higher. GPU memory access latency within a kernel is not one of the latencies considered here.

3.5.2.1 Green Contexts

Green contexts, also called *execution contexts*, is the name given to a CUDA feature which enables a program to create *CUDA contexts* which will execute work only on a subset of the SMs of a GPU. By default, the thread blocks of a kernel launch are dispatched to any SM within the GPU which can fulfill the resource requirements of the kernel. There are a large number of factors which can affect which SMs can execute a thread block, including but not necessarily limited to: shared memory use, register use, use of clusters, and total number of threads in the thread block.

Execution contexts allow a kernel to be launched into a specially created context which further limits the number of SMs available to execute the kernel. Importantly, when a program creates a green context which uses some set of SMs, other contexts on the GPU will not schedule thread blocks onto the SMs allocated to the green context. This includes the primary context, which is the default context used by the CUDA runtime. This allows these SMs to be reserved for workloads which are high priority or latency-sensitive.

Section 4.6 gives full details on the use of green contexts. Green contexts are available in the CUDA runtime in CUDA 13.1 and later.

3.5.2.2 Stream-Ordered Memory Allocation

The *stream-ordered memory allocator* allows programs to sequence allocation and freeing of GPU memory into a *CUDA stream*. Unlike `cudaMalloc` and `cudaFree` which execute immediately, `cudaMallocAsync` and `cudaFreeAsync` inserts a memory allocation or free operation into a CUDA stream. Section 4.3 covers all the details of these APIs.

3.5.2.3 CUDA Graphs

CUDA graphs enable an application to specify a sequence of CUDA operations such as kernel launches or memory copies and the dependencies between these operations so that they can be executed efficiently on the GPU. Similar behavior can be attained by using *CUDA streams*, and indeed one of the mechanisms for creating a graph is called *stream capture*, which enables the operations on a stream to be recorded into a CUDA graph. Graphs can also be created using the *CUDA graphs API*.

Once a graph has been created, it can be instantiated and executed many times. This is useful for specifying workloads that will be repeated. Graphs offer some performance benefits in reducing CPU launch costs associated with invoking CUDA operations as well as enabling optimizations only available when the whole workload is specified in advance.

Section 4.2 describes and demonstrates how to use CUDA graphs.

3.5.2.4 Programmatic Dependent Launch

Programmatic dependent launch is a CUDA feature which allows a dependent kernel, i.e. a kernel which depends on the output of a prior kernel, to begin execution before the primary kernel on which it depends has completed. The dependent kernel can execute setup code and unrelated work up until it requires data from the primary kernel and block there. The primary kernel can signal when the data required by the dependent kernel is ready, which will release the dependent kernel to continue executing. This enables some overlap between the kernels which can help keep GPU utilization high while minimizing the latency of the critical data path. Section 4.5 covers programmatic dependent launch.

3.5.2.5 Lazy Loading

Lazy loading is a feature which allows control over how the JIT compiler operates at application startup. Applications which have many kernels which need to be JIT compiled from PTX to cubin may experience long startup times if all kernels are JIT compiled as part of application startup. The default behavior is that modules are not compiled until they are needed. This can be changed by the use of *environment variables*, as detailed in Section 4.7.

3.5.3. Functionality Features

The features described here share a common trait that they are meant to enable additional capabilities or functionality.

3.5.3.1 Extended GPU Memory

Extended GPU memory is a feature available in NVLink-C2C connected systems that enables efficient access to all memory within the system from within a GPU. EGM is covered in detail in Section 4.17.

3.5.3.2 Dynamic Parallelism

CUDA applications most commonly launch kernels from code running on the CPU. It is also possible to create new kernel invocations from a kernel running on the GPU. This feature is referred to as *CUDA dynamic parallelism*. Section 4.18 covers the details of creating new GPU kernel launches from code running on the GPU.

3.5.4. CUDA Interoperability

3.5.4.1 CUDA Interoperability with other APIs

There are other mechanisms than CUDA for running code on GPUs. The application GPUs were originally built to accelerate, computer graphics, uses its own set of APIs such as Direct3D and Vulkan. Applications may wish to use one of the graphics APIs for 3D rendering while performing computations in CUDA. CUDA provides mechanisms for exchanging data stored on the GPU between the CUDA contexts and the GPU contexts used by the 3D APIs. For example, an application may perform a simulation using CUDA, and then use a 3D API to create visualizations of the results. This is achieved by making some buffers readable and/or writeable from both CUDA and the graphics API.

The same mechanisms which allow sharing of buffers with graphics APIs are also used to share buffers with communications mechanisms which can enable rapid, direct GPU-to-GPU communication within multi-node environments.

Section 4.19 describes how CUDA interoperates with other GPU APIs and how to share data between CUDA and other APIs, providing specific examples for a number of different APIs.

3.5.4.2 Interprocess Communication

For very large computations, it is common to use multiple GPUs together to make use of more memory and more compute resources working together on a problem. Within a single system, or node in cluster computing terminology, multiple GPUs can be used in a single host process. This is described in Section 3.4.

It is also common to use separate host processes spanning either a single computer or multiple computers. When multiple processes are working together, communication between them is known as interprocess communication. CUDA interprocess communication (CUDA IPC) provides mechanisms to share GPU buffers between different processes. Section 4.15 explains and demonstrates how CUDA IPC can be used to coordinate and communicate between different host processes.

3.5.5. Fine-Grained Control

3.5.5.1 Virtual Memory Management

As mentioned in Section 2.6.1, all GPUs in a system, along with the CPU memory, share a single unified virtual address space. Most applications can use the default memory management provided by CUDA without the need to change its behavior. However, *the CUDA driver API* provides advanced and detailed controls over the layout of this virtual memory space for those that need it. This is mostly applicable for controlling the behavior of buffers when sharing between GPUs both within and across multiple systems.

Section 4.16 covers the controls offered by the CUDA driver API, how they work and when a developer may find them advantageous.

3.5.5.2 Driver Entry Point Access

Driver entry point access refers to the ability, starting in CUDA 11.3, to retrieve function pointers to the CUDA Driver and CUDA Runtime APIs. It also allows developers to retrieve function pointers for specific variants of driver functions, and to access driver functions from drivers newer than those available in the CUDA toolkit. Section 4.20 covers driver entry point access.

3.5.5.3 Error Log Management

Error log management provides utilities for handling and logging errors from CUDA APIs. Setting a single environment variable `CUDA_LOG_FILE` enables capturing CUDA errors directly to stderr, stdout, or a file. Error log management also enables applications to register a callback which is triggered when CUDA encounters an error. Section 4.8 provides more details on error log management.

Chapter 4. CUDA Features

4.1. Unified Memory

This section explains the detailed behavior and use of each of the different paradigms of unified memory available. *The earlier section on unified memory* showed how to determine which unified memory paradigm applies and briefly introduced each.

As discussed previously there are four paradigms of unified memory programming:

- *Full support for explicit managed memory allocations*
- *Full support for all allocations with software coherence*
- *Full support for all allocations with hardware coherence*
- *Limited unified memory support*

The first three paradigms involving full unified memory support have very similar behavior and programming model and are covered in *Unified Memory on Devices with Full CUDA Unified Memory Support* with any differences highlighted.

The last paradigm, where unified memory support is limited, is discussed in detail in *Unified Memory on Windows, WSL, and Tegra*.

4.1.1. Unified Memory on Devices with Full CUDA Unified Memory Support

These systems include hardware-coherent memory systems, such as NVIDIA Grace Hopper and modern Linux systems with Heterogeneous Memory Management (HMM) enabled. HMM is a software-based memory management system, providing the same programming model as hardware-coherent memory systems.

Linux HMM requires Linux kernel version 6.1.24+, 6.2.11+ or 6.3+, devices with compute capability 7.5 or higher and a CUDA driver version 535+ installed with Open Kernel Modules.

Note

We refer to systems with a combined page table for both CPUs and GPUs as *hardware coherent* systems. Systems with separate page tables for CPUs and GPUs are referred to as *software-coherent*.

Hardware-coherent systems such as NVIDIA Grace Hopper offer a logically combined page table for both CPUs and GPUs, see *CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency*. The

following section only applies to hardware-coherent systems:

- *Access Counter Migration*

4.1.1.1 Unified Memory: In-Depth Examples

Systems with full CUDA unified memory support, see table *Overview of Unified Memory Paradigms*, allow the device to access any memory owned by the host process interacting with the device.

This section shows a few advanced use-cases, using a kernel that simply prints the first 8 characters of an input character array to the standard output stream:

```
__global__ void kernel(const char* type, const char* data) {
  static const int n_char = 8;
  printf("%s - first %d characters: '", type, n_char);
  for (int i = 0; i < n_char; ++i) printf("%c", data[i]);
  printf("'\n");
}
```

The following tabs show various ways of how this kernel may be called with system-allocated memory:

Malloc

```
void test_malloc() {
  const char test_string[] = "Hello World";
  char* heap_data = (char*)malloc(sizeof(test_string));
  strncpy(heap_data, test_string, sizeof(test_string));
  kernel<<<1, 1>>>("malloc", heap_data);
  ASSERT(cudaDeviceSynchronize() == cudaSuccess,
    "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
  free(heap_data);
}
```

Managed

```
void test_managed() {
  const char test_string[] = "Hello World";
  char* data;
  cudaMallocManaged(&data, sizeof(test_string));
  strncpy(data, test_string, sizeof(test_string));
  kernel<<<1, 1>>>("managed", data);
  ASSERT(cudaDeviceSynchronize() == cudaSuccess,
    "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
  cudaFree(data);
}
```

Stack variable

```
void test_stack() {
  const char test_string[] = "Hello World";
  kernel<<<1, 1>>>("stack", test_string);
  ASSERT(cudaDeviceSynchronize() == cudaSuccess,
```

(continues on next page)

(continued from previous page)

```
    "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
}
```

File-scope static variable

```
void test_static() {
  static const char test_string[] = "Hello World";
  kernel<<<1, 1>>>("static", test_string);
  ASSERT(cudaDeviceSynchronize() == cudaSuccess,
    "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
}
```

Global-scope variable

```
const char global_string[] = "Hello World";

void test_global() {
  kernel<<<1, 1>>>("global", global_string);
  ASSERT(cudaDeviceSynchronize() == cudaSuccess,
    "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
}
```

Global-scope extern variable

```
// declared in separate file, see below
extern char* ext_data;

void test_extern() {
  kernel<<<1, 1>>>("extern", ext_data);
  ASSERT(cudaDeviceSynchronize() == cudaSuccess,
    "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
}
```

```
/** This may be a non-CUDA file */
char* ext_data;
static const char global_string[] = "Hello World";

void __attribute__ ((constructor)) setup(void) {
  ext_data = (char*)malloc(sizeof(global_string));
  strncpy(ext_data, global_string, sizeof(global_string));
}

void __attribute__ ((destructor)) tear_down(void) {
  free(ext_data);
}
```

Note that for the extern variable, it could be declared and its memory owned and managed by a third-party library, which does not interact with CUDA at all.

Also note that stack variables as well as file-scope and global-scope variables can only be accessed through a pointer by the GPU. In this specific example, this is convenient because the character array

is already declared as a pointer: `const char*`. However, consider the following example with a global-scope integer:

```
// this variable is declared at global scope
int global_variable;

__global__ void kernel_uncompilable() {
  // this causes a compilation error: global (__host__) variables must not
  // be accessed from __device__ / __global__ code
  printf("%d\n", global_variable);
}

// On systems with pageableMemoryAccess set to 1, we can access the address
// of a global variable. The below kernel takes that address as an argument
__global__ void kernel(int* global_variable_addr) {
  printf("%d\n", *global_variable_addr);
}
int main() {
  kernel<<<1, 1>>>(&global_variable);
  ...
  return 0;
}
```

In the example above, we need to ensure to pass a *pointer* to the global variable to the kernel instead of directly accessing the global variable in the kernel. This is because global variables without the `__managed__` specifier are declared as `__host__`-only by default, thus most compilers won't allow using these variables directly in device code as of now.

4.1.1.1.1 File-backed Unified Memory

Since systems with full CUDA unified memory support allow the device to access any memory owned by the host process, they can directly access file-backed memory.

Here, we show a modified version of the initial example shown in the previous section to use file-backed memory in order to print a string from the GPU, read directly from an input file. In the following example, the memory is backed by a physical file, but the example applies to memory-backed files too.

```
__global__ void kernel(const char* type, const char* data) {
  static const int n_char = 8;
  printf("%s - first %d characters: '", type, n_char);
  for (int i = 0; i < n_char; ++i) printf("%c", data[i]);
  printf("'\n");
}
```

```
void test_file_backed() {
  int fd = open(INPUT_FILE_NAME, O_RDONLY);
  ASSERT(fd >= 0, "Invalid file handle");
  struct stat file_stat;
  int status = fstat(fd, &file_stat);
  ASSERT(status >= 0, "Invalid file stats");
  char* mapped = (char*)mmap(0, file_stat.st_size, PROT_READ, MAP_PRIVATE, fd,
↪ 0);
  ASSERT(mapped != MAP_FAILED, "Cannot map file into memory");
  kernel<<<1, 1>>>("file-backed", mapped);
```

(continues on next page)